

```

cell.fill = config.FILL_HDR
cell = ws.cell(row=2, column=col_idx)
for col_idx in range(1, len(config.HEADERS) + 1):
ws.append(config.HEADERS)
if ws.max_row == 1 and ws.cell(row=1, column=1).value is None:

ws.oddHeader.right.color = "7A7A7A"
ws.oddHeader.right.size = 9
ws.oddHeader.right.text = f".....: {now_str}"
now_str = datetime.datetime.now().strftime("%d.%m.%Y %H:%M")

ws.sheet_properties.outlinePr.summaryBelow = False
ws.freeze_panes = "A3"
def build_structure(ws, mock_data=None, saved_statuses=None, saved_sums=None):

..... .. Cmd+S!")
print(" [....] ..... LibreOffice ....., ....., ... .. builder.py
{width_val}")
print(f" - ..... {letters[col_idx]} (..... {col_idx}): ..... =
width_val = 14.0
for col_idx in columns_to_check:
print(" ..... openpyxl:")
letters = {4: "D", 5: "E", 6: "F", 7: "G", 8: "H"}
columns_to_check = [4, 5, 6, 7, 8] # D, E, F, G, H
def test_columns_width_logic():

excel_styler.apply_row_style(ws, row_idx, font, fill, border, alignment)
def apply_row_style(ws, row_idx, font, fill, border, alignment=None):

import excel_styler
import db_viewer
import db_core
import config
import datetime
import os
import json
import openpyxl

logging.getLogger().setLevel(logging.WARNING)
import logging
=== FILE: ./builder.py ===

=====
Files: 19
Generated: 2026-07-09T14:09:15.447631
Root: /Users/test/Desktop/cont_ks2_2
=== PROJECT CODE SNAPSHOT ===
=== FILE: ./ai_project_snapshot.txt ===

=====
Files: 20
Generated: 2026-07-09T14:09:54.464369
Root: /Users/test/Desktop/cont_ks2_2
=== PROJECT CODE SNAPSHOT ===
=== FILE: ./ai_project_snapshot_test.txt ===

=====
Files: 21
Generated: 2026-07-09T14:17:37.071368
Root: /Users/test/Desktop/cont_ks2_2
=== PROJECT CODE SNAPSHOT ===

```

```

config.THIN_BORDER,
config.FILL_MTH,
config.FONT_MTH,
current_row,
ws,
excel_styler.apply_row_style(
current_row = ws.max_row
)
[row_counter, mth, float(mth_data.get("sum", 0.0)), "", "", "", "", "", "", ""]
ws.append(
# .....: ..... 2

status_raw = mth_data.get("status", "")
mth_data = db[direction][sub_obj][mth]
for mth in available_months:

available_months.append(m)
if m not in available_months:
for m in raw_months:

available_months = [m for m in db_core.ALL_YEAR_MONTHS if m in raw_months]
raw_months = list(db[direction][sub_obj].keys())

ws.row_dimensions[ws.max_row].outline_level = 1
)
config.ALIGN_L,
config.THIN_BORDER,
config.FILL_OBJ,
config.FONT_OBJ,
ws.max_row,
ws,
excel_styler.apply_row_style(
ws.append(["", str(sub_obj), "", "", "", "", "", "", "", ""])
for sub_obj in db[direction].keys():

ws.row_dimensions[ws.max_row].outline_level = 0
)
config.ALIGN_L,
config.THIN_BORDER,
config.FILL_DIR,
config.FONT_DIR,
ws.max_row,
ws,
excel_styler.apply_row_style(
ws.append(["", str(direction), "", "", "", "", "", "", "", ""])
continue
if direction == "_meta":
for direction in db.keys():
row_counter = 1

is_new = meta.get("is_new_change", False)
target_sub = meta.get("last_changed_sub")
target_dir = meta.get("last_changed_dir")
meta = db.get("_meta", {}) if isinstance(db, dict) else {}
# .... 1: ..... .. .....

db = json.load(f)
with open(json_path, "r", encoding="utf-8") as f:
return ws.max_row
if not os.path.exists(json_path):
json_path = "database.json"

cell.border = config.THIN_BORDER
cell.font = config.FONT_HDR

```

```

excel_styler.format_status_cell(cell, clean_status_value)
if clean_status_value is not None:
else:
cell.value = ""
cell.fill = config.FILL_BLOCKED
if mask.get(col_name) == 0:

clean_status_value = doc_status_raw if col_name == "...." else None
else:
)
else doc_status_raw.get("value")
if col_name in doc_status_raw
doc_status_raw.get(col_name, {}).get("value")
clean_status_value = (
if isinstance(doc_status_raw, dict):

status_date = doc_status_raw.get("date", "") if isinstance(doc_status_raw, dict) else ""

doc_status_raw = status_raw
else:
doc_status_raw = status_raw[d_name]
if isinstance(status_raw, dict) and d_name in status_raw:
# .....: ...../...

cell = ws.cell(row=doc_row, column=col_idx)

col_idx = int(col_letter_or_idx)
else:
col_idx = openpyxl.utils.column_index_from_string(col_letter_or_idx)
if isinstance(col_letter_or_idx, str):
for col_name, col_letter_or_idx in config.COLUMN_MAPPING.items():

)
{"....": 1, "....": 1, ".....": 1, "1 .... .": 1, "1 .... .": 1, "....": 1},
d_name,
mask = config.DOCUMENT_ROLES.get(

ws.row_dimensions[doc_row].outline_level = 3
ws.cell(row=doc_row, column=3).number_format = "#,##0.00"
)
config.ALIGN_L,
config.THIN_BORDER,
None,
config.FONT_DATA,
doc_row,
ws,
excel_styler.apply_row_style(
doc_row = ws.max_row
ws.append(["", f". {d_name}", "", "", "", "", "", "", "", ""])
for d_name in config.DOCUMENTS_LIST:
# .....: ..... 3

ws.row_dimensions[current_row].hidden = hidden_default
hidden_default = False
if is_new and direction == target_dir and sub_obj == target_sub:
hidden_default = True
# ..... (.... 2)

row_counter += 1
ws.row_dimensions[current_row].outline_level = 2
ws.cell(row=current_row, column=3).number_format = "#,##0.00"
ws.cell(row=current_row, column=1).alignment = config.ALIGN_C
)
config.ALIGN_L,

```

```

elif col_idx == 3:
ws.column_dimensions[col_letter].width = max(max_len + 5, 45)
if col_idx == 2:

max_len = cell_len
if cell_len > max_len:
cell_len = max(len(sub_line) for sub_line in str(cell.value).split("\n"))
if cell.value is not None:
for cell in col:
max_len = 0
else:
ws.column_dimensions[col_letter].auto_fit = False
ws.column_dimensions[col_letter].width = 8.0
# ..... 1.35 ..
if col_letter in status_letters:

col_idx = first_cell.column
col_letter = first_cell.column_letter
first_cell = col[0]
for col in ws.columns:
status_letters = ["D", "E", "F", "G", "H"]
# .....

ws.row_dimensions[row_idx].hidden = True
if ws.row_dimensions[row_idx].outline_level in (3, 4):
for row_idx in range(2, ws.max_row + 1):
ws.sheet_view.showOutlineSymbols = True
# .....

ws.cell(row=row, column=10).value = f"=IF(C{row}>0, \".....\", \"\")"
if cell_val and not str(cell_val).startswith(".") and not str(cell_val).startswith(" "):
cell_val = ws.cell(row=row, column=2).value
for row in range(2, ws.max_row + 1):
# .....

ws.row_dimensions[log_row].outline_level = 4
)
config.ALIGN_L,
config.THIN_BORDER,
None,
excel_styler.FONT_LOG_DATE,
log_row,
ws,
excel_styler.apply_row_style(
log_row = ws.max_row
)
]
"""
"""
"""
"""
"""
"""
"""
"""
"""
"""
"""
f" .. ..: {status_date}",
"""
[
ws.append(
if mask.get("....", 1) == 1 and status_date:
# .....: ..... 4

cell.fill = openpyxl.styles.PatternFill(fill_type=None)
else:

```

```

from openpyxl.styles import Font, Alignment, PatternFill, Border, Side
import os
=== FILE: ./config.py ===

```

```

}
}
"sum": 0.0
"month": "....",
"sub_obj": ".....",
"direction": "02_.....",
"02_.....||.....": {
},
"sum": 850000.0
"month": ".....",
"sub_obj": ".....",
"direction": "02_.....",
"02_.....||.....": {
},
"sum": 0.0
"month": "....",
"sub_obj": "04_537_..",
"direction": "02_1505_..",
"02_1505_..||04_537_..||.....": {
},
"sum": 0.0
"month": ".....",
"sub_obj": "04_537_..",
"direction": "02_1505_..",
"02_1505_..||04_537_..||.....": {
},
"sum": 0.0
"month": "....",
"sub_obj": "03_538_..",
"direction": "02_1505_..",
"02_1505_..||03_538_..||.....": {
},
"sum": 0.0
"month": ".....",
"sub_obj": "03_538_..",
"direction": "02_1505_..",
"02_1505_..||03_538_..||.....": {
},
"sum": 13.0
"month": "....",
"sub_obj": ".. .... : 02.07",
"direction": "01_271_..",
"01_271_..||.. .... : 02.07||.....": {
{
=== FILE: ./changes.json ===

```

```

return ws.max_row

```

```

db_core.save_db(db)
db["_meta"]["is_new_change"] = False
if isinstance(db, dict) and "_meta" in db:
# .... 3: ..... (.....)

```

```

ws.column_dimensions[col_letter].width = max(max_len + 4, 12)
else:
ws.column_dimensions[col_letter].width = max(max_len + 4, 18)

```

```
DOCUMENT_ROLES = {
```

```
ROLE_EXEC_DOC = {".....": 1, "...": 0, ".....": 0, "1 .... .": 1, "1 .... .": 1, ".....": 0}
ROLE_INVOICE = {".....": 0, "...": 1, ".....": 0, "1 .... .": 1, "1 .... .": 1, ".....": 0}
ROLE_KS3 = {".....": 0, "...": 1, ".....": 1, "1 .... .": 1, "1 .... .": 1, ".....": 0}
ROLE_KS2 = {".....": 1, "...": 1, ".....": 1, "1 .... .": 1, "1 .... .": 1, ".....": 0}
# 1 - ..... (.....), 0 - ..... (.....)
# === ..... (.....) ===
```

```
]
".....",
"....",
"1 .... .",
"1 .... .",
".....",
"....",
"....",
"..... (.....)",
"..... / ..... / .....",
". ./.",
HEADERS = [
```

```
ALIGN_R = Alignment(horizontal="right", vertical="center")
ALIGN_L = Alignment(horizontal="left", vertical="center", wrap_text=True)
ALIGN_C = Alignment(horizontal="center", vertical="center", wrap_text=True)
```

```
)
bottom=Side(style="thin", color="B0B0B0"),
top=Side(style="thin", color="B0B0B0"),
right=Side(style="thin", color="B0B0B0"),
left=Side(style="thin", color="B0B0B0"),
THIN_BORDER = Border(
```

```
FILL_MTH = PatternFill(start_color="EAEAEA", end_color="EAEAEA", fill_type="solid")
FILL_OBJ = PatternFill(start_color="D9D9D9", end_color="D9D9D9", fill_type="solid")
FILL_HDR = FILL_DIR = PatternFill(start_color="1F4E78", end_color="1F4E78", fill_type="solid")
```

```
FONT_DATA = Font(name="Calibri", size=11)
FONT_MTH = Font(name="Calibri", size=11, bold=True, italic=True, color="000000")
FONT_OBJ = Font(name="Calibri", size=11, bold=True, color="000000")
FONT_DIR = Font(name="Calibri", size=12, bold=True, color="FFFFFF")
FONT_HDR = Font(name="Calibri", size=11, bold=True, color="FFFFFF")
```

```
FULL_PATH = os.path.join(FOLDER_NAME, FILE_NAME)
FILE_NAME = "....._..._.xlsx"
FOLDER_NAME = ""
```

```
.....']
DOCUMENTS_LIST = ['... -2', '..... -3', '.....', '.....', '.....']
MONTHS_LIST = ['.....', '.....', '.....', '.....', '.....', '.....', '.....']
```

```
]
('02_.....', '.....', '.....', 0, None),
('02_.....', '.....', '.....', 850000, 2),
('01_..... (.....)', '04_537_', '.....', 0, None),
('01_..... (.....)', '04_537_', '.....', 0, None),
('01_..... (.....)', '03_538_', '.....', 0, None),
('01_..... (.....)', '03_538_', '.....', 0, None),
('01_..... (.....)', '02_1505_', '.....', 0, None),
('01_..... (.....)', '02_1505_', '.....', 450000, 3),
('01_..... (.....)', '01_271_', '.....', 0, None),
('01_..... (.....)', '01_271_', '.....', 450000, 3),
DATA_ROWS = [
```

```

FILL_SDO_RED = FILL_RED
FILL_SDO_YELLOW = FILL_YELLOW
FILL_SDO_GREEN = FILL_GREEN
# .....
# === ..... (.....) ===

FILL_RED = PatternFill(start_color="FFC7CE", end_color="FFC7CE", fill_type="solid")
FILL_YELLOW = PatternFill(start_color="FFEB9C", end_color="FFEB9C", fill_type="solid")
FILL_GREEN = PatternFill(start_color="C6EFCE", end_color="C6EFCE", fill_type="solid")
# .....

VERSION = "v2. ...."

}
"1 .... .": ROLE_CUSTOMER,
"1 .... .": ROLE_CUSTOMER,
".....": ROLE_GEN_DIR,
"...": ROLE_KS2, # ... .. ROLE_KS2/ROLE_KS3 ..
ROLES = {

}
".....": 1,
"....": 1,
".....": 1,
"..... -3": 1,
"... -2": 1,
ROLE_CUSTOMER = {
# .....

}
".....": 0,
"....": 1,
".....": 0,
"..... -3": 1,
"... -2": 1,
ROLE_GEN_DIR = {
# ..... "....." .....
# ..... ROLE_KS2/ROLE_KS3/ROLE_INVOICE/ROLE_EXEC_DOC.
# === ..... ===

}
"1 .... .": "H",
"1 .... .": "G",
".....": "F",
"...": "E",
"....": "D",
COLUMN_MAPPING = {
# ....: D, ....: E, .....: F
# === ..... ===

FILL_BLOCKED = PatternFill(start_color="EAEAEA", end_color="EAEAEA", fill_type="solid")
# .....

}
".....": ROLE_EXEC_DOC,
"....": ROLE_INVOICE,
".....": ROLE_INVOICE,
"..... -3": ROLE_KS3,
"... -2": ROLE_KS2,

```

```

"value": 3,
".....": {
},
"date": "08.07"
"value": 3,
"...": {
"date": "02.07",
"value": 2,
"..... --3": {
},
}
"date": "08.07"
"value": 3,
"1 .... .": {
},
"date": "08.07"
"value": 3,
"1 .... .": {
},
"date": "08.07"
"value": 2,
".....": {
},
"date": "08.07"
"value": 1,
"...": {
},
"date": "08.07"
"value": 1,
".....": {
"date": "02.07",
"value": 2,
"... --2": {
"status": {
"sum": 3045600.78,
".....": {
"01_271_..": {
"01_..... (.....)": {
{
=== FILE: ./database.json ===

```

```

}
"red": FILL_RED,
"yellow": FILL_YELLOW,
"green": FILL_GREEN,
P1_STATUS_COLORS = {
# ..... 1 .... .

```

```

}
"red": FILL_RED,
"yellow": FILL_YELLOW,
"green": FILL_GREEN,
Z1_STATUS_COLORS = {
# ..... 1 .... .

```

```

}
"red": FILL_SDO_RED,
"yellow": FILL_SDO_YELLOW,
"green": FILL_SDO_GREEN,
SDO_STATUS_COLORS = {
# ..... (.....)

```



```

"status": {
"sum": 13.0,
"....": {
},
}
}
}
}
"date": "08.07"
"value": 3,
"1 .... .": {
},
"date": "08.07"
"value": 3,
"1 .... .": {
},
"date": "08.07"
"value": 3,
"....": {
"date": "02.07",
"value": 2,
".....": {
},
}
}
"date": "08.07"
"value": 3,
"1 .... .": {
},
"date": "08.07"
"value": 3,
"1 .... .": {
},
"date": "08.07"
"value": 3,
"....": {
"date": "02.07",
"value": 2,
"....": {
},
}
}
"date": "08.07"
"value": 3,
"1 .... .": {
},
"date": "08.07"
"value": 3,
"1 .... .": {
},
"date": "08.07"
"value": 3,
"....": {
"date": "02.07",
"value": 2,
".....": {
},
}
}
"date": "08.07"
"value": 3,
"1 .... .": {
},
"date": "08.07"
"value": 3,
"1 .... .": {
},
"date": "08.07"

```

```
}
}
"status": ""
"sum": 0,
"....": {
},
"status": 2
"sum": 850000,
".....": {
".....": {
"02_.....": {
},
}
}
"status": ""
"sum": 0,
"....": {
},
"status": ""
"sum": 0,
".....": {
"04_537_..": {
},
}
}
"status": ""
"sum": 0,
"....": {
},
"status": ""
"sum": 0,
".....": {
"03_538_..": {
},
}
}
"status": ""
"sum": 0,
"....": {
},
}
}
"date": "08.07"
"value": 3,
"....": {
".....": {
},
}
"date": "08.07"
"value": 1,
"...": {
},
"date": "08.07"
"value": 3,
"....": {
"... -2": {
"status": {
"sum": 4142769.05,
"....": {
"02_1505_...": {
},
}
}
"date": "07.07"
"value": 1,
```

```

"value": 3,
"...": {
".....": {
},
}
"date": ""
"value": "",
"...": {
"..... --3": {
},
}
"date": ""
"value": "",
"...": {
"... --2": {
"status": {
"sum": 12.0,
".....": {
},
}
}
"date": ""
"value": "",
".....": {
},
"date": "07.07"
"value": 3,
"...": {
},
"date": "07.07"
"value": 2,
".....": {
},
"date": "07.07"
"value": 2,
"..... --3": {
},
}
"date": "08.07"
"value": 1,
"...": {
"date": "07.07",
"value": 1,
"... --2": {
"status": {
"sum": 12.0,
".....": {
},
}
}
"date": ""
"value": "",
"status": {
"sum": 36532821.29,
".....": {
},
}
"date": ""
"value": "",
"status": {
"sum": 94859348.59,
"...": {
"01_.....": {
"03_.....3_.....": {
},

```

=== FILE: ./db_core.py ===

```
}
}
"is_new_change": false
"last_changed_sub": "02_1505_...",
"last_changed_dir": "01_..... (.....)",
"_meta": {
},
}
}
}
}
"date": ""
"value": "",
"status": {
"sum": 3422671.63,
".....": {
"01_.....": {
"05_....._....._.....": {
},
}
}
}
}
"date": ""
"value": "",
"status": {
"sum": 0.0,
".....": {
"01_..... ..": {
"04_.....1": {
},
}
}
}
}
"date": "03.07"
"value": 3,
"status": {
"sum": 57197064.42,
".....": {
},
}
}
"date": ""
"value": "",
"status": {
"sum": 13.0,
".....": {
},
}
}
}
}
"date": ""
"value": "",
"...": {
"..... ..": {
},
}
}
"date": ""
"value": "",
"...": {
".....": {
},
}
}
"date": "08.07"
```

```

import config
import db_core
=== FILE: ./db_viewer.py ===

print(f" [.....] ..... '{new_month}' ..... . ..... config.py")
f.write(code)
with open(config_path, "w", encoding="utf-8") as f:

code = re.sub(r"MONTHS_LIST\s*=\s*\[.*?\]", new_line_repr, code)
import re
else:
code = code.replace(old_line_repr, new_line_repr)
if old_line_repr in code:

new_line_repr = f"MONTHS_LIST = {repr(updated_months)}"
old_line_repr = f"MONTHS_LIST = {repr(config.MONTHS_LIST)}"
updated_months = [m for m in ALL_YEAR_MONTHS if m in config.MONTHS_LIST or m == new_month]

code = f.read()
with open(config_path, "r", encoding="utf-8") as f:
config_path = "config.py"
if new_month not in config.MONTHS_LIST:
"""..... . config.py . ....."""
def update_config_months(new_month):

json.dump(db, f, ensure_ascii=False, indent=4)
with open(json_path, "w", encoding="utf-8") as f:
json_path = "database.json"
def save_db(db):

return {}
return json.load(f)
with open(json_path, "r", encoding="utf-8") as f:
if os.path.exists(json_path):
json_path = "database.json"
def load_db():

return datetime.datetime.now().strftime("%d.%m")
"""....."""
def get_short_date():

return str(value)
except:
return f"{val_float:,.2f}".replace(",", " ")
return f"{int(val_float):,}".replace(",", " ")
if val_float.is_integer():
val_float = float(value)
return "0"
if value is None or value == "":
try:
"""....."""
def fmt_money(value):

]
".....", ".....", ".....", ".....", ".....", "....."
".....", ".....", ".....", ".....", ".....", ".....",
ALL_YEAR_MONTHS = [

import config
import datetime
import json
import os

```

```

if alignment: cell.alignment = alignment
if border: cell.border = border
if fill: cell.fill = fill
if font: cell.font = font
cell = ws.cell(row=row_idx, column=col)
for col in range(1, ws.max_column + 1):
    """....."""
def apply_row_style(ws, row_idx, font, fill, border, alignment=None):

FONT_LOG_DATE = openpyxl.styles.Font(name="Calibri", size=9, italic=True, color="7A7A7A")
# ..... 5-...

import config
import openpyxl
=== FILE: ./excel_styler.py ===

print("\n=====")

if dt else '{}')
print(f"      .. . {mth}: {fmt_sum} .... | ..... : {colors.get(val, '...')}{f' ({dt})'
dt = status_raw.get("date", "") if isinstance(status_raw, dict) else ""
val = status_raw.get("value", "") if isinstance(status_raw, dict) else status_raw
else:
print(f"      ... . {d_name} -> {colors.get(val, '...')}{f' ({dt})' if dt else '{}")
dt = doc_status.get("date", "") if isinstance(doc_status, dict) else ""
val = doc_status.get("value", "") if isinstance(doc_status, dict) else ""
doc_status = status_raw.get(d_name, {})
if mask.get("....", 1) == 1:
mask = config.DOCUMENT_ROLES.get(d_name, {})
for d_name in config.DOCUMENTS_LIST:
print(f"      .. . {mth}: {fmt_sum} .... | ..... : [..... .. ....]")
if isinstance(status_raw, dict) and "value" not in status_raw:

status_raw = mth_data.get("status", "")
fmt_sum = db_core.fmt_money(mth_data.get("sum", 0))
# .....: .....
mth_data = db[direction][sub_obj][mth]
for mth in available_months:

available_months.append(m)
if m not in available_months:
for m in raw_months:

available_months = [m for m in db_core.ALL_YEAR_MONTHS if m in raw_months]
raw_months = list(db[direction][sub_obj].keys())

print(f"      .. . .....: {sub_obj}")
for sub_obj in db[direction].keys():
print(f"\n. ....: {direction}")
for direction in db.keys():

....., "3": ". ...."}
colors = {1: ". ....", 2: ". ....", 3: ". ....", "": "...", "1": ". ....", "2": ".

return
print("=====")
print(".... .. database.json .....")
if not db:
db = db_core.load_db()
print("\n===== ..... =====")
"""....."""
def print_data():

```

```

sc = ws.cell(row=current_row, column=4, value=score)
ws.cell(row=current_row, column=2, value=text).font = config.FONT_DATA
current_row += 1
for score, text in specs:
    (..... ..), (1, "..... (..... ..)"]
specs = [(3, "... .. (..... ..)"), (2, ". .... / .. .....
Font(name="Calibri", size=11, bold=True)
ws.cell(row=current_row, column=2, value=".....:").font =
current_row += 2
def append_specifications(ws, current_row):

    fill=f_red, font=font_red))
ws.conditional_formatting.add("D2:I1000", CellIsRule(operator='equal', formula=['3'],
    fill=f_ylw, font=font_ylw))
ws.conditional_formatting.add("D2:I1000", CellIsRule(operator='equal', formula=['2'],
    fill=f_grn, font=font_grn))
ws.conditional_formatting.add("D2:I1000", CellIsRule(operator='equal', formula=['1'],
# ..... .. I (..... .. D, E, F, G, H, I)

Font(name="Calibri", size=11, color="9C6500"), Font(name="Calibri", size=11, color="9C0006")
font_grn, font_ylw, font_red = Font(name="Calibri", size=11, color="006100"),
f_red = config.PatternFill(start_color="FFC7CE", end_color="FFC7CE", fill_type="solid")
f_ylw = config.PatternFill(start_color="FFEB9C", end_color="FFEB9C", fill_type="solid")
f_grn = config.PatternFill(start_color="C6EFCE", end_color="C6EFCE", fill_type="solid")
def apply_conditional_formatting(ws):

import config
from openpyxl.formatting.rule import CellIsRule
from openpyxl.styles import Font
=== FILE: ./formatter.py ===

cell.fill = config.FILL_RED
elif status_str == "3":
cell.fill = config.FILL_YELLOW
elif status_str == "2":
cell.fill = config.FILL_GREEN
if status_str == "1":

cell.font = Font(name="Calibri", size=11, bold=True, color="000000")
# .....: ..... ..

cell.alignment = config.ALIGN_C

cell.value = status_str
except ValueError:
cell.value = int(status_str)
try:

status_str = str(status_val).strip()

return
if status_val == "" or status_val is None:

status_val = status_raw
else:
status_val = status_raw.get("value", "")
if isinstance(status_raw, dict):

from openpyxl.styles import Font
import config
"""....., ..... .., """"
def format_status_cell(cell, status_raw):

```

```

with open(json_path, "w", encoding="utf-8") as f:
try:
if os.path.exists(json_path):
json_path = "changes.json"
def clear_changes_json():

return "....."

pass
except Exception:
return f".....: {'', '.join(set(months_edited))}"
if months_edited:
months_edited.append(v["label"].strip())
elif isinstance(v, dict) and "label" in v:
months_edited.append(k.split("||")[-1])
if "||" in k:
for k, v in changes.items():
months_edited = []
if changes:
changes = json.load(f)
with open(json_path, "r", encoding="utf-8") as f:
try:
if os.path.exists(json_path) and os.path.getsize(json_path) > 2:
json_path = "changes.json"

return f".....: {'', '.join(directions)} ({' ', '.join(months)})"
months = set([r[2] for r in session_added_rows])
directions = set([r[0] for r in session_added_rows])
if session_added_rows:
def generate_auto_changelog(session_added_rows=None):

return "v2.1"
except Exception:
return "v2.1"
return f"v{''.join(parts)}"
parts[-1] = str(int(parts[-1]) + 1)
parts = version[1:].split('.')
if version.startswith('v'):
version = res.stdout.strip()
text=True, check=True)
res = subprocess.run(["git", "describe", "--tags", "--abbrev=0"], capture_output=True,
try:
def get_current_git_version():

import config
import subprocess
import json
import os
=== FILE: ./git_deployer.py ===

ws.column_dimensions['J'].width = 40
# ..... J (.....)
ws.column_dimensions['I'].width = 10
# ..... I (....)
ws.column_dimensions['H'].width = 7
ws.column_dimensions['F'].width = ws.column_dimensions['G'].width =
ws.column_dimensions['D'].width = ws.column_dimensions['E'].width =
ws.column_dimensions['C'].width = 8, 55, 16
ws.column_dimensions['A'].width, ws.column_dimensions['B'].width,
def set_column_widths(ws):

return current_row
sc.font, sc.alignment, sc.border = config.FONT_DATA, config.ALIGN_C, config.THIN_BORDER

```



```

".....", ".....", ".....", ".....", ".....", "....."]
valid_months = [".....", ".....", ".....", ".....", ".....", ".....", "....."]

sheet = model.CurrentController.ActiveSheet
model = event.Source
try:
    """
    ..... , ..... . changes.json.
    ..... LibreOffice Calc. ....
    """
def log_cell_change(event):

import os
import json
=== FILE: ./macros/cell_watcher.py ===

return True
clear_changes_json()
print(" [....] ..... .. Mac. ....")
else:
return False
print(" [..... GIT] .. .. .")
else:
return True
clear_changes_json()
print(" [.....] ... .. .. GitHub!")
if push_res.returncode == 0:

push_res = subprocess.run(["git", "push", "origin", "main", "--tags"])
subprocess.run(["git", "tag", "-a", final_ver, "-m", final_logs])
subprocess.run(["git", "commit", "-m", commit_message])
subprocess.run(["git", "add", "."])

print(f"\n [GIT] ..... : \"{commit_message}\"")
commit_message = f"{final_ver} . {final_logs}"

final_logs = user_logs if user_logs else default_logs
user_logs = input(f"..... [.. .....: {default_logs}]: ").strip()

final_ver = user_ver if user_ver else default_ver
user_ver = input(f"..... [.. .....: {default_ver}]: ").strip()
print("\n ..... :")

default_logs = generate_auto_changelog(session_added_rows)
default_ver = get_current_git_version()
if push_choice in (".", "y", "yes"):

).strip().lower()
push_choice = input("..... GitHub? (../.):")
print("\n [.....] ... 2: .....")

return False
print(" [.....] ..... .")
if result.returncode != 0:
result = subprocess.run(["python3", "main.py"])
print("\n [.....] ... 1: .....")
def run_production_pipeline(session_added_rows=None):

print(f" [.....] .. ..... changes.json: {e}")
except Exception as e:
print(" [.....] ..... changes.json .....")
json.dump({}, f)

```

```

saved_statuses, saved_sums = storage.load_historical_data(file_path)
# 1. ....

file_path = config.FULL_PATH if config.FULL_PATH else "....._..._.....xlsx"
print(".. [.....] .....")
def run():

import formatter
import builder
import storage
import config
import openpyxl
import os
=== FILE: ./main.py ===

g_exportedScripts = (log_cell_change,)

pass
except Exception:

json.dump(changes, f, ensure_ascii=False, indent=4)
with open(json_path, "w", encoding="utf-8") as f:

sub_obj = lbl
else:
}
"sum": float(val_cell) if val_cell else 0.0
"month": lbl,
"sub_obj": sub_obj,
"direction": direction,
changes[storage_key] = {
storage_key = f"{direction}||{sub_obj}||{lbl}"
if direction and sub_obj:
val_cell = sheet.getCellByPosition(2, r_idx).Value
if lbl in valid_months:

continue
lbl.startswith("....") or lbl == "..... / ..... / .....":
if (lbl.startswith(".") or lbl.startswith("...") or lbl.startswith(".....") or

continue
sub_obj = ""
direction = lbl
if lbl.startswith("01_") or lbl.startswith("02_"):

continue
if not lbl:
lbl = str(sheet.getCellByPosition(1, r_idx).String).strip()
for r_idx in range(1, 300):

sub_obj = ""
direction = ""

pass
except:
changes = json.load(f)
with open(json_path, "r", encoding="utf-8") as f:
try:
if os.path.exists(json_path):

json_path = os.path.join(project_dir, "changes.json")
project_dir = "/Users/test/Desktop/cont_ks2_2"
changes = {}

```

```

for row in range(2, old_ws.max_row + 1):
    c_dir, c_sub, c_mth = "", "", ""
    old_ws = old_wb["....."]
    return saved_statuses, saved_sums
old_wb.close()
if "....." not in old_wb.sheetnames:
    old_wb = openpyxl.load_workbook(file_path, data_only=True)
try:
    return saved_statuses, saved_sums
if not os.path.exists(file_path) or os.path.getsize(file_path) == 0:
    # .....
    saved_statuses, saved_sums = {}, {}
def load_historical_data(file_path):

from config import MONTHS_LIST
import openpyxl
import os
=== FILE: ./storage.py ===

run()
if __name__ == "__main__":

print(f". [.....] .....: {e}")
except Exception as e:
print(f". [.....] .... Excel/LibreOffice .....! .....")
except PermissionError:

.....!")
print(f". [.....] .... ..")
else:
print(f". [.....] .....!")
print(f".. ..: {file_size_kb} ..")
print(f". ....: {os.path.abspath(file_path)}")
print(f". [.....] .....!")
file_size_kb = round(os.path.getsize(file_path) / 1024, 1)
if os.path.exists(file_path) and os.path.getsize(file_path) > 0:
    # .....

wb.save(file_path)
try:
# 5. ....

formatter.set_column_widths(ws)
formatter.apply_conditional_formatting(ws)
formatter.append_specifications(ws, last_row)
last_row = builder.build_structure(ws, mock_data, saved_statuses, saved_sums)
# 4. ....

config.ALIGN_C)
builder.apply_row_style(ws, 1, config.FONT_HDR, config.FILL_HDR, config.THIN_BORDER,
ws.row_dimensions.height = 28
ws.append(config.HEADERS)
# 3. ....

ws.sheet_properties.outlinePr.summaryBelow = False
ws.sheet_view.showOutlineSymbols = True
ws.sheet_view.showGridLines = True
ws.title = "....."
ws = wb.active
wb = openpyxl.Workbook()
# 2. ....

mock_data = storage.group_linear_data(config.DATA_ROWS)

```

```

print("\n>> .....")
"""....., ..... (..... 3)."""
def menu_edit_data(select_target_func):

import wizard_git
import config
import db_core
=== FILE: ./wizard_edits.py ===

- [x] ....., ... .. 5 -> 4 .. ..... (1-3)

.....
- [x] ....., ... .. `ai_project_snapshot.txt` .....

`.css/.html/.htm`, `.xml` (.....)
- [x] ..... «.....» .....: `.py`, `.json`, `.txt`, `.md`, `.yaml/.yml`, `.js/.ts`,
(`.xlsx`, `.zip`, `.pdf`, `.odt`, `.png/.jpg` .. ...)
- [x] .....: `.git/`, `__pycache__`, `node_modules/` (.....), .....

- [x] ..... `ai_project_snapshot.txt` (.....)

.... txt»
- [x] ..... `wizard_manager.py` .....: `5 -> 4` «.....

# TODO . ..... txt
=== FILE: ./TODO.md ===

return mock_data
mock_data[direction][sub_obj][month] = [val, status]
if sub_obj not in mock_data[direction]: mock_data[direction][sub_obj] = {}
if direction not in mock_data: mock_data[direction] = {}
for direction, sub_obj, month, val, status in data_rows:
mock_data = {}
def group_linear_data(data_rows):

return saved_statuses, saved_sums
pass
except Exception:
old_wb.close()
if s_val is not None: saved_sums[(c_dir, c_sub, c_mth)] = s_val
s_val = old_ws.cell(row=row, column=3).value
if d_name == "... -2":
saved_statuses[(c_dir, c_sub, c_mth, d_name)] = (st_d, st_e, st_f, st_g)
if any(st in (1, 2, 3) for st in (st_d, st_e, st_f, st_g)):
# ..... 4 .....
st_g = old_ws.cell(row=row, column=7).value
st_f = old_ws.cell(row=row, column=6).value
st_e = old_ws.cell(row=row, column=5).value
st_d = old_ws.cell(row=row, column=4).value
d_name = val_b.replace(".", "").strip()
elif val_b.startswith(". "):
elif val_b in MONTHS_LIST: c_mth = val_b
elif val_b.startswith("..... "): c_sub = val_b.replace("..... ", "").strip()
elif "02_....." in val_b: c_dir = "02_....."
if "01_....." in val_b: c_dir = "01_..... (.....)"
if not val_b: continue
val_b = str(old_ws.cell(row=row, column=2).value or "").strip()

```

```

print(" . [.....] ..... .. ..!")
except ValueError:
break
print(f" [.....] ..... ..: {db_core.fmt_money(cleaned_sum)} ....")
# .....: ..... .. ..
wizard_git.register_action("sum_changed")

db_core.save_db(db)
}
"is_new_change": True
"last_changed_sub": target_sub,
"last_changed_dir": target_dir,
db["_meta"] = {
db[target_dir][target_sub][target_mth]["sum"] = cleaned_sum

continue
print(" . [.....] ..... .. ..!")
if cleaned_sum < 0:
cleaned_sum = float(new_sum_str.replace(" ", "").replace(",", "."))
try:
break
if new_sum_str.lower() in (".....", "0"):
new_sum_str = input("\n..... (.. '0' .. ..): ").strip()
while True:
if sub_choice == "1":

target_mth = months_in_db[m_idx]
return
if m_idx < 0 or m_idx >= len(months_in_db):
m_idx = int(m_choice) - 1
return
if not m_choice or not m_choice.isdigit():
m_choice = input("\n.....: ").strip()

print(f" {idx}. {m} (.....: {fmt_sum} .... | .....: {st_val})")
fmt_sum = db_core.fmt_money(db[target_dir][target_sub][m].get("sum", 0))
# .....: ..... .. ..
)
)
else (st_raw if st_raw else "...")
if isinstance(st_raw, dict)
st_raw.get("value", "...")
else (
if isinstance(st_raw, dict) and "value" not in st_raw
"..... .. ."
st_val = (
st_raw = db[target_dir][target_sub][m].get("status", "")
for idx, m in enumerate(months_in_db, 1):
print("\n.....: ")
months_in_db = list(db[target_dir][target_sub].keys())

return
if not target_dir or not target_sub:
target_dir, target_sub = select_target_func(db, allow_new=False)

return
if sub_choice not in ("1", "2", "3"):
sub_choice = input("..... (1-3): ").strip()
.....)
print(".....: \n 1. .... \n 2. .... \n 3.

return
if not db:
db = db_core.load_db()

```

```

print("\n... ..?")

status_date = "" if st_choice == "0" else db_core.get_short_date()
status_value = "" if st_choice == "0" else int(st_choice)

return
print(" [.....] .....")
if st_choice not in ("1", "2", "3", "0"):
st_choice = input("..... (0-3): ").strip()
print(f"\n..... {status_key}:\n 1. .... | 2. .... | 3. .... | 0. ....")

status_key = "1 .... ."
else:
status_key = "1 .... ."
elif which == "4":
status_key = "....."
elif which == "3":
status_key = "..."
elif which == "2":
status_key = "...."
if which == "1":

return
print(" [.....] .....")
if which not in ("1", "2", "3", "4", "5"):
which = input("..... (1-5): ").strip()
print("\n... ..? 1 - ..., 2 - ..., 3 - ....., 4 - 1 .... ., 5 - 1 .... .")
elif sub_choice == "3":

db_core.update_config_months(new_mth_name)
wizard_git.register_action("branch_added")
db_core.save_db(db)
}
"is_new_change": True
"last_changed_sub": target_sub,
"last_changed_dir": target_dir,
db["_meta"] = {
db[target_dir][target_sub][new_mth_name] = db[target_dir][target_sub].pop(target_mth)

return
wizard_git.register_action("sum_changed")
db_core.save_db(db)
}
"is_new_change": True
"last_changed_sub": target_sub,
"last_changed_dir": target_dir,
db["_meta"] = {
db[target_dir][target_sub][new_mth_name]["sum"] += float(old.get("sum", 0))
old = db[target_dir][target_sub].pop(target_mth)
if input().strip() == "1":
print(" . .... ! .....? (1-... / 2-...)")
if new_mth_name in db[target_dir][target_sub]:

new_mth_name = db_core.ALL_YEAR_MONTHS[int(new_mth_choice) - 1]

return
if not new_mth_choice or not new_mth_choice.isdigit():
new_mth_choice = input("\n..... (1-12): ").strip()

print(f" {i}. {m_name}")
for i, m_name in enumerate(db_core.ALL_YEAR_MONTHS, 1):
print("\n..... :")
elif sub_choice == "2":

```

```

print(f" . [.....] ... ..... {status_key}!")
if not allowed_docs:

allowed_docs.append(doc_name)
if mask.get(status_key, 1) == 1:
mask = config.DOCUMENT_ROLES.get(doc_name, {})
for doc_name in config.DOCUMENTS_LIST:
allowed_docs = []
elif apply_mode == "2":

")
print(f" [.....] ..... {status_key} ..... ! (.....)
wizard_git.register_action("status_changed")
db_core.save_db(db)
}
"is_new_change": True
"last_changed_sub": target_sub,
"last_changed_dir": target_dir,
db["_meta"] = {

doc_entry[status_key] = {"value": status_value, "date": status_date}
# ... ..... doc_entry ..... status_key.
# .. ..... status_key (.... ..).
.....,
# .... .....: status[doc] = {value,date}, .. ..... value/date ... .....
else:
doc_entry[status_key] = {"value": status_value, "date": status_date}
if status_key in doc_entry and isinstance(doc_entry[status_key], dict):
# .... ..... status[doc][status_key]
# .....

db[target_dir][target_sub][target_mth].setdefault("status", {})[d_name] = doc_entry
doc_entry = {}
if not isinstance(doc_entry, dict):
doc_entry = db[target_dir][target_sub][target_mth].setdefault("status", {}).get(d_name)

continue
if mask.get(status_key, 1) != 1:
mask = config.DOCUMENT_ROLES.get(d_name, {})
for d_name in config.DOCUMENTS_LIST:

db[target_dir][target_sub][target_mth]["status"] = {}
if not isinstance(db[target_dir][target_sub][target_mth].get("status", {}), dict):
# ....., ... .. dict .. status

current_status = {}
if not isinstance(current_status, dict):
# .....: status[doc] = {value,date} ... status[doc][status_key] = {value,date}
else:
db[target_dir][target_sub][target_mth]["status"] = {"value": status_value, "date": status_date}
# . .....
if isinstance(current_status, dict) and "value" in current_status:
# .....: ..... "status" = {value,date} . ... ..

current_status = db[target_dir][target_sub][target_mth].get("status", {})
# .. .....
# .....: status = {<doc_name>: {<status_key>: {value,date}}}}
# ..... , .. .....
if apply_mode == "1":

apply_mode = input("..... (1-2): ").strip()
print(" 2. ....")
print(" 1. ....")

```

```

"status_changed": False,
"sum_changed": False,
session_flags = {

import config
import sys
import os
=== FILE: ./wizard_git.py ===

print(f" [.....] ..... {status_key} ..... '{target_doc}' ..... !")
wizard_git.register_action("status_changed")
db_core.save_db(db)
}
"is_new_change": True
"last_changed_sub": target_sub,
"last_changed_dir": target_dir,
db["_meta"] = {

doc_entry[status_key] = {"value": status_value, "date": status_date}
# ..... , .. .

db[target_dir][target_sub][target_mth].setdefault("status", {})[target_doc] = doc_entry
doc_entry = {}
if not isinstance(doc_entry, dict):
doc_entry = db[target_dir][target_sub][target_mth].setdefault("status", {}).get(target_doc)
# ..... entry .....

}
}
"date": current_status.get("date", ""),
"value": current_status.get("value", ""),
status_key: {
db[target_dir][target_sub][target_mth]["status"][d_name] = {
for d_name in config.DOCUMENTS_LIST:
db[target_dir][target_sub][target_mth]["status"] = {}
# .....: .....
# ..... status[doc] = {status_key: {value,date}}.
# .....: status = {value,date} (.... ....) .
if "value" in current_status:

current_status = {}
if not isinstance(current_status, dict):
# ..... status[doc] .... status[doc][status_key].

current_status = db[target_dir][target_sub][target_mth].get("status", {})

target_doc = allowed_docs[doc_idx]

return
if doc_idx < 0 or doc_idx >= len(allowed_docs):
doc_idx = int(doc_choice) - 1

return
if not doc_choice or not doc_choice.isdigit():
doc_choice = input(f"..... (1-{len(allowed_docs)}): ").strip()

print(f" {idx}. {doc_name}")
for idx, doc_name in enumerate(allowed_docs, 1):
print(f"\n..... {status_key}:")

return

```



```

print(f"\n.....: {current_ver}")

for a in actions: print(a)
else:
print(" ..... ( ..... Excel).")
if not actions:

if session_flags["status_changed"]: actions.append(". .....")
if session_flags["sum_changed"]: actions.append(". .....")
(...../...../.....")
if session_flags["branch_added"]: actions.append(". .....")
actions = []
print("\n=== ..... ===")

suggested_ver = calculate_next_version(current_ver)
current_ver = getattr(config, "VERSION", getattr(config, "CURRENT_VERSION", "v1.0.0"))
print("\n. .... . GitHub...")
def run_git_sync():

f.writelines(lines)
with open("config.py", "w", encoding="utf-8") as f:

lines.append(f'\nVERSION = "{new_version}"\n')
if not version_replaced:

break
version_replaced = True
lines[idx] = f'{var_name} = "{new_version}"\n'
var_name = "VERSION" if line.strip().startswith("VERSION") else "CURRENT_VERSION"
if line.strip().startswith("VERSION") or line.strip().startswith("CURRENT_VERSION"):
for idx, line in enumerate(lines):
version_replaced = False

lines = f.readlines()
with open("config.py", "r", encoding="utf-8") as f:
return
if not os.path.exists("config.py"):
def update_config_version(new_version):

return f"v{major}.{minor}.{patch}"

patch += 1
elif session_flags["sum_changed"] or session_flags["status_changed"]:
patch = 0
minor += 1
if session_flags["branch_added"]:

major, minor, patch = parts

parts = [1, 0, 0]
except ValueError:
parts += [0] * (3 - len(parts))
if len(parts) < 3:
parts = [int(p) for p in ver_clean.split(".")]
try:
ver_clean = current_ver.strip().lower().replace("v", "")
def calculate_next_version(current_ver):

session_flags[action_type] = True
if action_type in session_flags:
def register_action(action_type):

}
"branch_added": False

```

```

exclude_dirs = {
def build_code_snapshot_txt(output_path: str):

import os
import wizard_git
import config
import builder
import db_viewer
import wizard_edits
import wizard_menus
=== FILE: ./wizard_manager.py ===

return True

session_flags[key] = False
for key in session_flags:

print(". [GITHUB] ... .....!")
os.system("git push")
os.system(f'git commit -m "{commit_msg}"')
os.system("git add .")
print("\n. .... GitHub...")

update_config_version(final_version)
commit_msg = f"{final_version} {comment}".strip()
comment = input("\n.....: ").strip()

print(" . [.....] .....!")
else:
sys.exit(0)
print("\n.....!")
elif choice == "0":
return False
print("\n. ....")
elif choice == "9":
break
final_version = f"v{final_version}"
if not final_version.startswith("v"):
final_version = input("..... (....., v1.2.5): ").strip()
elif choice == "3":
break
final_version = current_ver
elif choice == "2":
break
final_version = suggested_ver
if choice == "1":

choice = input("\n.....: ").strip()

print(" 0. ....")
print(" 9. .... (....., .....)")
print(" 3. ....")
print(f" 2. .... ({current_ver})")
print(f" 1. .... ({suggested_ver})")
print("\n.....:")
while True:

print("=====")
print(f".....: ... ({current_ver})")
else:
print(f".....: {suggested_ver}")
if current_ver != suggested_ver:

```

```
return False
if ext == "":
return False
if ext in exclude_exts:
ext = ext.lower()
_, ext = os.path.splitext(file_path)
def should_include_file(file_path: str) -> bool:
```

```
return dir_name in exclude_dirs
def should_skip_dir(dir_name: str) -> bool:
```

```
}
".xml",
".htm",
".html",
".css",
".ts",
".js",
".yaml",
".yml",
".md",
".txt",
".json",
".py",
code_exts = {
```

```
}
".dll",
".so",
".dylib",
".exe",
".bin",
".a",
".o",
".obj",
".gz",
".tar",
".rar",
".7z",
".avi",
".wav",
".mov",
".mp4",
".mp3",
".webp",
".gif",
".jpeg",
".jpg",
".png",
".odt",
".pdf",
".zip",
".xls",
".xlsx",
exclude_exts = {
```

```
}
"build",
"dist",
".mypy_cache",
"node_modules",
".blackbox",
"__pycache__",
".git",
```

```
wizard_menus.menu_add_record()
elif choice == "2":
    db_viewer.print_data()
if choice == "1":

continue
print(" [.....] .....")
if choice not in ("1", "2", "3", "4", "5", "0"):

choice = input("\n..... (0-4): ").strip()

print(" 0. ....")
print(" 5. ....")
print(" 4. .... Excel (. .... GitHub)")
print(" 3. .... (..... / ..... / .....)")
print(" 2. .... (..... / ..... / .....)")
print(" 1. .... (.....)")
print("\n>>> ..... <<<")
while True:
def main_menu():

out.write("\n\n")
out.write(f"[SKIPPED: {e}]\n")
except Exception as e:
out.write("[SKIPPED: not UTF-8]\n")
except UnicodeDecodeError:
out.write("\n")
if not content.endswith("\n"):
out.write(content)
content = f.read()
with open(fp, "r", encoding="utf-8") as f:
try:
out.write(f"=== FILE: ./{{rel}} ===\n")
rel = os.path.relpath(fp, root_dir)
for fp in files_collected:

out.write("=====\n\n")
out.write(f"Files: {{len(files_collected)}}\n")
out.write(f"Generated: {{__import__('datetime').datetime.now().isoformat()}}\n")
out.write(f"Root: {{root_dir}}\n")
out.write("=== PROJECT CODE SNAPSHOT ===\n")
with open(output_path, "w", encoding="utf-8") as out:

files_collected.sort(key=lambda p: os.path.relpath(p, root_dir).lower())

continue
except Exception:
files_collected.append(fp)
if should_include_file(fp):
try:
continue
if os.path.abspath(fp) == output_path_abs:
fp = os.path.join(cur_root, fn)
for fn in files:

dirs[:] = [d for d in dirs if not should_skip_dir(d)]
for cur_root, dirs, files in os.walk(root_dir):

output_path_abs = os.path.abspath(output_path)
root_dir = os.path.abspath(os.getcwd())
files_collected = []

return ext in code_exts
```

```

print("\n.....! ")
elif choice == "0":

print(" [.....] .....")
else:
print(" [..] ..... DEBUG .....")
logging.getLogger().setLevel(logging.DEBUG)
elif srv_choice == "2":
print(" [..] ..... DEBUG .....")
logging.getLogger().setLevel(logging.WARNING)
if srv_choice == "1":

import logging
else:

print(" [.....] · builder.py .. ..... test_columns_width_logic().")
else:
builder.test_columns_width_logic()
if hasattr(builder, "test_columns_width_logic"):

print("..... (D, E, F, G, H)...")
print(".....: 1.35 .. -> ..... · builder.py/openpyxl: 14.0")
print("\n[.....]")
elif srv_choice == "3":

input("..... Enter...")
print(f"\n· [.....] .....: {output_path}")
build_code_snapshot_txt(output_path)
output_path = os.path.join(os.getcwd(), output_name)
output_name = "ai_project_snapshot.txt"
if srv_choice == "4":

srv_choice = input(".....: ").strip()

print("4 - ..... txt (ai_project_snapshot.txt)")
print("3 - ..... builder.py")
print("2 - ..... DEBUG .....")
print("1 - ..... DEBUG · .....")
print("\n[.....]")
elif choice == "5":
print(f"· [.....] .....: {e}")
except Exception as e:
print("· [.....] .... Excel .....! .....")
except PermissionError:

wizard_git.run_git_sync()
print(f"· [.....] .....: {file_path}")
wb.save(file_path)
try:
file_path = config.FULL_PATH if config.FULL_PATH else "....._..._.xlsx"

builder.build_structure(ws)
ws.sheet_view.showGridLines = True

ws.title = "....."
ws = wb.active
wb = openpyxl.Workbook()

import openpyxl
print("\n· ..... Excel...")
elif choice == "4":
wizard_edits.menu_edit_data(wizard_menus.select_hierarchy_target)
elif choice == "3":
wizard_git.register_action("branch_added")

```

```

db = db_core.load_db()
print("\n>> .....")
def menu_add_record():

    return target_dir, target_sub

else: return None, None
if allow_new: target_sub = input(".....: ").strip()
else:
else: return None, None
target_sub = input(".....: ").strip()
elif allow_new and s_idx == len(sub_objs):
if 0 <= s_idx < len(sub_objs): target_sub = sub_objs[s_idx]

s_idx = int(s_choice) - 1
if not s_choice or not s_choice.isdigit(): return None, None
s_choice = input(f"..... (1-{{limit}}): ").strip()
limit = len(sub_objs) + 1 if allow_new else len(sub_objs)

if allow_new: print(f" {{len(sub_objs) + 1}}. [.....]")
for idx, s in enumerate(sub_objs, 1): print(f" {{idx}}. {{s}}")
print(f"\n..... .. '{target_dir}':")
sub_objs = list(db[target_dir].keys())
if target_dir in db and db[target_dir]:

target_sub = ""
if not target_dir: return None, None

else: return None, None
if allow_new: target_dir = input(".... ..: ").strip()
else:
else: return None, None
target_dir = input(".....: ").strip()
elif allow_new and d_idx == len(directions):
if 0 <= d_idx < len(directions): target_dir = directions[d_idx]

d_idx = int(d_choice) - 1
if not d_choice or not d_choice.isdigit(): return None, None
d_choice = input(f"\n..... (1-{{limit}}): ").strip()
limit = len(directions) + 1 if allow_new else len(directions)

if allow_new: print(f" {{len(directions) + 1}}. [.....]")
for idx, d in enumerate(directions, 1): print(f" {{idx}}. {{d}}")
print("\n.....:")
if directions:

directions = list(db.keys())
target_dir = ""
"""....."""
def select_hierarchy_target(db, allow_new=False):

import config
import db_core
=== FILE: ./wizard_menus.py ===

main_menu()
if __name__ == "__main__":

print(" [.....] .....")
else:
break

```

```

print("\n=== ..... ==")
db = db_core.load_db()
def print_data():

import git_deployer
import wizard_menus
import db_core
-----
### ....: wizard_manager.py ###

=====
..... (..... JSON)
=====
=== FILE: ./....._..._.txt ===

print(f" [.....] ..... '{month}' ..... !")
db_core.update_config_months(month)
db_core.save_db(db)
db[target_dir][target_sub][month] = {"sum": 0.0, "status": {"value": "", "date": ""}}

if target_sub not in db[target_dir]: db[target_dir][target_sub] = {}
if target_dir not in db: db[target_dir] = {}

break

return
input("\n..... Enter...")
print(" ..... -> ..... 3 .....")
print(f"\n . [.....] ..... '{month}' ..... !")
if target_dir in db and target_sub in db[target_dir] and month in db[target_dir][target_sub]:

month = db_core.ALL_YEAR_MONTHS[mth_idx]

continue
print(" . [.....] ..... .. 1 .. 12!")
if mth_idx < 0 or mth_idx >= 12:
mth_idx = int(mth_choice) - 1

continue
print(" . [.....] ..... .. 1 .. 12!")
if not mth_choice or not mth_choice.isdigit():

return
if mth_choice == "0":
mth_choice = input("\n..... (1-12) ... '0' .....: ").strip()

print(f" {i}. {m_name}{marker}")
marker = " [... ..]" if m_name in active_months else ""
for i, m_name in enumerate(db_core.ALL_YEAR_MONTHS, 1):

active_months = list(db[target_dir][target_sub].keys())
if isinstance(db[target_dir][target_sub], dict):
if target_dir in db and target_sub in db[target_dir]:
active_months = []

print("\n.....:")
while True:

if not target_dir or not target_sub: return
target_dir, target_sub = select_hierarchy_target(db, allow_new=True)

```

```

else:
target_dir = input("..... : ").strip()
elif allow_new and d_idx == len(directions):
if 0 <= d_idx < len(directions): target_dir = directions[d_idx]

d_idx = int(d_choice) - 1
if not d_choice or not d_choice.isdigit(): return None, None
d_choice = input(f"\n..... (1-{{limit}}): ").strip()
limit = len(directions) + 1 if allow_new else len(directions)

if allow_new: print(f"  {{len(directions)} + 1}. [.....]")
for idx, d in enumerate(directions, 1): print(f"  {{idx}}. {{d}}")
print("\n.....:")
if directions:

directions = list(db.keys())
target_dir = ""
""",....., """"
def select_hierarchy_target(db, allow_new=False):

import db_core
-----
### .....: wizard_menus.py ###

-----

run_wizard()
if __name__ == "__main__":

print(" .....")
else:
print(".. .....!"); break
elif choice == "5":
git_deployer.run_production_pipeline(session_added_rows=["....."])
elif choice == "4":
wizard_menus.menu_edit_data()
elif choice == "3":
wizard_menus.menu_add_record()
elif choice == "2":
print_data()
if choice == "1":

choice = input("..... (1-5): ").strip()
print("5. ....")
print("4. .... Excel (. .... GitHub)")
print("3. ....")
print("2. .... (...../.....)")
print("1. .... (.....)")
print("\n--- ..... (JSON ....) ---")
while True:
def run_wizard():

print("=====")
{data.get('status', '...')}")
print(f"      .. {{mth}}: {{db_core.fmt_money(data.get('sum', 0))}} .... | ..... :
for mth, data in months.items():
print(f"      .. .....: {{sub_obj}}")
for sub_obj, months in sub_objs.items():
print(f"\n .....: {{direction}}")
for direction, sub_objs in db.items():
return
print(".... ....")
if not db:

```



```

if sub_choice not in ("1", "2", "3"): return
sub_choice = input("..... (1-3): ").strip()
.....")
print(".....:\n 1. ....\n 2. ....\n 3.

if not db: return
db = db_core.load_db()
print("\n>> .....")
"""....., ..... (..... 3)."""
def menu_edit_data():

print(f" [.....] ..... '{month}' .....!")
db_core.update_config_months(month)
db_core.save_db(db)
db[target_dir][target_sub][month] = {"sum": 0.0, "status": {"value": "", "date": ""}}

if month in db[target_dir][target_sub]: return
if target_sub not in db[target_dir]: db[target_dir][target_sub] = {}
if target_dir not in db: db[target_dir] = {}

month = db_core.ALL_YEAR_MONTHS[mth_idx]
if mth_idx < 0 or mth_idx >= 12: return
mth_idx = int(mth_choice) - 1
if not mth_choice or not mth_choice.isdigit(): return
mth_choice = input("\n..... (1-12): ").strip()

print(f" {i}. {m_name}{marker}")
marker = " [... ..]" if m_name in active_months else ""
for i, m_name in enumerate(db_core.ALL_YEAR_MONTHS, 1):
db[target_dir]) else []
active_months = list(db[target_dir][target_sub].keys()) if (target_dir in db and target_sub in
print("\n.....:")

if not target_dir or not target_sub: return
target_dir, target_sub = select_hierarchy_target(db, allow_new=True)

db = db_core.load_db()
print("\n>> .....")
"""..... (..... 2)."""
def menu_add_record():

return target_dir, target_sub

target_sub = input(".....: ").strip()
else:
target_sub = input(".....: ").strip()
elif allow_new and s_idx == len(sub_objs):
if 0 <= s_idx < len(sub_objs): target_sub = sub_objs[s_idx]

s_idx = int(s_choice) - 1
if not s_choice or not s_choice.isdigit(): return None, None
s_choice = input(f"..... (1-{limit}): ").strip()
limit = len(sub_objs) + 1 if allow_new else len(sub_objs)

if allow_new: print(f" {len(sub_objs) + 1}. [.....]")
for idx, s in enumerate(sub_objs, 1): print(f" {idx}. {s}")
print(f"\n..... '{target_dir}':")
sub_objs = list(db[target_dir].keys())
if target_dir in db and db[target_dir]:
target_sub = ""

if not target_dir: return None, None

target_dir = input("....: ").strip()

```

```

-----
### .....: db_core.py ###
-----

print(" [.....] ..... .")
db_core.save_db(db)
db[target_dir][target_sub][target_mth]["status"] = {"value": "", "date": ""}
elif st_choice == "0":
print(" [.....] ..... .")
db_core.save_db(db)
db_core.get_short_date()}
db[target_dir][target_sub][target_mth]["status"] = {"value": int(st_choice), "date":
if st_choice in ("1", "2", "3"):
st_choice = input("..... (0-3): ").strip()
print("\n..... : \n 1. .... | 2. .... | 3. .... | 0. ....")
elif sub_choice == "3":

db_core.update_config_months(new_mth_name)
db_core.save_db(db)
db[target_dir][target_sub][new_mth_name] = db[target_dir][target_sub].pop(target_mth)

return
db_core.save_db(db)
db[target_dir][target_sub][new_mth_name]["sum"] += float(old.get("sum", 0))
old = db[target_dir][target_sub].pop(target_mth)
if input().strip() == "1":
print(" .. .....! .....? (1-... / 2-....)")
if new_mth_name in db[target_dir][target_sub]:

new_mth_name = db_core.ALL_YEAR_MONTHS[int(new_mth_choice) - 1]
if not new_mth_choice or not new_mth_choice.isdigit(): return
new_mth_choice = input("\n..... (1-12): ").strip()
for i, m_name in enumerate(db_core.ALL_YEAR_MONTHS, 1): print(f" {i}. {m_name}")
print("\n..... :")
elif sub_choice == "2":

except: print(" [.....] ..... .")
print(" [.....] ..... .")
db_core.save_db(db)
db[target_dir][target_sub][target_mth]["sum"] = float(new_sum_str.replace(",", "."))
try:
new_sum_str = input(f"\n..... : ").strip()
if sub_choice == "1":

target_mth = months_in_db[m_idx]
if m_idx < 0 or m_idx >= len(months_in_db): return
m_idx = int(m_choice) - 1
if not m_choice or not m_choice.isdigit(): return
m_choice = input("\n..... : ").strip()

.... | .....: {st_val}))
print(f" {idx}. {m} (.....: {db_core.fmt_money(db[target_dir][target_sub][m].get('sum', 0))}
"...")
st_val = st_raw.get("value", "...") if isinstance(st_raw, dict) else (st_raw if st_raw else
st_raw = db[target_dir][target_sub][m].get("status", "")
for idx, m in enumerate(months_in_db, 1):
print(f"\n..... :")
months_in_db = list(db[target_dir][target_sub].keys())

if not target_dir or not target_sub: return
target_dir, target_sub = select_hierarchy_target(db, allow_new=False)

```

```
### .....: builder.py ###
```

```
-----

print(f" [.....] ..... '{new_month}' ..... . ..... config.py")
f.write(code)
with open(config_path, "w", encoding="utf-8") as f:

code = re.sub(r"MONTHS_LIST\s*=\s*\[.*?\]", new_line_repr, code)
import re
else:
code = code.replace(old_line_repr, new_line_repr)
if old_line_repr in code:

new_line_repr = f"MONTHS_LIST = {repr(updated_months)}"
old_line_repr = f"MONTHS_LIST = {repr(config.MONTHS_LIST)}"
updated_months = [m for m in ALL_YEAR_MONTHS if m in config.MONTHS_LIST or m == new_month]

code = f.read()
with open(config_path, "r", encoding="utf-8") as f:
config_path = "config.py"
if new_month not in config.MONTHS_LIST:
"""..... . config.py . ....."""
def update_config_months(new_month):

json.dump(db, f, ensure_ascii=False, indent=4)
with open(json_path, "w", encoding="utf-8") as f:
json_path = "database.json"
def save_db(db):

return {}
return json.load(f)
with open(json_path, "r", encoding="utf-8") as f:
if os.path.exists(json_path):
json_path = "database.json"
def load_db():

return datetime.datetime.now().strftime("%d.%m")
"""..... .."""
def get_short_date():

return str(value)
except:
return f"{thousand_part},{parts}"
thousand_part = f"{int(parts):_}".replace("_", " ")
parts = f"{val_float:.2f}".split(".")
else:
return f"{int(val_float):_}".replace("_", " ")
if val_float.is_integer():
val_float = float(value)
try:
"""..... .."""
def fmt_money(value):

]
".....", ".....", ".....", ".....", ".....", "....."
".....", ".....", ".....", ".....", ".....", ".....",
ALL_YEAR_MONTHS = [

import config
import datetime
import json
import os
```

```

row_counter += 1
ws.row_dimensions[current_row].outline_level = 2
ws.cell(row=current_row, column=3).number_format = '#,##0.00'
ws.cell(row=current_row, column=1).alignment = config.ALIGN_C
config.THIN_BORDER, config.ALIGN_L)
excel_styler.apply_row_style(ws, current_row, config.FONT_MTH, config.FILL_MTH,
current_row = ws.max_row
ws.append([row_counter, mth, float(mth_data.get("sum", 0.0)), "", "", "", "", "", "", ""])
# .....: ..... 2

status_date = status_raw.get("date", "") if isinstance(status_raw, dict) else ""
status_raw = mth_data.get("status", "")
# ..... ..

mth_data = db[direction][sub_obj][mth]
if mth not in db[direction][sub_obj]: continue
for mth in config.MONTHS_LIST:

ws.row_dimensions[ws.max_row].outline_level = 1
config.THIN_BORDER, config.ALIGN_L)
excel_styler.apply_row_style(ws, ws.max_row, config.FONT_OBJ, config.FILL_OBJ,
ws.append(["", str(sub_obj), "", "", "", "", "", "", "", ""])
for sub_obj in db[direction].keys():

ws.row_dimensions[ws.max_row].outline_level = 0
config.THIN_BORDER, config.ALIGN_L)
excel_styler.apply_row_style(ws, ws.max_row, config.FONT_DIR, config.FILL_DIR,
ws.append(["", str(direction), "", "", "", "", "", "", "", ""])
for direction in db.keys():

row_counter = 1

db = json.load(f)
with open(json_path, "r", encoding="utf-8") as f:

if not os.path.exists(json_path): return ws.max_row
json_path = "database.json"

config.ALIGN_C)
excel_styler.apply_row_style(ws, 1, config.FONT_HDR, config.FILL_HDR, config.THIN_BORDER,
ws.append(config.HEADERS)
if ws.max_row == 1 and ws.cell(row=1, column=1).value is None:

ws.oddHeader.right.color = "7A7A7A"
ws.oddHeader.right.size = 9
ws.oddHeader.right.text = f".....: {now_str}"
now_str = datetime.datetime.now().strftime("%d.%m.%Y %H:%M")

ws.sheet_properties.outlinePr.summaryBelow = False
ws.freeze_panes = "A2"
"""..... 5-....."""
def build_structure(ws, mock_data=None, saved_statuses=None, saved_sums=None):

excel_styler.apply_row_style(ws, row_idx, font, fill, border, alignment)
"""..... .. main.py"""
def apply_row_style(ws, row_idx, font, fill, border, alignment=None):

import excel_styler
import config
import datetime
import os
import json
import openpyxl
-----

```

```

if isinstance(status_raw, dict):

status_val = ""
"""....., """
def format_status_cell(cell, status_raw):

if alignment: cell.alignment = alignment
if border: cell.border = border
if fill: cell.fill = fill
if font: cell.font = font
cell = ws.cell(row=row_idx, column=col)
for col in range(1, ws.max_column + 1):
"""....."""
def apply_row_style(ws, row_idx, font, fill, border, alignment=None):

FONT_LOG_DATE = openpyxl.styles.Font(name="Calibri", size=9, italic=True, color="7A7A7A")
# ..... 5-.. .....

import config
import openpyxl
-----
### .....: excel_styler.py ###

-----

return ws.max_row

ws.cell(row=row, column=10).value = f"=IF(C{row}>0, \".....\", \"\")"
if ws.cell(row=row, column=2).value in config.MONTHS_LIST:
for row in range(2, ws.max_row + 1):

ws.row_dimensions[log_row].outline_level = 4
# ..... 4 (. Excel ..... 5-.. ..... )
config.ALIGN_L)
excel_styler.apply_row_style(ws, log_row, excel_styler.FONT_LOG_DATE, None, config.THIN_BORDER,
# .....
log_row = ws.max_row
ws.append(["", f" .. ..... : {status_date}", "", "", "", "", "", "", "", "", ""])
if mask.get(".....", 1) == 1 and status_date:
# ..... 5-.. ..... : ..... , .....

excel_styler.format_status_cell(cell, status_raw)
if col_name == ".....":
else:
cell.value = ""
cell.fill = config.FILL_BLOCKED
if mask.get(col_name, 1) == 0:
cell = ws.cell(row=doc_row, column=col_idx)
for col_name, col_idx in column_mapping.items():

column_mapping = {".....": 4, ".....": 5, ".....": 6, "1 ..... ": 7, "1 ..... ": 8, ".....": 9}
..... ": 1, ".....": 1})
mask = config.DOCUMENT_ROLES.get(d_name, {".....": 1, ".....": 1, ".....": 1, "1 ..... ": 1, "1

ws.row_dimensions[doc_row].outline_level = 3
ws.cell(row=doc_row, column=3).number_format = '#,##0.00'
config.ALIGN_L)
excel_styler.apply_row_style(ws, doc_row, config.FONT_DATA, None, config.THIN_BORDER,
doc_row = ws.max_row
ws.append(["", f". {d_name}", "", "", "", "", "", "", "", "", ""])
for d_name in config.DOCUMENTS_LIST:
# .....: ..... 3

```

```

ALIGN_R = Alignment(horizontal="right", vertical="center")
ALIGN_L = Alignment(horizontal="left", vertical="center", wrap_text=True)
ALIGN_C = Alignment(horizontal="center", vertical="center", wrap_text=True)

color="B0B0B0"))
color="B0B0B0"), top=Side(style="thin", color="B0B0B0"), bottom=Side(style="thin",
THIN_BORDER = Border(left=Side(style="thin", color="B0B0B0"), right=Side(style="thin",

FILL_MTH = PatternFill(start_color="EAEAEA", end_color="EAEAEA", fill_type="solid")
FILL_OBJ = PatternFill(start_color="D9D9D9", end_color="D9D9D9", fill_type="solid")
FILL_HDR = FILL_DIR = PatternFill(start_color="1F4E78", end_color="1F4E78", fill_type="solid")

FONT_DATA = Font(name="Calibri", size=11)
FONT_MTH = Font(name="Calibri", size=11, bold=True, italic=True, color="000000")
FONT_OBJ = Font(name="Calibri", size=11, bold=True, color="000000")
FONT_DIR = Font(name="Calibri", size=12, bold=True, color="FFFFFF")
FONT_HDR = Font(name="Calibri", size=11, bold=True, color="FFFFFF")

FULL_PATH = os.path.join(FOLDER_NAME, FILE_NAME)
FILE_NAME = "....._..._.....xlsx"
FOLDER_NAME = ""

.....']
DOCUMENTS_LIST = ['... -2', '..... -3', '.....', '...', '.....']
MONTHS_LIST = ['.....', '...', '...', '...']

]
('02_.....', '.....', '...', 0, None),
('02_.....', '.....', '.....', 850000, 2),
('01_..... (.....)', '04_537_..', '...', 0, None),
('01_..... (.....)', '04_537_..', '.....', 0, None),
('01_..... (.....)', '03_538_..', '...', 0, None),
('01_..... (.....)', '03_538_..', '.....', 0, None),
('01_..... (.....)', '02_1505_..', '...', 0, None),
('01_..... (.....)', '02_1505_..', '.....', 450000, 3),
('01_..... (.....)', '01_271_..', '...', 0, None),
('01_..... (.....)', '01_271_..', '.....', 450000, 3),
DATA_ROWS = [

from openpyxl.styles import Font, Alignment, PatternFill, Border, Side
import os
-----
### ....: config.py ###

-----

return cell.value
cell.alignment = config.ALIGN_C
cell.font = config.FONT_DATA

cell.value = str(status_val)
except ValueError:
cell.value = int(status_val)
# .....: ..... int, ..... Excel
try:

return ""
if status_val == "" or status_val is None:

status_val = status_raw
else:
status_val = status_raw.get("value", "")

```

```

return f"..... : {'', '.join(set(months_edited))}"
if months_edited:
months_edited.append(v["label"].strip())
elif isinstance(v, dict) and "label" in v:
months_edited.append(k.split("||")[-1])
if "||" in k:
for k, v in changes.items():
months_edited = []
if changes:
changes = json.load(f)
with open(json_path, "r", encoding="utf-8") as f:
try:
if os.path.exists(json_path) and os.path.getsize(json_path) > 2:
json_path = "changes.json"

return f"..... : {'', '.join(directions)} ({'', '.join(months)})"
months = set([r[2] for r in session_added_rows])
directions = set([r[0] for r in session_added_rows])
if session_added_rows:
def generate_auto_changelog(session_added_rows=None):

return "v2.1"
except Exception:
return "v2.1"
return f"v{''.join(parts)}"
parts[-1] = str(int(parts[-1]) + 1)
parts = version[1:].split('.')
if version.startswith('v'):
version = res.stdout.strip()
text=True, check=True)
res = subprocess.run(["git", "describe", "--tags", "--abbrev=0"], capture_output=True,
try:
def get_current_git_version():

import config
import subprocess
import json
import os

-----
### .....: git_deployer.py ###

-----

FILL_BLOCKED = PatternFill(start_color="EAEAEA", end_color="EAEAEA", fill_type="solid")
# .....

}
".....": ROLE_EXEC_DOC
"....": ROLE_INVOICE,
".....": ROLE_INVOICE,
"..... -3": ROLE_KS3,
"... -2": ROLE_KS2,
DOCUMENT_ROLES = {

ROLE_EXEC_DOC = {".....": 1, "...": 0, ".....": 0, "1 .... .": 1, "1 .... .": 1, ".....": 0}
ROLE_INVOICE = {".....": 0, "...": 1, ".....": 0, "1 .... .": 1, "1 .... .": 1, ".....": 0}
ROLE_KS3 = {".....": 0, "...": 1, ".....": 1, "1 .... .": 1, "1 .... .": 1, ".....": 0}
ROLE_KS2 = {".....": 1, "...": 1, ".....": 1, "1 .... .": 1, "1 .... .": 1, ".....": 0}
# 1 - ..... (.....), 0 - ..... (.....)
# === ..... (.....) ===

".....", "1 .... .", "1 .... .", ".....", "..... ....."]
HEADERS = ["· ./.", "..... / ..... / .....", "..... (.....)", ".....", ".....",

```

```

import openpyxl
import os
-----

### ....: main.py ###

-----

return True
clear_changes_json()
print(" [....] ..... .. Mac. ....")
else:
return False
print(" [..... GIT] .. .., .....")
else:
return True
clear_changes_json()
print(" [.....] ... .. .. GitHub!")
if push_res.returncode == 0:

push_res = subprocess.run(["git", "push", "origin", "main", "--tags"])
subprocess.run(["git", "tag", "-a", final_ver, "-m", final_logs])
subprocess.run(["git", "commit", "-m", commit_message])
subprocess.run(["git", "add", "."])

print(f"\n [GIT] ..... : \"{commit_message}\"")
commit_message = f"{final_ver} . {final_logs}"

final_logs = user_logs if user_logs else default_logs
user_logs = input(f"..... [.. .....: {default_logs}]: ").strip()

final_ver = user_ver if user_ver else default_ver
user_ver = input(f"..... [.. .....: {default_ver}]: ").strip()
print("\n .....: ")

default_logs = generate_auto_changelog(session_added_rows)
default_ver = get_current_git_version()
if push_choice in (".", "y", "yes"):

).strip().lower()
push_choice = input("..... GitHub? (../...): ")
print("\n [.....] ... 2: .....")

return False
print(" [.....] ..... ..")
if result.returncode != 0:
result = subprocess.run(["python3", "main.py"])
print("\n [.....] ... 1: .....")
def run_production_pipeline(session_added_rows=None):

print(f" [.....] .. ..... changes.json: {e}")
except Exception as e:
print(" [.....] ..... changes.json .....")
json.dump({}, f)
with open(json_path, "w", encoding="utf-8") as f:
try:
if os.path.exists(json_path):
json_path = "changes.json"
def clear_changes_json():

return "....."

pass
except Exception:

```



```

"01_271_..": {
"01_..... (.....)": {
{
-----
### .....: database.json ###

-----

run()
if __name__ == "__main__":

print(f". [.....] .....: {e}")
except Exception as e:
print(". [.....] .... Excel/LibreOffice .....! .....")
except PermissionError:

.....!")
print(". [.....] .... .., .. ..... ..")
else:
print(f". [.....] ..... ..!")
print(f".. ..: {file_size_kb} ..")
print(f". ....: {os.path.abspath(file_path)}")
print(f". [.....] .... ..!")
file_size_kb = round(os.path.getsize(file_path) / 1024, 1)
if os.path.exists(file_path) and os.path.getsize(file_path) > 0:
# .....

wb.save(file_path)
try:
# 5. ....

formatter.set_column_widths(ws)
formatter.apply_conditional_formatting(ws)
formatter.append_specifications(ws, last_row)
last_row = builder.build_structure(ws, mock_data, saved_statuses, saved_sums)
# 4. ....

config.ALIGN_C)
builder.apply_row_style(ws, 1, config.FONT_HDR, config.FILL_HDR, config.THIN_BORDER,
ws.row_dimensions.height = 28
ws.append(config.HEADERS)
# 3. ....

ws.sheet_properties.outlinePr.summaryBelow = False
ws.sheet_view.showOutlineSymbols = True
ws.sheet_view.showGridLines = True
ws.title = "....."
ws = wb.active
wb = openpyxl.Workbook()
# 2. ....

mock_data = storage.group_linear_data(config.DATA_ROWS)
saved_statuses, saved_sums = storage.load_historical_data(file_path)
# 1. ....

file_path = config.FULL_PATH if config.FULL_PATH else "....._..._.xlsx"
print(f".. [.....] .....")
def run():

import formatter
import builder
import storage
import config

```

```
=====
.....
=====
=== FILE: ./..... /....._......txt ===
```

```
-----
}
}
}
}
"status": ""
"sum": 0,
"....": {
},
"status": 2
"sum": 850000,
".....": {
".....": {
"02_.....": {
},
}
}
"status": ""
"sum": 0,
"....": {
},
"status": ""
"sum": 0,
".....": {
"04_537_..": {
},
}
}
"status": ""
"sum": 0,
"....": {
},
"status": ""
"sum": 0,
".....": {
"03_538_..": {
},
}
}
"status": ""
"sum": 0,
"....": {
},
"status": 3
"sum": 450000,
".....": {
"02_1505_..": {
},
}
}
"status": ""
"sum": 0.0,
"....": {
},
}
"date": "01.07"
"value": 2,
"status": {
"sum": 17730436.92,
"....": {
```

```

- ..... (.....).
- ..... macOS · openpyxl:
6. .... MAC

- `config.py` · ....., ....., .....
- `git_deployer.py` · ..... GitHub .....
- `excel_styler.py` · ....., .....
- ..... (`freeze_panes`) · .....
- `builder.py` · ..... Excel. .... JSON ....., .....
- `db_core.py` · ...../..... JSON, .....
- ..... (..... 1-12, .....
- `wizard_menus.py` · ....., .....
- ..... (..... 1-5).
- `wizard_manager.py` · ....., .....
- .....
5. ....

- ..... LibreOffice Calc.
- ..... (1, 2, 3), ..
- ..... (`#EAEAEA`).
- .....
- .....), .....:
- ..... (....., .....
- ..... (.....).
- ..... , · `config.py` .....
4. .... (.....)

01.07")
- ..... 4: ..... (".....:
- ..... , ..)
- ..... 3: ..... (.....: .....-2, ..... -3, .....,
- ..... 2: ..... (....., .....-.....)
- ..... 1: ..... (.....)
- ..... 0: ..... (.....)
- ..... B: .....:
- .....
- ..... A: ..... , .....
- ..... (Outline Level) · 0 · 4:
- ..... Excel-..... JSON. ....
3. .... EXCEL (5 ..... OUTLINE)

... status (dict: {"value": 3, "date": "01.07"})
... sum (float: ....., .....)
... ..... (".....")
... ..... ("01_271_..")
... ..... ("01_..... (.....)")
... ..... (.....):
... ..... Excel-..... `database.json`.
... ..... ".....".
2. .... (Single Source of Truth)

- .....: LibreOffice Calc / MS Excel (macOS)
- ...../.....: ..... GitHub
- .....: ..... JSON
- .....: openpyxl (..... Excel)
- .....: Python 3
- ..... (.....-2, .....-3).
1. ....

```

```

return ws.max_row
if not os.path.exists(json_path):
    json_path = "database.json"

cell.border = config.THIN_BORDER
cell.font = config.FONT_HDR
cell.fill = config.FILL_HDR
cell = ws.cell(row=2, column=col_idx)
for col_idx in range(1, len(config.HEADERS) + 1):
    ws.append(config.HEADERS)
if ws.max_row == 1 and ws.cell(row=1, column=1).value is None:

ws.oddHeader.right.color = "7A7A7A"
ws.oddHeader.right.size = 9
ws.oddHeader.right.text = f".....: {now_str}"
now_str = datetime.datetime.now().strftime("%d.%m.%Y %H:%M")

ws.sheet_properties.outlinePr.summaryBelow = False
ws.freeze_panes = "A3"
def build_structure(ws, mock_data=None, saved_statuses=None, saved_sums=None):

..... .. Cmd+S!")
print(" [....] ..... . LibreOffice ....., ....., ... .. builder.py
{width_val}")
print(f" - ..... {letters[col_idx]} (..... {col_idx}): ..... =
width_val = 14.0
for col_idx in columns_to_check:
print(" ..... openpyxl:")
letters = {4: "D", 5: "E", 6: "F", 7: "G", 8: "H"}
columns_to_check = [4, 5, 6, 7, 8] # D, E, F, G, H
def test_columns_width_logic():

excel_styler.apply_row_style(ws, row_idx, font, fill, border, alignment)
def apply_row_style(ws, row_idx, font, fill, border, alignment=None):

import excel_styler
import db_viewer
import db_core
import config
import datetime
import os
import json
import openpyxl

logging.getLogger().setLevel(logging.WARNING)
import logging
=== FILE: ./builder.py ===

=====
....., .....!
=====

`.xlsx` . ..... `__pycache__`.
- ..... `.gitignore`, ..... GitHub .....
..... 5- .....
- ..... `TextBlock` . `append` . ..... `RichText` .....
.....,
- ..... , .....

```

```

row_counter += 1
ws.row_dimensions[current_row].outline_level = 2
ws.cell(row=current_row, column=3).number_format = "#,##0.00"
ws.cell(row=current_row, column=1).alignment = config.ALIGN_C
)
config.ALIGN_L,
config.THIN_BORDER,
config.FILL_MTH,
config.FONT_MTH,
current_row,
ws,
excel_styler.apply_row_style(
current_row = ws.max_row
)
[row_counter, mth, float(mth_data.get("sum", 0.0)), "", "", "", "", "", "", ""]
ws.append(
# .....: ..... 2

status_raw = mth_data.get("status", "")
mth_data = db[direction][sub_obj][mth]
for mth in available_months:

available_months.append(m)
if m not in available_months:
for m in raw_months:

available_months = [m for m in db_core.ALL_YEAR_MONTHS if m in raw_months]
raw_months = list(db[direction][sub_obj].keys())

ws.row_dimensions[ws.max_row].outline_level = 1
)
config.ALIGN_L,
config.THIN_BORDER,
config.FILL_OBJ,
config.FONT_OBJ,
ws.max_row,
ws,
excel_styler.apply_row_style(
ws.append(["", str(sub_obj), "", "", "", "", "", "", "", ""])
for sub_obj in db[direction].keys():

ws.row_dimensions[ws.max_row].outline_level = 0
)
config.ALIGN_L,
config.THIN_BORDER,
config.FILL_DIR,
config.FONT_DIR,
ws.max_row,
ws,
excel_styler.apply_row_style(
ws.append(["", str(direction), "", "", "", "", "", "", "", ""])
continue
if direction == "_meta":
for direction in db.keys():
row_counter = 1

is_new = meta.get("is_new_change", False)
target_sub = meta.get("last_changed_sub")
target_dir = meta.get("last_changed_dir")
meta = db.get("_meta", {}) if isinstance(db, dict) else {}
# .... 1: ..... .. .....

db = json.load(f)
with open(json_path, "r", encoding="utf-8") as f:

```

```

ws.append(
if mask.get("....", 1) == 1 and status_date:
# .....: ..... 4

cell.fill = openpyxl.styles.PatternFill(fill_type=None)
else:
excel_styler.format_status_cell(cell, clean_status_value)
if clean_status_value is not None:
else:
cell.value = ""
cell.fill = config.FILL_BLOCKED
if mask.get(col_name) == 0:

clean_status_value = doc_status_raw if col_name == "...." else None
else:
)
else doc_status_raw.get("value")
if col_name in doc_status_raw
doc_status_raw.get(col_name, {}).get("value")
clean_status_value = (
if isinstance(doc_status_raw, dict):

status_date = doc_status_raw.get("date", "") if isinstance(doc_status_raw, dict) else ""

doc_status_raw = status_raw
else:
doc_status_raw = status_raw[d_name]
if isinstance(status_raw, dict) and d_name in status_raw:
# .....: ...../...

cell = ws.cell(row=doc_row, column=col_idx)

col_idx = int(col_letter_or_idx)
else:
col_idx = openpyxl.utils.column_index_from_string(col_letter_or_idx)
if isinstance(col_letter_or_idx, str):
for col_name, col_letter_or_idx in config.COLUMN_MAPPING.items():

)
{"....": 1, "....": 1, ".....": 1, "1 .... .": 1, "1 .... .": 1, "....": 1},
d_name,
mask = config.DOCUMENT_ROLES.get(

ws.row_dimensions[doc_row].outline_level = 3
ws.cell(row=doc_row, column=3).number_format = "#,##0.00"
)
config.ALIGN_L,
config.THIN_BORDER,
None,
config.FONT_DATA,
doc_row,
ws,
excel_styler.apply_row_style(
doc_row = ws.max_row
ws.append(["", f". {d_name}", "", "", "", "", "", "", "", ""])
for d_name in config.DOCUMENTS_LIST:
# .....: ..... 3

ws.row_dimensions[current_row].hidden = hidden_default
hidden_default = False
if is_new and direction == target_dir and sub_obj == target_sub:
hidden_default = True
# ..... (.... 2)

```

[illegible]

```

('01_..... (.....)', '03_538_..', '.....', 0, None),
('01_..... (.....)', '02_1505_..', '.....', 0, None),
('01_..... (.....)', '02_1505_..', '.....', 450000, 3),
('01_..... (.....)', '01_271_..', '.....', 0, None),
('01_..... (.....)', '01_271_..', '.....', 450000, 3),
DATA_ROWS = [

from openpyxl.styles import Font, Alignment, PatternFill, Border, Side
import os
=== FILE: ./config.py ===

}
}
"sum": 0.0
"month": ".....",
"sub_obj": ".....",
"direction": "02_.....",
"02_..... ||..... ||.....": {
},
"sum": 850000.0
"month": ".....",
"sub_obj": ".....",
"direction": "02_.....",
"02_..... ||..... ||.....": {
},
"sum": 0.0
"month": ".....",
"sub_obj": "04_537_..",
"direction": "02_1505_..",
"02_1505_.. ||04_537_.. ||.....": {
},
"sum": 0.0
"month": ".....",
"sub_obj": "04_537_..",
"direction": "02_1505_..",
"02_1505_.. ||04_537_.. ||.....": {
},
"sum": 0.0
"month": ".....",
"sub_obj": "03_538_..",
"direction": "02_1505_..",
"02_1505_.. ||03_538_.. ||.....": {
},
"sum": 0.0
"month": ".....",
"sub_obj": "03_538_..",
"direction": "02_1505_..",
"02_1505_.. ||03_538_.. ||.....": {
},
"sum": 13.0
"month": ".....",
"sub_obj": ".. .... : 02.07",
"direction": "01_271_..",
"01_271_.. ||.. .... : 02.07 ||.....": {
{
=== FILE: ./changes.json ===

return ws.max_row

db_core.save_db(db)
db["_meta"]["is_new_change"] = False

```



```

}
".....": ROLE_EXEC_DOC,
"....": ROLE_INVOICE,
".....": ROLE_INVOICE,
"..... -3": ROLE_KS3,
"... -2": ROLE_KS2,
DOCUMENT_ROLES = {

ROLE_EXEC_DOC = {"....": 1, "...": 0, ".....": 0, "1 .... .": 1, "1 .... .": 1, "....": 0}
ROLE_INVOICE = {"....": 0, "...": 1, ".....": 0, "1 .... .": 1, "1 .... .": 1, "....": 0}
ROLE_KS3 = {"....": 0, "...": 1, ".....": 1, "1 .... .": 1, "1 .... .": 1, "....": 0}
ROLE_KS2 = {"....": 1, "...": 1, ".....": 1, "1 .... .": 1, "1 .... .": 1, "....": 0}
# 1 - ..... (.....), 0 - ..... (.....)
# == ..... (.....) ==

]
".....",
"....",
"1 .... .",
"1 .... .",
".....",
"....",
"....",
"..... (.....)",
"..... / ..... / .....",
". ./.",
HEADERS = [

ALIGN_R = Alignment(horizontal="right", vertical="center")
ALIGN_L = Alignment(horizontal="left", vertical="center", wrap_text=True)
ALIGN_C = Alignment(horizontal="center", vertical="center", wrap_text=True)

)
bottom=Side(style="thin", color="B0B0B0"),
top=Side(style="thin", color="B0B0B0"),
right=Side(style="thin", color="B0B0B0"),
left=Side(style="thin", color="B0B0B0"),
THIN_BORDER = Border(

FILL_MTH = PatternFill(start_color="EAEAEA", end_color="EAEAEA", fill_type="solid")
FILL_OBJ = PatternFill(start_color="D9D9D9", end_color="D9D9D9", fill_type="solid")
FILL_HDR = FILL_DIR = PatternFill(start_color="1F4E78", end_color="1F4E78", fill_type="solid")

FONT_DATA = Font(name="Calibri", size=11)
FONT_MTH = Font(name="Calibri", size=11, bold=True, italic=True, color="000000")
FONT_OBJ = Font(name="Calibri", size=11, bold=True, color="000000")
FONT_DIR = Font(name="Calibri", size=12, bold=True, color="FFFFFF")
FONT_HDR = Font(name="Calibri", size=11, bold=True, color="FFFFFF")

FULL_PATH = os.path.join(FOLDER_NAME, FILE_NAME)
FILE_NAME = "....._..._.xlsx"
FOLDER_NAME = ""

.....']
DOCUMENTS_LIST = ['... -2', '..... -3', '.....', '.....', '.....']
MONTHS_LIST = ['.....', '.....', '.....', '.....', '.....', '.....', '.....']

]
('02_.....', '.....', '.....', 0, None),
('02_.....', '.....', '.....', 850000, 2),
('01_..... (.....)', '04_537_...', '.....', 0, None),
('01_..... (.....)', '04_537_...', '.....', 0, None),
('01_..... (.....)', '03_538_...', '.....', 0, None),

```

```

}
"red": FILL_SDO_RED,
"yellow": FILL_SDO_YELLOW,
"green": FILL_SDO_GREEN,
SDO_STATUS_COLORS = {
# ..... (.....)

FILL_SDO_RED = FILL_RED
FILL_SDO_YELLOW = FILL_YELLOW
FILL_SDO_GREEN = FILL_GREEN
# .....
# === ..... ===

FILL_RED = PatternFill(start_color="FFC7CE", end_color="FFC7CE", fill_type="solid")
FILL_YELLOW = PatternFill(start_color="FFEB9C", end_color="FFEB9C", fill_type="solid")
FILL_GREEN = PatternFill(start_color="C6EFCE", end_color="C6EFCE", fill_type="solid")
# .....

VERSION = "v2. ...."

}
"1 .... ": ROLE_CUSTOMER,
"1 .... ": ROLE_CUSTOMER,
".....": ROLE_GEN_DIR,
"...": ROLE_KS2, # ... .. ROLE_KS2/ROLE_KS3 ..
ROLES = {

}
".....": 1,
"....": 1,
".....": 1,
"..... -3": 1,
"... -2": 1,
ROLE_CUSTOMER = {
# .....

}
".....": 0,
"....": 1,
".....": 0,
"..... -3": 1,
"... -2": 1,
ROLE_GEN_DIR = {
# ..... "....." .....
# ..... ROLE_KS2/ROLE_KS3/ROLE_INVOICE/ROLE_EXEC_DOC.
# === ..... ===

}
"1 .... ": "H",
"1 .... ": "G",
".....": "F",
"....": "E",
"....": "D",
COLUMN_MAPPING = {
# ....: D, ....: E, .....: F
# === ..... ===

FILL_BLOCKED = PatternFill(start_color="EAEAEA", end_color="EAEAEA", fill_type="solid")
# .....

```

```

},
"date": "08.07"
"value": 3,
"1 .... .": {
},
"date": "08.07"
"value": 3,
".....": {
},
"date": "08.07"
"value": 3,
"...": {
"date": "02.07",
"value": 2,
"..... --3": {
},
}
"date": "08.07"
"value": 3,
"1 .... .": {
},
"date": "08.07"
"value": 3,
"1 .... .": {
},
"date": "08.07"
"value": 2,
".....": {
},
"date": "08.07"
"value": 1,
"...": {
},
"date": "08.07"
"value": 1,
".....": {
"date": "02.07",
"value": 2,
"... --2": {
"status": {
"sum": 3045600.78,
".....": {
"01_271_..": {
"01_..... (.....)": {
{
=== FILE: ./database.json ===

```

```

}
"red": FILL_RED,
"yellow": FILL_YELLOW,
"green": FILL_GREEN,
P1_STATUS_COLORS = {
# ..... 1 .... .

}
"red": FILL_RED,
"yellow": FILL_YELLOW,
"green": FILL_GREEN,
Z1_STATUS_COLORS = {
# ..... 1 .... .

```

```
"02_1505_...": {
},
}
}
"date": "07.07"
"value": 1,
"status": {
"sum": 13.0,
"....": {
},
}
}
}
"date": "08.07"
"value": 3,
"1 .... .": {
},
"date": "08.07"
"value": 3,
"1 .... .": {
},
"date": "08.07"
"value": 3,
"....": {
"date": "02.07",
"value": 2,
".....": {
},
}
"date": "08.07"
"value": 3,
"1 .... .": {
},
"date": "08.07"
"value": 3,
"1 .... .": {
},
"date": "08.07"
"value": 3,
"....": {
"date": "02.07",
"value": 2,
"....": {
},
}
"date": "08.07"
"value": 3,
"1 .... .": {
},
"date": "08.07"
"value": 3,
"1 .... .": {
},
"date": "08.07"
"value": 3,
"....": {
"date": "02.07",
"value": 2,
".....": {
},
}
"date": "08.07"
"value": 3,
"1 .... .": {
```

```
"status": {
"sum": 94859348.59,
"...": {
"01_.....": {
"03_.....3_.....": {
},
}
}
"status": ""
"sum": 0,
"....": {
},
"status": 2
"sum": 850000,
".....": {
".....": {
"02_.....": {
},
}
}
"status": ""
"sum": 0,
"....": {
},
"status": ""
"sum": 0,
".....": {
"04_537_..": {
},
}
}
"status": ""
"sum": 0,
"....": {
},
"status": ""
"sum": 0,
".....": {
"03_538_..": {
},
}
}
"status": ""
"sum": 0,
"....": {
},
}
}
}
"date": "08.07"
"value": 3,
"....": {
".....": {
},
}
"date": "08.07"
"value": 1,
"...": {
},
"date": "08.07"
"value": 3,
"....": {
"... --2": {
"status": {
"sum": 4142769.05,
"....": {
```

```
"value": "",
"...": {
  "....": {
  },
}
"date": "08.07"
"value": 3,
"...": {
  ".....": {
  },
}
"date": ""
"value": "",
"...": {
  "..... --3": {
  },
}
"date": ""
"value": "",
"...": {
  "... --2": {
    "status": {
    },
    "sum": 12.0,
    ".....": {
    },
  },
}
}
"date": ""
"value": "",
".....": {
},
"date": "07.07"
"value": 3,
"....": {
},
"date": "07.07"
"value": 2,
".....": {
},
"date": "07.07"
"value": 2,
"..... --3": {
},
}
"date": "08.07"
"value": 1,
"....": {
"date": "07.07",
"value": 1,
"... --2": {
"status": {
"sum": 12.0,
"....": {
},
},
}
"date": ""
"value": "",
"status": {
"sum": 36532821.29,
"....": {
},
}
"date": ""
"value": "",
```

```
ALL_YEAR_MONTHS = [
```

```
import config
import datetime
import json
import os
=== FILE: ./db_core.py ===
```

```
}
}
"is_new_change": false
"last_changed_sub": "02_1505_...",
"last_changed_dir": "01_..... (.....)",
"_meta": {
},
}
}
}
"date": ""
"value": "",
"status": {
"sum": 3422671.63,
".....": {
"01_.....": {
"05_....._....._.....": {
},
}
}
}
"date": ""
"value": "",
"status": {
"sum": 0.0,
".....": {
"01_..... ..": {
"04_.....1": {
},
}
}
}
"date": "03.07"
"value": 3,
"status": {
"sum": 57197064.42,
".....": {
},
}
"date": ""
"value": "",
"status": {
"sum": 13.0,
".....": {
},
}
}
}
"date": ""
"value": "",
"...": {
"..... ..": {
},
}
}
"date": ""
```

```

print(".... database.json .....")
if not db:
db = db_core.load_db()
print("\n=====")
"""....."""
def print_data():

import config
import db_core
=== FILE: ./db_viewer.py ===

print(f" [.....] ..... '{new_month}' ..... config.py")
f.write(code)
with open(config_path, "w", encoding="utf-8") as f:

code = re.sub(r"MONTHS_LIST\s*=\s*\[.*?\]", new_line_repr, code)
import re
else:
code = code.replace(old_line_repr, new_line_repr)
if old_line_repr in code:

new_line_repr = f"MONTHS_LIST = {repr(updated_months)}"
old_line_repr = f"MONTHS_LIST = {repr(config.MONTHS_LIST)}"
updated_months = [m for m in ALL_YEAR_MONTHS if m in config.MONTHS_LIST or m == new_month]

code = f.read()
with open(config_path, "r", encoding="utf-8") as f:
config_path = "config.py"
if new_month not in config.MONTHS_LIST:
"""..... config.py ....."""
def update_config_months(new_month):

json.dump(db, f, ensure_ascii=False, indent=4)
with open(json_path, "w", encoding="utf-8") as f:
json_path = "database.json"
def save_db(db):

return {}
return json.load(f)
with open(json_path, "r", encoding="utf-8") as f:
if os.path.exists(json_path):
json_path = "database.json"
def load_db():

return datetime.datetime.now().strftime("%d.%m")
"""....."""
def get_short_date():

return str(value)
except:
return f"{val_float:,.2f}".replace(",", " ")
return f"{int(val_float):,}".replace(",", " ")
if val_float.is_integer():
val_float = float(value)
return "0"
if value is None or value == "":
try:
"""....."""
def fmt_money(value):

]
".....", ".....", ".....", ".....", ".....", "....."
".....", ".....", ".....", ".....", ".....", ".....",

```



```

if isinstance(status_raw, dict):

from openpyxl.styles import Font
import config
"""....., ..... . ..... . ..... .""
def format_status_cell(cell, status_raw):

if alignment: cell.alignment = alignment
if border: cell.border = border
if fill: cell.fill = fill
if font: cell.font = font
cell = ws.cell(row=row_idx, column=col)
for col in range(1, ws.max_column + 1):
"""..... . ..... . ..... .""
def apply_row_style(ws, row_idx, font, fill, border, alignment=None):

FONT_LOG_DATE = openpyxl.styles.Font(name="Calibri", size=9, italic=True, color="7A7A7A")
# ..... 5-... .....

import config
import openpyxl
=== FILE: ./excel_styler.py ===

print("\n=====")

if dt else '{}')
print(f"      .. . {mth}: {fmt_sum} .... | ..... : {colors.get(val, '...')}{f' ({dt})'
dt = status_raw.get("date", "") if isinstance(status_raw, dict) else ""
val = status_raw.get("value", "") if isinstance(status_raw, dict) else status_raw
else:
print(f"      ... . {d_name} -> {colors.get(val, '...')}{f' ({dt})' if dt else '{}")
dt = doc_status.get("date", "") if isinstance(doc_status, dict) else ""
val = doc_status.get("value", "") if isinstance(doc_status, dict) else ""
doc_status = status_raw.get(d_name, {})
if mask.get("....", 1) == 1:
mask = config.DOCUMENT_ROLES.get(d_name, {})
for d_name in config.DOCUMENTS_LIST:
print(f"      .. . {mth}: {fmt_sum} .... | ..... : [..... .. ....]")
if isinstance(status_raw, dict) and "value" not in status_raw:

status_raw = mth_data.get("status", "")
fmt_sum = db_core.fmt_money(mth_data.get("sum", 0))
# .....: .....
mth_data = db[direction][sub_obj][mth]
for mth in available_months:

available_months.append(m)
if m not in available_months:
for m in raw_months:

available_months = [m for m in db_core.ALL_YEAR_MONTHS if m in raw_months]
raw_months = list(db[direction][sub_obj].keys())

print(f"      .. . .....: {sub_obj}")
for sub_obj in db[direction].keys():
print(f"\n. ....: {direction}")
for direction in db.keys():

....., "3": ". ...."}
colors = {1: ". ....", 2: ". ....", 3: ". ....", "": "...", "1": ". ....", "2": ".

return
print("=====")

```

```

ws.column_dimensions['C'].width = 8, 55, 16
ws.column_dimensions['A'].width, ws.column_dimensions['B'].width,
def set_column_widths(ws):

return current_row
sc.font, sc.alignment, sc.border = config.FONT_DATA, config.ALIGN_C, config.THIN_BORDER
sc = ws.cell(row=current_row, column=4, value=score)
ws.cell(row=current_row, column=2, value=text).font = config.FONT_DATA
current_row += 1
for score, text in specs:
(..... ....), (1, "..... (..... ....)")]
specs = [(3, "... .. (..... ....)"), (2, ". .... / .. .....
Font(name="Calibri", size=11, bold=True)
ws.cell(row=current_row, column=2, value=".....:").font =
current_row += 2
def append_specifications(ws, current_row):

fill=f_red, font=font_red))
ws.conditional_formatting.add("D2:I1000", CellIsRule(operator='equal', formula=['3'],
fill=f_ylw, font=font_ylw))
ws.conditional_formatting.add("D2:I1000", CellIsRule(operator='equal', formula=['2'],
fill=f_grn, font=font_grn))
ws.conditional_formatting.add("D2:I1000", CellIsRule(operator='equal', formula=['1'],
# ..... I (..... D, E, F, G, H, I)

Font(name="Calibri", size=11, color="9C6500"), Font(name="Calibri", size=11, color="9C0006")
font_grn, font_ylw, font_red = Font(name="Calibri", size=11, color="006100"),
f_red = config.PatternFill(start_color="FFC7CE", end_color="FFC7CE", fill_type="solid")
f_ylw = config.PatternFill(start_color="FFEB9C", end_color="FFEB9C", fill_type="solid")
f_grn = config.PatternFill(start_color="C6EFCE", end_color="C6EFCE", fill_type="solid")
def apply_conditional_formatting(ws):

import config
from openpyxl.formatting.rule import CellIsRule
from openpyxl.styles import Font
=== FILE: ./formatter.py ===

cell.fill = config.FILL_RED
elif status_str == "3":
cell.fill = config.FILL_YELLOW
elif status_str == "2":
cell.fill = config.FILL_GREEN
if status_str == "1":

cell.font = Font(name="Calibri", size=11, bold=True, color="000000")
# .....: .....

cell.alignment = config.ALIGN_C

cell.value = status_str
except ValueError:
cell.value = int(status_str)
try:

status_str = str(status_val).strip()

return
if status_val == "" or status_val is None:

status_val = status_raw
else:
status_val = status_raw.get("value", "")

```

```

def run_production_pipeline(session_added_rows=None):

print(f" [.....] .. ..... changes.json: {e}")
except Exception as e:
print(" [.....] ..... changes.json .....")
json.dump({}, f)
with open(json_path, "w", encoding="utf-8") as f:
try:
if os.path.exists(json_path):
json_path = "changes.json"
def clear_changes_json():

return "....."

pass
except Exception:
return f".....: {' '.join(set(months_edited))}"
if months_edited:
months_edited.append(v["label"].strip())
elif isinstance(v, dict) and "label" in v:
months_edited.append(k.split("||")[-1])
if "||" in k:
for k, v in changes.items():
months_edited = []
if changes:
changes = json.load(f)
with open(json_path, "r", encoding="utf-8") as f:
try:
if os.path.exists(json_path) and os.path.getsize(json_path) > 2:
json_path = "changes.json"

return f".....: {' '.join(directions)} ({' '.join(months)})"
months = set([r[2] for r in session_added_rows])
directions = set([r[0] for r in session_added_rows])
if session_added_rows:
def generate_auto_changelog(session_added_rows=None):

return "v2.1"
except Exception:
return "v2.1"
return f"v{' '.join(parts)}"
parts[-1] = str(int(parts[-1]) + 1)
parts = version[1:].split('.')
if version.startswith('v'):
version = res.stdout.strip()
text=True, check=True)
res = subprocess.run(["git", "describe", "--tags", "--abbrev=0"], capture_output=True,
try:
def get_current_git_version():

import config
import subprocess
import json
import os
=== FILE: ./git_deployer.py ===

ws.column_dimensions['J'].width = 40
# ..... J (.....)
ws.column_dimensions['I'].width = 10
# ..... I (....)
ws.column_dimensions['H'].width = 7
ws.column_dimensions['F'].width = ws.column_dimensions['G'].width =
ws.column_dimensions['D'].width = ws.column_dimensions['E'].width =

```

```

try:
if os.path.exists(json_path):

json_path = os.path.join(project_dir, "changes.json")
project_dir = "/Users/test/Desktop/cont_ks2_2"
changes = {}

".....", ".....", ".....", ".....", ".....", "....."]
valid_months = [".....", ".....", ".....", ".....", ".....", ".....",

sheet = model.CurrentController.ActiveSheet
model = event.Source
try:
"""
....., ..... changes.json.
..... LibreOffice Calc. ....
"""
def log_cell_change(event):

import os
import json
=== FILE: ./macros/cell_watcher.py ===

return True
clear_changes_json()
print(" [....] ..... .. Mac. ....")
else:
return False
print(" [..... GIT] .. ..... ..")
else:
return True
clear_changes_json()
print(" [.....] ... .. .. GitHub!")
if push_res.returncode == 0:

push_res = subprocess.run(["git", "push", "origin", "main", "--tags"])
subprocess.run(["git", "tag", "-a", final_ver, "-m", final_logs])
subprocess.run(["git", "commit", "-m", commit_message])
subprocess.run(["git", "add", "."])

print(f"\n [GIT] ..... : \"{commit_message}\"")
commit_message = f"{final_ver} · {final_logs}"

final_logs = user_logs if user_logs else default_logs
user_logs = input(f"..... [.. .....: {default_logs}]: ").strip()

final_ver = user_ver if user_ver else default_ver
user_ver = input(f"..... [.. .....: {default_ver}]: ").strip()
print("\n .....: ")

default_logs = generate_auto_changelog(session_added_rows)
default_ver = get_current_git_version()
if push_choice in (".", "y", "yes"):

).strip().lower()
push_choice = input("..... GitHub? (../...): ")
print("\n [.....] ... 2: .....")

return False
print(" [.....] ..... ..")
if result.returncode != 0:
result = subprocess.run(["python3", "main.py"])
print("\n [.....] ... 1: .....")

```

```

ws.title = "....."
ws = wb.active
wb = openpyxl.Workbook()
# 2. ....

mock_data = storage.group_linear_data(config.DATA_ROWS)
saved_statuses, saved_sums = storage.load_historical_data(file_path)
# 1. ....

file_path = config.FULL_PATH if config.FULL_PATH else "....._..._.xlsx"
print(".. [.....] .....")
def run():

import formatter
import builder
import storage
import config
import openpyxl
import os
=== FILE: ./main.py ===

g_exportedScripts = (log_cell_change,)

pass
except Exception:

json.dump(changes, f, ensure_ascii=False, indent=4)
with open(json_path, "w", encoding="utf-8") as f:

sub_obj = lbl
else:
}
"sum": float(val_cell) if val_cell else 0.0
"month": lbl,
"sub_obj": sub_obj,
"direction": direction,
changes[storage_key] = {
storage_key = f"{direction}|{sub_obj}|{lbl}"
if direction and sub_obj:
val_cell = sheet.getCellByPosition(2, r_idx).Value
if lbl in valid_months:

continue
lbl.startswith("....") or lbl == "..... / ..... / .....":
if (lbl.startswith(".") or lbl.startswith("...") or lbl.startswith(".....") or

continue
sub_obj = ""
direction = lbl
if lbl.startswith("01_") or lbl.startswith("02_"):

continue
if not lbl:
lbl = str(sheet.getCellByPosition(1, r_idx).String).strip()
for r_idx in range(1, 300):

sub_obj = ""
direction = ""

pass
except:
changes = json.load(f)
with open(json_path, "r", encoding="utf-8") as f:

```

```

elif val_b in MONTHS_LIST: c_mth = val_b
elif val_b.startswith("..... "): c_sub = val_b.replace("..... ", "").strip()
elif "02_....." in val_b: c_dir = "02_....."
if "01_....." in val_b: c_dir = "01_..... (.....)"
if not val_b: continue
val_b = str(old_ws.cell(row=row, column=2).value or "").strip()
for row in range(2, old_ws.max_row + 1):
c_dir, c_sub, c_mth = "", "", ""
old_ws = old_wb["....."]
return saved_statuses, saved_sums
old_wb.close()
if "....." not in old_wb.sheetnames:
old_wb = openpyxl.load_workbook(file_path, data_only=True)
try:
return saved_statuses, saved_sums
if not os.path.exists(file_path) or os.path.getsize(file_path) == 0:
# .....
saved_statuses, saved_sums = {}, {}
def load_historical_data(file_path):

from config import MONTHS_LIST
import openpyxl
import os
=== FILE: ./storage.py ===

run()
if __name__ == "__main__":

print(f". [.....] .....: {e}")
except Exception as e:
print(f". [.....] .... Excel/LibreOffice .....! .....")
except PermissionError:

.....!")
print(f". [.....] .... , . .....
else:
print(f". [.....] .....!")
print(f".. ..: {file_size_kb} ..")
print(f". ....: {os.path.abspath(file_path)}")
print(f". [.....] .....!")
file_size_kb = round(os.path.getsize(file_path) / 1024, 1)
if os.path.exists(file_path) and os.path.getsize(file_path) > 0:
# .....

wb.save(file_path)
try:
# 5. ....

formatter.set_column_widths(ws)
formatter.apply_conditional_formatting(ws)
formatter.append_specifications(ws, last_row)
last_row = builder.build_structure(ws, mock_data, saved_statuses, saved_sums)
# 4. ....

config.ALIGN_C)
builder.apply_row_style(ws, 1, config.FONT_HDR, config.FILL_HDR, config.THIN_BORDER,
ws.row_dimensions.height = 28
ws.append(config.HEADERS)
# 3. ....

ws.sheet_properties.outlinePr.summaryBelow = False
ws.sheet_view.showOutlineSymbols = True
ws.sheet_view.showGridLines = True

```

```
print("..... : \n 1. .... \n 2. .... \n 3.
```

```
return
if not db:
db = db_core.load_db()
print("\n>> .....")
"""....., ..... (..... 3)."""
def menu_edit_data(select_target_func):
```

```
import wizard_git
import config
import db_core
=== FILE: ./wizard_edits.py ===
```

```
- [x] ....., ... .. 5 -> 4 .. ..... (1-3)
```

```
.....
- [x] ....., ... .. `ai_project_snapshot.txt` .....
`.css/.html/.htm`, `.xml` (.....)
- [x] ..... «.....» .....: `.py`, `.json`, `.txt`, `.md`, `.yaml/.yml`, `.js/.ts`,
(`.xlsx`, `.zip`, `.pdf`, `.odt`, `.png/.jpg` . ....)
- [x] .....: `.git/`, `__pycache__`, `node_modules/` (.....), .....
- [x] ..... `ai_project_snapshot.txt` (.....)
.... txt»
- [x] ..... `wizard_manager.py` .....: `5 -> 4` «.....
```

```
# TODO . ..... txt
=== FILE: ./TODO.md ===
```

```
return mock_data
mock_data[direction][sub_obj][month] = [val, status]
if sub_obj not in mock_data[direction]: mock_data[direction][sub_obj] = {}
if direction not in mock_data: mock_data[direction] = {}
for direction, sub_obj, month, val, status in data_rows:
mock_data = {}
def group_linear_data(data_rows):

return saved_statuses, saved_sums
pass
except Exception:
old_wb.close()
if s_val is not None: saved_sums[(c_dir, c_sub, c_mth)] = s_val
s_val = old_ws.cell(row=row, column=3).value
if d_name == "... -2":
saved_statuses[(c_dir, c_sub, c_mth, d_name)] = (st_d, st_e, st_f, st_g)
if any(st in (1, 2, 3) for st in (st_d, st_e, st_f, st_g)):
# ..... 4 .....
st_g = old_ws.cell(row=row, column=7).value
st_f = old_ws.cell(row=row, column=6).value
st_e = old_ws.cell(row=row, column=5).value
st_d = old_ws.cell(row=row, column=4).value
d_name = val_b.replace(".", "").strip()
elif val_b.startswith(". "):
```

```

print(f" {i}. {m_name}")
for i, m_name in enumerate(db_core.ALL_YEAR_MONTHS, 1):
print("\n..... :")
elif sub_choice == "2":

print(" . [.....] ..... ..!")
except ValueError:
break
print(f" [.....] ..... :: {db_core.fmt_money(cleaned_sum)} ....")
# .....: .....
wizard_git.register_action("sum_changed")

db_core.save_db(db)
}
"is_new_change": True
"last_changed_sub": target_sub,
"last_changed_dir": target_dir,
db["_meta"] = {
db[target_dir][target_sub][target_mth]["sum"] = cleaned_sum

continue
print(" . [.....] ..... ..!")
if cleaned_sum < 0:
cleaned_sum = float(new_sum_str.replace(" ", "").replace(",", "."))
try:
break
if new_sum_str.lower() in (".....", "0"):
new_sum_str = input("\n..... ( ... '0' ... ): ").strip()
while True:
if sub_choice == "1":

target_mth = months_in_db[m_idx]
return
if m_idx < 0 or m_idx >= len(months_in_db):
m_idx = int(m_choice) - 1
return
if not m_choice or not m_choice.isdigit():
m_choice = input("\n..... : ").strip()

print(f" {idx}. {m} (.....: {fmt_sum} .... | .....: {st_val})")
fmt_sum = db_core.fmt_money(db[target_dir][target_sub][m].get("sum", 0))
# .....: .....
)
)
else (st_raw if st_raw else "...")
if isinstance(st_raw, dict)
st_raw.get("value", "...")
else (
if isinstance(st_raw, dict) and "value" not in st_raw
"....."
st_val = (
st_raw = db[target_dir][target_sub][m].get("status", "")
for idx, m in enumerate(months_in_db, 1):
print("\n..... :")
months_in_db = list(db[target_dir][target_sub].keys())

return
if not target_dir or not target_sub:
target_dir, target_sub = select_target_func(db, allow_new=False)

return
if sub_choice not in ("1", "2", "3"):
sub_choice = input("..... (1-3): ").strip()
.....")

```



```

if apply_mode == "1":

apply_mode = input("..... (1-2): ").strip()
print(" 2. ....")
print(" 1. ....")
print("\n... ..?")

status_date = "" if st_choice == "0" else db_core.get_short_date()
status_value = "" if st_choice == "0" else int(st_choice)

return
print(" [.....] .....")
if st_choice not in ("1", "2", "3", "0"):
st_choice = input("..... (0-3): ").strip()
print(f"\n..... {status_key}:\n 1. .... | 2. .... | 3. .... | 0. ....")

status_key = "1 .... ."
else:
status_key = "1 .... ."
elif which == "4":
status_key = "....."
elif which == "3":
status_key = "..."
elif which == "2":
status_key = "....."
if which == "1":

return
print(" [.....] .....")
if which not in ("1", "2", "3", "4", "5"):
which = input("..... (1-5): ").strip()
print("\n... ..? 1 - ..., 2 - ..., 3 - ....., 4 - 1 .... ., 5 - 1 .... .")
elif sub_choice == "3":

db_core.update_config_months(new_mth_name)
wizard_git.register_action("branch_added")
db_core.save_db(db)
}
"is_new_change": True
"last_changed_sub": target_sub,
"last_changed_dir": target_dir,
db["_meta"] = {
db[target_dir][target_sub][new_mth_name] = db[target_dir][target_sub].pop(target_mth)

return
wizard_git.register_action("sum_changed")
db_core.save_db(db)
}
"is_new_change": True
"last_changed_sub": target_sub,
"last_changed_dir": target_dir,
db["_meta"] = {
db[target_dir][target_sub][new_mth_name]["sum"] += float(old.get("sum", 0))
old = db[target_dir][target_sub].pop(target_mth)
if input().strip() == "1":
print(" . ....! .....? (1-... / 2-....)")
if new_mth_name in db[target_dir][target_sub]:

new_mth_name = db_core.ALL_YEAR_MONTHS[int(new_mth_choice) - 1]

return
if not new_mth_choice or not new_mth_choice.isdigit():
new_mth_choice = input("\n..... (1-12): ").strip()

```

```

print(f" {idx}. {doc_name}")
for idx, doc_name in enumerate(allowed_docs, 1):
print(f"\n..... {status_key}:")

return
print(f" . [.....] ..... {status_key}!")
if not allowed_docs:

allowed_docs.append(doc_name)
if mask.get(status_key, 1) == 1:
mask = config.DOCUMENT_ROLES.get(doc_name, {})
for doc_name in config.DOCUMENTS_LIST:
allowed_docs = []
elif apply_mode == "2":

")
print(f" [.....] ..... {status_key} ..... ! (.....)
wizard_git.register_action("status_changed")
db_core.save_db(db)
}
"is_new_change": True
"last_changed_sub": target_sub,
"last_changed_dir": target_dir,
db["_meta"] = {

doc_entry[status_key] = {"value": status_value, "date": status_date}
# ... ..... doc_entry ..... status_key.
# .. ..... status_key (.....).
.....,
# .....: status[doc] = {value,date}, .. ..... value/date ... .....
else:
doc_entry[status_key] = {"value": status_value, "date": status_date}
if status_key in doc_entry and isinstance(doc_entry[status_key], dict):
# ..... status[doc][status_key]
# .....

db[target_dir][target_sub][target_mth].setdefault("status", {})[d_name] = doc_entry
doc_entry = {}
if not isinstance(doc_entry, dict):
doc_entry = db[target_dir][target_sub][target_mth].setdefault("status", {}).get(d_name)

continue
if mask.get(status_key, 1) != 1:
mask = config.DOCUMENT_ROLES.get(d_name, {})
for d_name in config.DOCUMENTS_LIST:

db[target_dir][target_sub][target_mth]["status"] = {}
if not isinstance(db[target_dir][target_sub][target_mth].get("status", {}), dict):
# ....., ... .. dict .. status

current_status = {}
if not isinstance(current_status, dict):
# .....: status[doc] = {value,date} ... status[doc][status_key] = {value,date}
else:
db[target_dir][target_sub][target_mth]["status"] = {"value": status_value, "date": status_date}
# .....
if isinstance(current_status, dict) and "value" in current_status:
# .....: ..... "status" = {value,date} .....

current_status = db[target_dir][target_sub][target_mth].get("status", {})
# .....
# .....: status = {<doc_name>: {<status_key>: {value,date}}}}
# ....., .....

```

```

if action_type in session_flags:
def register_action(action_type):

}
"branch_added": False
"status_changed": False,
"sum_changed": False,
session_flags = {

import config
import sys
import os
=== FILE: ./wizard_git.py ===

print(f" [.....] ..... {status_key} ..... '{target_doc}' ..... !")
wizard_git.register_action("status_changed")
db_core.save_db(db)
}
"is_new_change": True
"last_changed_sub": target_sub,
"last_changed_dir": target_dir,
db["_meta"] = {

doc_entry[status_key] = {"value": status_value, "date": status_date}
# ..... , .. .

db[target_dir][target_sub][target_mth].setdefault("status", {})[target_doc] = doc_entry
doc_entry = {}
if not isinstance(doc_entry, dict):
doc_entry = db[target_dir][target_sub][target_mth].setdefault("status", {}).get(target_doc)
# ..... entry .....

}
}
"date": current_status.get("date", ""),
"value": current_status.get("value", ""),
status_key: {
db[target_dir][target_sub][target_mth]["status"][d_name] = {
for d_name in config.DOCUMENTS_LIST:
db[target_dir][target_sub][target_mth]["status"] = {}
# .....
# ..... status[doc] = {status_key: {value,date}}.
# ..... status = {value,date} (.....) .
if "value" in current_status:

current_status = {}
if not isinstance(current_status, dict):
# ..... status[doc] .... status[doc][status_key].

current_status = db[target_dir][target_sub][target_mth].get("status", {})

target_doc = allowed_docs[doc_idx]

return
if doc_idx < 0 or doc_idx >= len(allowed_docs):
doc_idx = int(doc_choice) - 1

return
if not doc_choice or not doc_choice.isdigit():
doc_choice = input(f"..... (1-{len(allowed_docs)}): ").strip()

```

```

print("=====")
print(f".....: ... ({{current_ver}})")
else:
print(f".....: {{suggested_ver}}")
if current_ver != suggested_ver:
print(f"\n.....: {{current_ver}}")

for a in actions: print(a)
else:
print(" ..... (..... Excel).")
if not actions:

if session_flags["status_changed"]: actions.append(". .....")
if session_flags["sum_changed"]: actions.append(". .....")
(...../...../.....)")
if session_flags["branch_added"]: actions.append(". .....")
actions = []
print("\n=== ..... ===")

suggested_ver = calculate_next_version(current_ver)
current_ver = getattr(config, "VERSION", getattr(config, "CURRENT_VERSION", "v1.0.0"))
print("\n. .... . GitHub...")
def run_git_sync():

f.writelines(lines)
with open("config.py", "w", encoding="utf-8") as f:

lines.append(f'\nVERSION = "{new_version}"\n')
if not version_replaced:

break
version_replaced = True
lines[idx] = f'{{var_name}} = "{new_version}"\n'
var_name = "VERSION" if line.strip().startswith("VERSION") else "CURRENT_VERSION"
if line.strip().startswith("VERSION") or line.strip().startswith("CURRENT_VERSION"):
for idx, line in enumerate(lines):
version_replaced = False

lines = f.readlines()
with open("config.py", "r", encoding="utf-8") as f:
return
if not os.path.exists("config.py"):
def update_config_version(new_version):

return f"v{{major}}.{{minor}}.{{patch}}"

patch += 1
elif session_flags["sum_changed"] or session_flags["status_changed"]:
patch = 0
minor += 1
if session_flags["branch_added"]:

major, minor, patch = parts

parts = [1, 0, 0]
except ValueError:
parts += [0] * (3 - len(parts))
if len(parts) < 3:
parts = [int(p) for p in ver_clean.split(".")]
try:
ver_clean = current_ver.strip().lower().replace("v", "")
def calculate_next_version(current_ver):

session_flags[action_type] = True

```

```

".mypy_cache",
"node_modules",
".blackbox",
"__pycache__",
".git",
exclude_dirs = {
def build_code_snapshot_txt(output_path: str):

import os
import wizard_git
import config
import builder
import db_viewer
import wizard_edits
import wizard_menus
=== FILE: ./wizard_manager.py ===

return True

session_flags[key] = False
for key in session_flags:

print(". [GITHUB] ... .....!")
os.system("git push")
os.system(f'git commit -m "{commit_msg}"')
os.system("git add .")
print("\n. .... GitHub...")

update_config_version(final_version)
commit_msg = f"{final_version} {comment}".strip()
comment = input("\n.....: ").strip()

print(" . [.....] .....!")
else:
sys.exit(0)
print("\n.....!")
elif choice == "0":
return False
print("\n. ....")
elif choice == "9":
break
final_version = f"v{final_version}"
if not final_version.startswith("v"):
final_version = input("..... (....., v1.2.5): ").strip()
elif choice == "3":
break
final_version = current_ver
elif choice == "2":
break
final_version = suggested_ver
if choice == "1":

choice = input("\n.....: ").strip()

print(" 0. ....")
print(" 9. .... (....., .....)")
print(" 3. ....")
print(f" 2. .... ({current_ver})")
print(f" 1. .... ({suggested_ver})")
print("\n.....:")
while True:

```

```

output_path_abs = os.path.abspath(output_path)
root_dir = os.path.abspath(os.getcwd())
files_collected = []

return ext in code_exts
return False
if ext == "":
return False
if ext in exclude_exts:
ext = ext.lower()
_, ext = os.path.splitext(file_path)
def should_include_file(file_path: str) -> bool:

return dir_name in exclude_dirs
def should_skip_dir(dir_name: str) -> bool:

}
".xml",
".htm",
".html",
".css",
".ts",
".js",
".yaml",
".yml",
".md",
".txt",
".json",
".py",
code_exts = {

}
".dll",
".so",
".dylib",
".exe",
".bin",
".a",
".o",
".obj",
".gz",
".tar",
".rar",
".7z",
".avi",
".wav",
".mov",
".mp4",
".mp3",
".webp",
".gif",
".jpeg",
".jpg",
".png",
".odt",
".pdf",
".zip",
".xls",
".xlsx",
exclude_exts = {

}
"build",
"dist",

```

```

print("\n. .... Excel...")
elif choice == "4":
wizard_edits.menu_edit_data(wizard_menus.select_hierarchy_target)
elif choice == "3":
wizard_git.register_action("branch_added")
wizard_menus.menu_add_record()
elif choice == "2":
db_viewer.print_data()
if choice == "1":

continue
print(" [.....] .....")
if choice not in ("1", "2", "3", "4", "5", "0"):

choice = input("\n..... (0-4): ").strip()

print(" 0. ....")
print(" 5. ....")
print(" 4. .... Excel (. .... GitHub)")
print(" 3. .... (..... / ..... / .....)")
print(" 2. .... (..... / ..... / .....)")
print(" 1. .... (.....)")
print("\n>>> ..... <<<")
while True:
def main_menu():

out.write("\n\n")
out.write(f"[SKIPPED: {e}]\n")
except Exception as e:
out.write("[SKIPPED: not UTF-8]\n")
except UnicodeDecodeError:
out.write("\n")
if not content.endswith("\n"):
out.write(content)
content = f.read()
with open(fp, "r", encoding="utf-8") as f:
try:
out.write(f"=== FILE: ./{{rel}} ===\n")
rel = os.path.relpath(fp, root_dir)
for fp in files_collected:

out.write("=====\n\n")
out.write(f"Files: {len(files_collected)}\n")
out.write(f"Generated: {__import__('datetime').datetime.now().isoformat()}\n")
out.write(f"Root: {root_dir}\n")
out.write("=== PROJECT CODE SNAPSHOT ===\n")
with open(output_path, "w", encoding="utf-8") as out:

files_collected.sort(key=lambda p: os.path.relpath(p, root_dir).lower())

continue
except Exception:
files_collected.append(fp)
if should_include_file(fp):
try:
continue
if os.path.abspath(fp) == output_path_abs:
fp = os.path.join(cur_root, fn)
for fn in files:

dirs[:] = [d for d in dirs if not should_skip_dir(d)]
for cur_root, dirs, files in os.walk(root_dir):

```

```

print(" [.....] .....")
else:
break
print("\n.....!")
elif choice == "0":

print(" [.....] .....")
else:
print(" [..] ..... DEBUG .....")
logging.getLogger().setLevel(logging.DEBUG)
elif srv_choice == "2":
print(" [..] ..... DEBUG .....")
logging.getLogger().setLevel(logging.WARNING)
if srv_choice == "1":

import logging
else:

print(" [.....] · builder.py · ..... test_columns_width_logic().")
else:
builder.test_columns_width_logic()
if hasattr(builder, "test_columns_width_logic"):

print("..... (D, E, F, G, H)...")
print(".....: 1.35 · -> ..... · builder.py/openpyxl: 14.0")
print("\n[.....]")
elif srv_choice == "3":

input("..... Enter...")
print(f"\n· [.....] .....: {output_path}")
build_code_snapshot_txt(output_path)
output_path = os.path.join(os.getcwd(), output_name)
output_name = "ai_project_snapshot.txt"
if srv_choice == "4":

srv_choice = input(".....: ").strip()

print("4 - ..... txt (ai_project_snapshot.txt)")
print("3 - ..... builder.py")
print("2 - ..... DEBUG .....")
print("1 - ..... DEBUG .....")
print("\n[.....]")
elif choice == "5":
print(f"· [.....] .....: {e}")
except Exception as e:
print("· [.....] ..... Excel .....!")
except PermissionError:

wizard_git.run_git_sync()
print(f"· [.....] .....: {file_path}")
wb.save(file_path)
try:
file_path = config.FULL_PATH if config.FULL_PATH else "....._..._.xlsx"

builder.build_structure(ws)
ws.sheet_view.showGridLines = True

ws.title = "....."
ws = wb.active
wb = openpyxl.Workbook()

import openpyxl

```



```

while True:

if not target_dir or not target_sub: return
target_dir, target_sub = select_hierarchy_target(db, allow_new=True)

db = db_core.load_db()
print("\n>> .....")
def menu_add_record():

return target_dir, target_sub

else: return None, None
if allow_new: target_sub = input(".....: ").strip()
else:
else: return None, None
target_sub = input(".....: ").strip()
elif allow_new and s_idx == len(sub_objs):
if 0 <= s_idx < len(sub_objs): target_sub = sub_objs[s_idx]

s_idx = int(s_choice) - 1
if not s_choice or not s_choice.isdigit(): return None, None
s_choice = input(f"..... (1-{{limit}}): ").strip()
limit = len(sub_objs) + 1 if allow_new else len(sub_objs)

if allow_new: print(f" {{len(sub_objs) + 1}}. [.....]")
for idx, s in enumerate(sub_objs, 1): print(f" {{idx}}. {{s}}")
print(f"\n..... '{target_dir}':")
sub_objs = list(db[target_dir].keys())
if target_dir in db and db[target_dir]:

target_sub = ""
if not target_dir: return None, None

else: return None, None
if allow_new: target_dir = input(".... ..: ").strip()
else:
else: return None, None
target_dir = input(".....: ").strip()
elif allow_new and d_idx == len(directions):
if 0 <= d_idx < len(directions): target_dir = directions[d_idx]

d_idx = int(d_choice) - 1
if not d_choice or not d_choice.isdigit(): return None, None
d_choice = input(f"\n..... (1-{{limit}}): ").strip()
limit = len(directions) + 1 if allow_new else len(directions)

if allow_new: print(f" {{len(directions) + 1}}. [.....]")
for idx, d in enumerate(directions, 1): print(f" {{idx}}. {{d}}")
print("\n.....:")
if directions:

directions = list(db.keys())
target_dir = ""
"""....."""
def select_hierarchy_target(db, allow_new=False):

import config
import db_core
=== FILE: ./wizard_menus.py ===

main_menu()
if __name__ == "__main__":

```

```

print(f"\n .....: {direction}")
for direction, sub_objs in db.items():
    return
print("....")
if not db:
    print("\n=== ..... ===")
db = db_core.load_db()
def print_data():

import git_deployer
import wizard_menus
import db_core
-----
### .....: wizard_manager.py ###

=====
..... (..... JSON)
=====
=== FILE: ./....._..._.txt ===

print(f" [.....] ..... '{month}' ..... !")
db_core.update_config_months(month)
db_core.save_db(db)
db[target_dir][target_sub][month] = {"sum": 0.0, "status": {"value": "", "date": ""}}

if target_sub not in db[target_dir]: db[target_dir][target_sub] = {}
if target_dir not in db: db[target_dir] = {}

break

return
input("\n..... Enter...")
print(" ..... -> ..... 3 .....")
print(f"\n . [.....] ..... '{month}' ..... !")
if target_dir in db and target_sub in db[target_dir] and month in db[target_dir][target_sub]:

month = db_core.ALL_YEAR_MONTHS[mth_idx]

continue
print(" . [.....] ..... .. 1 .. 12!")
if mth_idx < 0 or mth_idx >= 12:
mth_idx = int(mth_choice) - 1

continue
print(" . [.....] ..... .. 1 .. 12!")
if not mth_choice or not mth_choice.isdigit():

return
if mth_choice == "0":
mth_choice = input("\n..... (1-12) ... '0' .....: ").strip()

print(f" {i}. {m_name}{marker}")
marker = " [... ..]" if m_name in active_months else ""
for i, m_name in enumerate(db_core.ALL_YEAR_MONTHS, 1):

active_months = list(db[target_dir][target_sub].keys())
if isinstance(db[target_dir][target_sub], dict):
if target_dir in db and target_sub in db[target_dir]:
active_months = []

print("\n.....:")

```

```

target_sub = ""

if not target_dir: return None, None

target_dir = input(".... ..: ").strip()
else:
target_dir = input(".....: ").strip()
elif allow_new and d_idx == len(directions):
if 0 <= d_idx < len(directions): target_dir = directions[d_idx]

d_idx = int(d_choice) - 1
if not d_choice or not d_choice.isdigit(): return None, None
d_choice = input(f"\n..... (1-{{limit}}): ").strip()
limit = len(directions) + 1 if allow_new else len(directions)

if allow_new: print(f" {{len(directions) + 1}}. [.....]")
for idx, d in enumerate(directions, 1): print(f" {{idx}}. {{d}}")
print("\n.....:")
if directions:

directions = list(db.keys())
target_dir = ""
"""....., """
def select_hierarchy_target(db, allow_new=False):

import db_core
-----
### ....: wizard_menus.py ###

-----

run_wizard()
if __name__ == "__main__":

print(" .....")
else:
print(".. .....!"); break
elif choice == "5":
git_deployer.run_production_pipeline(session_added_rows=["....."])
elif choice == "4":
wizard_menus.menu_edit_data()
elif choice == "3":
wizard_menus.menu_add_record()
elif choice == "2":
print_data()
if choice == "1":

choice = input("..... (1-5): ").strip()
print("5. ....")
print("4. .... Excel (. .... GitHub)")
print("3. ....")
print("2. .... (...../.....)")
print("1. .... (.....)")
print("\n--- ..... (JSON ....) ---")
while True:
def run_wizard():

print("=====")
{data.get('status', '...')}")
print(f" .. {{mth}}: {{db_core.fmt_money(data.get('sum', 0))}} .... | .....:
for mth, data in months.items():
print(f" .. .....: {{sub_obj}}")
for sub_obj, months in sub_objs.items():

```

```

months_in_db = list(db[target_dir][target_sub].keys())

if not target_dir or not target_sub: return
target_dir, target_sub = select_hierarchy_target(db, allow_new=False)

if sub_choice not in ("1", "2", "3"): return
sub_choice = input("..... (1-3): ").strip()
.....")
print(".....:\n 1. ....\n 2. ....\n 3.

if not db: return
db = db_core.load_db()
print("\n>> .....")
"""....., ..... (..... 3)."""
def menu_edit_data():

print(f" [.....] ..... '{month}' .....!")
db_core.update_config_months(month)
db_core.save_db(db)
db[target_dir][target_sub][month] = {"sum": 0.0, "status": {"value": "", "date": ""}}

if month in db[target_dir][target_sub]: return
if target_sub not in db[target_dir]: db[target_dir][target_sub] = {}
if target_dir not in db: db[target_dir] = {}

month = db_core.ALL_YEAR_MONTHS[mth_idx]
if mth_idx < 0 or mth_idx >= 12: return
mth_idx = int(mth_choice) - 1
if not mth_choice or not mth_choice.isdigit(): return
mth_choice = input("\n..... (1-12): ").strip()

print(f" {i}. {m_name}{marker}")
marker = " [... ..]" if m_name in active_months else ""
for i, m_name in enumerate(db_core.ALL_YEAR_MONTHS, 1):
db[target_dir]) else []
active_months = list(db[target_dir][target_sub].keys()) if (target_dir in db and target_sub in
print("\n.....:")

if not target_dir or not target_sub: return
target_dir, target_sub = select_hierarchy_target(db, allow_new=True)

db = db_core.load_db()
print("\n>> .....")
"""..... (..... 2)."""
def menu_add_record():

return target_dir, target_sub

target_sub = input(".....: ").strip()
else:
target_sub = input(".....: ").strip()
elif allow_new and s_idx == len(sub_objs):
if 0 <= s_idx < len(sub_objs): target_sub = sub_objs[s_idx]

s_idx = int(s_choice) - 1
if not s_choice or not s_choice.isdigit(): return None, None
s_choice = input(f"..... (1-{limit}): ").strip()
limit = len(sub_objs) + 1 if allow_new else len(sub_objs)

if allow_new: print(f" {len(sub_objs) + 1}. [.....]")
for idx, s in enumerate(sub_objs, 1): print(f" {idx}. {s}")
print(f"\n..... '{target_dir}':")
sub_objs = list(db[target_dir].keys())
if target_dir in db and db[target_dir]:

```

```

import config
import datetime
import json
import os
-----
### .....: db_core.py ###

-----

print(" [.....] ..... .")
db_core.save_db(db)
db[target_dir][target_sub][target_mth]["status"] = {"value": "", "date": ""}
elif st_choice == "0":
print(" [.....] ..... .")
db_core.save_db(db)
db_core.get_short_date()}
db[target_dir][target_sub][target_mth]["status"] = {"value": int(st_choice), "date":
if st_choice in ("1", "2", "3"):
st_choice = input("..... (0-3): ").strip()
print("\n..... : \n 1. .... | 2. .... | 3. .... | 0. ....")
elif sub_choice == "3":

db_core.update_config_months(new_mth_name)
db_core.save_db(db)
db[target_dir][target_sub][new_mth_name] = db[target_dir][target_sub].pop(target_mth)

return
db_core.save_db(db)
db[target_dir][target_sub][new_mth_name]["sum"] += float(old.get("sum", 0))
old = db[target_dir][target_sub].pop(target_mth)
if input().strip() == "1":
print(" .. .... ! .....? (1-... / 2-....)")
if new_mth_name in db[target_dir][target_sub]:

new_mth_name = db_core.ALL_YEAR_MONTHS[int(new_mth_choice) - 1]
if not new_mth_choice or not new_mth_choice.isdigit(): return
new_mth_choice = input("\n..... (1-12): ").strip()
for i, m_name in enumerate(db_core.ALL_YEAR_MONTHS, 1): print(f" {i}. {m_name}")
print("\n..... :")
elif sub_choice == "2":

except: print(" [.....] ..... .")
print(" [.....] ..... .")
db_core.save_db(db)
db[target_dir][target_sub][target_mth]["sum"] = float(new_sum_str.replace(",", "."))
try:
new_sum_str = input(f"\n..... : ").strip()
if sub_choice == "1":

target_mth = months_in_db[m_idx]
if m_idx < 0 or m_idx >= len(months_in_db): return
m_idx = int(m_choice) - 1
if not m_choice or not m_choice.isdigit(): return
m_choice = input("\n..... : ").strip()

.... | .....: {st_val}"))
print(f" {idx}. {m} (.....: {db_core.fmt_money(db[target_dir][target_sub][m].get('sum', 0))}
"...")
st_val = st_raw.get("value", "...") if isinstance(st_raw, dict) else (st_raw if st_raw else
st_raw = db[target_dir][target_sub][m].get("status", "")
for idx, m in enumerate(months_in_db, 1):
print(f"\n..... :")

```

```

import datetime
import os
import json
import openpyxl
-----
### .....: builder.py ###

-----

print(f" [.....] ..... '{new_month}' ..... config.py")
f.write(code)
with open(config_path, "w", encoding="utf-8") as f:

code = re.sub(r"MONTHS_LIST\s*=\s*\[.*?\]", new_line_repr, code)
import re
else:
code = code.replace(old_line_repr, new_line_repr)
if old_line_repr in code:

new_line_repr = f"MONTHS_LIST = {repr(updated_months)}"
old_line_repr = f"MONTHS_LIST = {repr(config.MONTHS_LIST)}"
updated_months = [m for m in ALL_YEAR_MONTHS if m in config.MONTHS_LIST or m == new_month]

code = f.read()
with open(config_path, "r", encoding="utf-8") as f:
config_path = "config.py"
if new_month not in config.MONTHS_LIST:
"""..... config.py ....."""
def update_config_months(new_month):

json.dump(db, f, ensure_ascii=False, indent=4)
with open(json_path, "w", encoding="utf-8") as f:
json_path = "database.json"
def save_db(db):

return {}
return json.load(f)
with open(json_path, "r", encoding="utf-8") as f:
if os.path.exists(json_path):
json_path = "database.json"
def load_db():

return datetime.datetime.now().strftime("%d.%m")
"""....."""
def get_short_date():

return str(value)
except:
return f"{thousand_part},{parts}"
thousand_part = f"{int(parts):_}".replace("_", " ")
parts = f"{val_float:.2f}".split(".")
else:
return f"{int(val_float):_}".replace("_", " ")
if val_float.is_integer():
val_float = float(value)
try:
"""....."""
def fmt_money(value):

]
".....", ".....", ".....", ".....", ".....", "....."
".....", ".....", ".....", ".....", ".....", ".....",
ALL_YEAR_MONTHS = [

```

```

doc_row = ws.max_row
ws.append(["", f". {d_name}", "", "", "", "", "", "", "", ""])
for d_name in config.DOCUMENTS_LIST:
# .....: ..... 3

row_counter += 1
ws.row_dimensions[current_row].outline_level = 2
ws.cell(row=current_row, column=3).number_format = '#,##0.00'
ws.cell(row=current_row, column=1).alignment = config.ALIGN_C
config.THIN_BORDER, config.ALIGN_L)
excel_styler.apply_row_style(ws, current_row, config.FONT_MTH, config.FILL_MTH,
current_row = ws.max_row
ws.append([row_counter, mth, float(mth_data.get("sum", 0.0)), "", "", "", "", "", ""])
# .....: ..... 2

status_date = status_raw.get("date", "") if isinstance(status_raw, dict) else ""
status_raw = mth_data.get("status", "")
# ..... ..

mth_data = db[direction][sub_obj][mth]
if mth not in db[direction][sub_obj]: continue
for mth in config.MONTHS_LIST:

ws.row_dimensions[ws.max_row].outline_level = 1
config.THIN_BORDER, config.ALIGN_L)
excel_styler.apply_row_style(ws, ws.max_row, config.FONT_OBJ, config.FILL_OBJ,
ws.append(["", str(sub_obj), "", "", "", "", "", "", ""])
for sub_obj in db[direction].keys():

ws.row_dimensions[ws.max_row].outline_level = 0
config.THIN_BORDER, config.ALIGN_L)
excel_styler.apply_row_style(ws, ws.max_row, config.FONT_DIR, config.FILL_DIR,
ws.append(["", str(direction), "", "", "", "", "", "", ""])
for direction in db.keys():

row_counter = 1

db = json.load(f)
with open(json_path, "r", encoding="utf-8") as f:

if not os.path.exists(json_path): return ws.max_row
json_path = "database.json"

config.ALIGN_C)
excel_styler.apply_row_style(ws, 1, config.FONT_HDR, config.FILL_HDR, config.THIN_BORDER,
ws.append(config.HEADERS)
if ws.max_row == 1 and ws.cell(row=1, column=1).value is None:

ws.oddHeader.right.color = "7A7A7A"
ws.oddHeader.right.size = 9
ws.oddHeader.right.text = f".....: {now_str}"
now_str = datetime.datetime.now().strftime("%d.%m.%Y %H:%M")

ws.sheet_properties.outlinePr.summaryBelow = False
ws.freeze_panes = "A2"
"""..... 5-... .."""
def build_structure(ws, mock_data=None, saved_statuses=None, saved_sums=None):

excel_styler.apply_row_style(ws, row_idx, font, fill, border, alignment)
"""..... .. main.py"""
def apply_row_style(ws, row_idx, font, fill, border, alignment=None):

import excel_styler
import config

```

```

if status_val == "" or status_val is None:

status_val = status_raw
else:
status_val = status_raw.get("value", "")
if isinstance(status_raw, dict):

status_val = ""
"""..... . ..... . ..... . ..... . ..... . ..... ."""
def format_status_cell(cell, status_raw):

if alignment: cell.alignment = alignment
if border: cell.border = border
if fill: cell.fill = fill
if font: cell.font = font
cell = ws.cell(row=row_idx, column=col)
for col in range(1, ws.max_column + 1):
"""..... . ..... . ..... . ..... . ..... ."""
def apply_row_style(ws, row_idx, font, fill, border, alignment=None):

FONT_LOG_DATE = openpyxl.styles.Font(name="Calibri", size=9, italic=True, color="7A7A7A")
# ..... . ..... . ..... . ..... 5-.. ..... . ..... .

import config
import openpyxl
-----
### .....: excel_styler.py ###

-----

return ws.max_row

ws.cell(row=row, column=10).value = f"=IF(C{row}>0, \" . .....\", \"\")"
if ws.cell(row=row, column=2).value in config.MONTHS_LIST:
for row in range(2, ws.max_row + 1):

ws.row_dimensions[log_row].outline_level = 4
# ..... . ..... . ..... 4 (. Excel ... ..... 5-.. ..... . ..... .)
config.ALIGN_L)
excel_styler.apply_row_style(ws, log_row, excel_styler.FONT_LOG_DATE, None, config.THIN_BORDER,
# ..... . ..... . ..... . ..... . ..... .
log_row = ws.max_row
ws.append(["", f" .. .... : {status_date}", "", "", "", "", "", "", "", "", ""])
if mask.get(".....", 1) == 1 and status_date:
# ..... 5-.. .....: ..... . ..... . ..... . ..... . ..... . ..... .

excel_styler.format_status_cell(cell, status_raw)
if col_name == ".....":
else:
cell.value = ""
cell.fill = config.FILL_BLOCKED
if mask.get(col_name, 1) == 0:
cell = ws.cell(row=doc_row, column=col_idx)
for col_name, col_idx in column_mapping.items():

column_mapping = {".....": 4, "...": 5, ".....": 6, "1 .... .": 7, "1 .... .": 8, ".....": 9}
.... .": 1, "...": 1})
mask = config.DOCUMENT_ROLES.get(d_name, {".....": 1, "...": 1, ".....": 1, "1 .... .": 1, "1

ws.row_dimensions[doc_row].outline_level = 3
ws.cell(row=doc_row, column=3).number_format = '#,##0.00'
config.ALIGN_L)
excel_styler.apply_row_style(ws, doc_row, config.FONT_DATA, None, config.THIN_BORDER,

```



```

# 1 - ..... (.....), 0 - ..... (.....)
# == ..... (.....) ==

".....", "1 .... .", "1 .... .", "....", "..... ....."]
HEADERS = ["./.", "..... / ..... / .....", "..... (....)", ".....", "....",

ALIGN_R = Alignment(horizontal="right", vertical="center")
ALIGN_L = Alignment(horizontal="left", vertical="center", wrap_text=True)
ALIGN_C = Alignment(horizontal="center", vertical="center", wrap_text=True)

color="B0B0B0"))
color="B0B0B0"), top=Side(style="thin", color="B0B0B0"), bottom=Side(style="thin",
THIN_BORDER = Border(left=Side(style="thin", color="B0B0B0"), right=Side(style="thin",

FILL_MTH = PatternFill(start_color="EAEAEA", end_color="EAEAEA", fill_type="solid")
FILL_OBJ = PatternFill(start_color="D9D9D9", end_color="D9D9D9", fill_type="solid")
FILL_HDR = FILL_DIR = PatternFill(start_color="1F4E78", end_color="1F4E78", fill_type="solid")

FONT_DATA = Font(name="Calibri", size=11)
FONT_MTH = Font(name="Calibri", size=11, bold=True, italic=True, color="000000")
FONT_OBJ = Font(name="Calibri", size=11, bold=True, color="000000")
FONT_DIR = Font(name="Calibri", size=12, bold=True, color="FFFFFF")
FONT_HDR = Font(name="Calibri", size=11, bold=True, color="FFFFFF")

FULL_PATH = os.path.join(FOLDER_NAME, FILE_NAME)
FILE_NAME = "....._..._.xlsx"
FOLDER_NAME = ""

.....']
DOCUMENTS_LIST = ['... -2', '..... -3', '.....', '....', '.....']
MONTHS_LIST = ['.....', '.....', '.....', '.....']

]
('02_.....', '.....', '....', 0, None),
('02_.....', '.....', '.....', 850000, 2),
('01_..... (.....)', '04_537_..', '....', 0, None),
('01_..... (.....)', '04_537_..', '.....', 0, None),
('01_..... (.....)', '03_538_..', '....', 0, None),
('01_..... (.....)', '03_538_..', '.....', 0, None),
('01_..... (.....)', '02_1505_..', '....', 0, None),
('01_..... (.....)', '02_1505_..', '.....', 450000, 3),
('01_..... (.....)', '01_271_..', '....', 0, None),
('01_..... (.....)', '01_271_..', '.....', 450000, 3),
DATA_ROWS = [

from openpyxl.styles import Font, Alignment, PatternFill, Border, Side
import os
-----
### ....: config.py ###
-----

return cell.value
cell.alignment = config.ALIGN_C
cell.font = config.FONT_DATA

cell.value = str(status_val)
except ValueError:
cell.value = int(status_val)
# .....: ..... int, ..... Excel
try:

return ""

```

```

return "....."

pass
except Exception:
return f".....: {'', '.join(set(months_edited))}"
if months_edited:
months_edited.append(v["label"].strip())
elif isinstance(v, dict) and "label" in v:
months_edited.append(k.split("||")[-1])
if "||" in k:
for k, v in changes.items():
months_edited = []
if changes:
changes = json.load(f)
with open(json_path, "r", encoding="utf-8") as f:
try:
if os.path.exists(json_path) and os.path.getsize(json_path) > 2:
json_path = "changes.json"

return f".....: {'', '.join(directions)} ({'', '.join(months)})"
months = set([r[2] for r in session_added_rows])
directions = set([r[0] for r in session_added_rows])
if session_added_rows:
def generate_auto_changelog(session_added_rows=None):

return "v2.1"
except Exception:
return "v2.1"
return f"v{''.join(parts)}"
parts[-1] = str(int(parts[-1]) + 1)
parts = version[1:].split('.')
if version.startswith('v'):
version = res.stdout.strip()
text=True, check=True)
res = subprocess.run(["git", "describe", "--tags", "--abbrev=0"], capture_output=True,
try:
def get_current_git_version():

import config
import subprocess
import json
import os
-----
### .....: git_deployer.py ###
-----

FILL_BLOCKED = PatternFill(start_color="EAEAEA", end_color="EAEAEA", fill_type="solid")
# .....

}
".....": ROLE_EXEC_DOC
"....": ROLE_INVOICE,
".....": ROLE_INVOICE,
"..... -3": ROLE_KS3,
"... -2": ROLE_KS2,
DOCUMENT_ROLES = {

ROLE_EXEC_DOC = {"....": 1, "...": 0, ".....": 0, "1 .... .": 1, "1 .... .": 1, "....": 0}
ROLE_INVOICE = {"....": 0, "...": 1, ".....": 0, "1 .... .": 1, "1 .... .": 1, "....": 0}
ROLE_KS3 = {"....": 0, "...": 1, ".....": 1, "1 .... .": 1, "1 .... .": 1, "....": 0}
ROLE_KS2 = {"....": 1, "...": 1, ".....": 1, "1 .... .": 1, "1 .... .": 1, "....": 0}

```

```

import formatter
import builder
import storage
import config
import openpyxl
import os
-----
### .....: main.py ###

-----

return True
clear_changes_json()
print(" [...] ..... .. Mac. ....")
else:
return False
print(" [..... GIT] .. ..... . ....")
else:
return True
clear_changes_json()
print(" [...] ... .. .. GitHub!")
if push_res.returncode == 0:

push_res = subprocess.run(["git", "push", "origin", "main", "--tags"])
subprocess.run(["git", "tag", "-a", final_ver, "-m", final_logs])
subprocess.run(["git", "commit", "-m", commit_message])
subprocess.run(["git", "add", "."])

print(f"\n [GIT] ..... : \"{commit_message}\"")
commit_message = f"{final_ver} . {final_logs}"

final_logs = user_logs if user_logs else default_logs
user_logs = input(f"..... [...] .....: {default_logs}"): ").strip()

final_ver = user_ver if user_ver else default_ver
user_ver = input(f"..... [...] .....: {default_ver}"): ").strip()
print("\n ..... :")

default_logs = generate_auto_changelog(session_added_rows)
default_ver = get_current_git_version()
if push_choice in (".", "y", "yes"):

).strip().lower()
push_choice = input("..... GitHub? (../.):")
print("\n [...] .. 2: .....")

return False
print(" [...] ..... . ....")
if result.returncode != 0:
result = subprocess.run(["python3", "main.py"])
print("\n [...] .. 1: .....")
def run_production_pipeline(session_added_rows=None):

print(f" [...] .. ..... changes.json: {e}")
except Exception as e:
print(" [...] ..... changes.json .....")
json.dump({}, f)
with open(json_path, "w", encoding="utf-8") as f:
try:
if os.path.exists(json_path):
json_path = "changes.json"
def clear_changes_json():

```

```

"date": "01.07"
"value": 2,
"status": {
"sum": 17730436.92,
"....": {
"01_271_..": {
"01_..... (.....)": {
{
-----
### .....: database.json ###

-----

run()
if __name__ == "__main__":

print(f". [.....] .....: {e}")
except Exception as e:
print(f". [.....] .... Excel/LibreOffice .....! .....")
except PermissionError:

.....!")
print(f". [.....] .... .., .. ..... ..")
else:
print(f". [.....] ..... ..!")
print(f".. ..: {file_size_kb} ..")
print(f"..: {os.path.abspath(file_path)}")
print(f". [.....] ..... ..!")
file_size_kb = round(os.path.getsize(file_path) / 1024, 1)
if os.path.exists(file_path) and os.path.getsize(file_path) > 0:
# .....

wb.save(file_path)
try:
# 5. ....

formatter.set_column_widths(ws)
formatter.apply_conditional_formatting(ws)
formatter.append_specifications(ws, last_row)
last_row = builder.build_structure(ws, mock_data, saved_statuses, saved_sums)
# 4. ....

config.ALIGN_C)
builder.apply_row_style(ws, 1, config.FONT_HDR, config.FILL_HDR, config.THIN_BORDER,
ws.row_dimensions.height = 28
ws.append(config.HEADERS)
# 3. ....

ws.sheet_properties.outlinePr.summaryBelow = False
ws.sheet_view.showOutlineSymbols = True
ws.sheet_view.showGridLines = True
ws.title = "....."
ws = wb.active
wb = openpyxl.Workbook()
# 2. ....

mock_data = storage.group_linear_data(config.DATA_ROWS)
saved_statuses, saved_sums = storage.load_historical_data(file_path)
# 1. ....

file_path = config.FULL_PATH if config.FULL_PATH else "....._..._.xlsx"
print(f".. [.....] .....")
def run():

```

```
- .....: openpyxl (..... Excel)
- .....: Python 3
....., ..... (---2, ---3).
.....
1. ....
```

```
=====
.....
=====
=== FILE: ./....._......txt ===
```

```
-----
}
}
}
}
"status": ""
"sum": 0,
".....": {
},
"status": 2
"sum": 850000,
".....": {
".....": {
"02_.....": {
},
}
}
"status": ""
"sum": 0,
".....": {
},
"status": ""
"sum": 0,
".....": {
"04_537_..": {
},
}
"status": ""
"sum": 0,
".....": {
},
"status": ""
"sum": 0,
".....": {
"03_538_..": {
},
}
"status": ""
"sum": 0,
".....": {
},
"status": 3
"sum": 450000,
".....": {
"02_1505_..": {
},
}
"status": ""
"sum": 0.0,
".....": {
},
}
```

```

- ..... `.gitignore`, ..... GitHub .....
- ..... 5- .....
- ..... `TextBlock` · `append` · ..... `RichText` .....
- .....
- ..... , .....
- ..... (.....).
- ..... macOS · openpyxl:
6. .... MAC

- `config.py` · .....
- `git_deployer.py` · ..... GitHub .....
- .....
- `excel_styler.py` · .....
- .....
- ..... (`freeze_panes`) · .....
- `builder.py` · ..... Excel. .... JSON .....
- .....
- `db_core.py` · ...../..... JSON, .....
- .....
- ..... (..... 1-12, .....
- `wizard_menus.py` · .....
- ..... (..... 1-5).
- `wizard_manager.py` · .....
- .....
- .....
5. ....

..... LibreOffice Calc.
..... (1, 2, 3), ..
- ..... (`#EAEAEA`).
- .....
- .....), .....:
- ..... (....., .....
- ..... (.....).
- ..... , · `config.py` .....
4. .... (.....)

01.07")
· ..... 4: ..... (".....:
- ..... , ..)
· ..... 3: ..... (.....: .....-2, ..... -3, .....,
· ..... 2: ..... (....., .....-.....)
· ..... 1: ..... (.....)
· ..... 0: ..... (.....)
- ..... B: .....:
- .....
- ..... A: ..... , .....
- ..... (Outline Level) .. 0 .. 4:
- ..... Excel-..... JSON. ....
3. .... EXCEL (5 ..... OUTLINE)

... status (dict: {"value": 3, "date": "01.07"})
... sum (float: .....
... ..... (".....")
... ..... ("01_271_..")
... ..... ("01_..... (.....)")
... ..... (.....):
- ..... Excel-..... `database.json`.
- ..... ".....".
2. .... (Single Source of Truth)

- ..... LibreOffice Calc / MS Excel (macOS)
- ...../: ..... GitHub
- .....: ..... JSON

```

```

meta = db.get("_meta", {}) if isinstance(db, dict) else {}
# .... 1: ..... ..

db = json.load(f)
with open(json_path, "r", encoding="utf-8") as f:
    return ws.max_row
if not os.path.exists(json_path):
    json_path = "database.json"

cell.border = config.THIN_BORDER
cell.font = config.FONT_HDR
cell.fill = config.FILL_HDR
cell = ws.cell(row=2, column=col_idx)
for col_idx in range(1, len(config.HEADERS) + 1):
    ws.append(config.HEADERS)
if ws.max_row == 1 and ws.cell(row=1, column=1).value is None:

ws.oddHeader.right.color = "7A7A7A"
ws.oddHeader.right.size = 9
ws.oddHeader.right.text = f".....: {now_str}"
now_str = datetime.datetime.now().strftime("%d.%m.%Y %H:%M")

ws.sheet_properties.outlinePr.summaryBelow = False
ws.freeze_panes = "A3"
def build_structure(ws, mock_data=None, saved_statuses=None, saved_sums=None):

..... Cmd+S!")
print(" [....] ..... LibreOffice ....., ....., ... .. builder.py
{width_val}")
print(f" - ..... {letters[col_idx]} (..... {col_idx}): ..... =
width_val = 14.0
for col_idx in columns_to_check:
print(" ..... openpyxl:")
letters = {4: "D", 5: "E", 6: "F", 7: "G", 8: "H"}
columns_to_check = [4, 5, 6, 7, 8] # D, E, F, G, H
def test_columns_width_logic():

excel_styler.apply_row_style(ws, row_idx, font, fill, border, alignment)
def apply_row_style(ws, row_idx, font, fill, border, alignment=None):

import excel_styler
import db_viewer
import db_core
import config
import datetime
import os
import json
import openpyxl

logging.getLogger().setLevel(logging.WARNING)
import logging
=== FILE: ./builder.py ===

=====
....., .....!
=====

`.xlsx` . ..... `__pycache__`.

```

```

hidden_default = False
if is_new and direction == target_dir and sub_obj == target_sub:
hidden_default = True
# ..... (.... 2)

row_counter += 1
ws.row_dimensions[current_row].outline_level = 2
ws.cell(row=current_row, column=3).number_format = "#,##0.00"
ws.cell(row=current_row, column=1).alignment = config.ALIGN_C
)
config.ALIGN_L,
config.THIN_BORDER,
config.FILL_MTH,
config.FONT_MTH,
current_row,
ws,
excel_styler.apply_row_style(
current_row = ws.max_row
)
[row_counter, mth, float(mth_data.get("sum", 0.0)), "", "", "", "", "", "", ""]
ws.append(
# .....: ..... 2

status_raw = mth_data.get("status", "")
mth_data = db[direction][sub_obj][mth]
for mth in available_months:

available_months.append(m)
if m not in available_months:
for m in raw_months:

available_months = [m for m in db_core.ALL_YEAR_MONTHS if m in raw_months]
raw_months = list(db[direction][sub_obj].keys())

ws.row_dimensions[ws.max_row].outline_level = 1
)
config.ALIGN_L,
config.THIN_BORDER,
config.FILL_OBJ,
config.FONT_OBJ,
ws.max_row,
ws,
excel_styler.apply_row_style(
ws.append(["", str(sub_obj), "", "", "", "", "", "", "", ""])
for sub_obj in db[direction].keys():

ws.row_dimensions[ws.max_row].outline_level = 0
)
config.ALIGN_L,
config.THIN_BORDER,
config.FILL_DIR,
config.FONT_DIR,
ws.max_row,
ws,
excel_styler.apply_row_style(
ws.append(["", str(direction), "", "", "", "", "", "", "", ""])
continue
if direction == "_meta":
for direction in db.keys():
row_counter = 1

is_new = meta.get("is_new_change", False)
target_sub = meta.get("last_changed_sub")
target_dir = meta.get("last_changed_dir")

```



```

"""
"""
f"      .. .... : {status_date}",
"""
[
ws.append(
if mask.get("....", 1) == 1 and status_date:
# ..... : ..... 4

cell.fill = openpyxl.styles.PatternFill(fill_type=None)
else:
excel_styler.format_status_cell(cell, clean_status_value)
if clean_status_value is not None:
else:
cell.value = ""
cell.fill = config.FILL_BLOCKED
if mask.get(col_name) == 0:

clean_status_value = doc_status_raw if col_name == "...." else None
else:
)
else doc_status_raw.get("value")
if col_name in doc_status_raw
doc_status_raw.get(col_name, {}).get("value")
clean_status_value = (
if isinstance(doc_status_raw, dict):

status_date = doc_status_raw.get("date", "") if isinstance(doc_status_raw, dict) else ""

doc_status_raw = status_raw
else:
doc_status_raw = status_raw[d_name]
if isinstance(status_raw, dict) and d_name in status_raw:
# ..... : ...../....

cell = ws.cell(row=doc_row, column=col_idx)

col_idx = int(col_letter_or_idx)
else:
col_idx = openpyxl.utils.column_index_from_string(col_letter_or_idx)
if isinstance(col_letter_or_idx, str):
for col_name, col_letter_or_idx in config.COLUMN_MAPPING.items():

)
{"....": 1, "...": 1, ".....": 1, "1 .... .": 1, "1 .... .": 1, "....": 1},
d_name,
mask = config.DOCUMENT_ROLES.get(

ws.row_dimensions[doc_row].outline_level = 3
ws.cell(row=doc_row, column=3).number_format = "#,##0.00"
)
config.ALIGN_L,
config.THIN_BORDER,
None,
config.FONT_DATA,
doc_row,
ws,
excel_styler.apply_row_style(
doc_row = ws.max_row
ws.append(["", f". {d_name}", "", "", "", "", "", "", "", ""])
for d_name in config.DOCUMENTS_LIST:
# ..... : ..... 3

ws.row_dimensions[current_row].hidden = hidden_default

```

```

return ws.max_row

db_core.save_db(db)
db["_meta"]["is_new_change"] = False
if isinstance(db, dict) and "_meta" in db:
# .... 3: ..... (.....)

ws.column_dimensions[col_letter].width = max(max_len + 4, 12)
else:
ws.column_dimensions[col_letter].width = max(max_len + 4, 18)
elif col_idx == 3:
ws.column_dimensions[col_letter].width = max(max_len + 5, 45)
if col_idx == 2:

max_len = cell_len
if cell_len > max_len:
cell_len = max(len(sub_line) for sub_line in str(cell.value).split("\n"))
if cell.value is not None:
for cell in col:
max_len = 0
else:
ws.column_dimensions[col_letter].auto_fit = False
ws.column_dimensions[col_letter].width = 8.0
# ..... 1.35 ..
if col_letter in status_letters:

col_idx = first_cell.column
col_letter = first_cell.column_letter
first_cell = col[0]
for col in ws.columns:
status_letters = ["D", "E", "F", "G", "H"]
# .....

ws.row_dimensions[row_idx].hidden = True
if ws.row_dimensions[row_idx].outline_level in (3, 4):
for row_idx in range(2, ws.max_row + 1):
ws.sheet_view.showOutlineSymbols = True
# .....

ws.cell(row=row, column=10).value = f"=IF(C{row}>0, \".....\", \"\")"
if cell_val and not str(cell_val).startswith(".") and not str(cell_val).startswith(" "):
cell_val = ws.cell(row=row, column=2).value
for row in range(2, ws.max_row + 1):
# .....

ws.row_dimensions[log_row].outline_level = 4
)
config.ALIGN_L,
config.THIN_BORDER,
None,
excel_styler.FONT_LOG_DATE,
log_row,
ws,
excel_styler.apply_row_style(
log_row = ws.max_row
)
]
"""
"""
"""
"""
"""
"""
"""
"""

```

```
( '02_.....', '.....', '....', 0, None),
( '02_.....', '.....', '.....', 850000, 2),
( '01_.... (.....)', '04_537_..', '....', 0, None),
( '01_.... (.....)', '04_537_..', '.....', 0, None),
( '01_.... (.....)', '03_538_..', '....', 0, None),
( '01_.... (.....)', '03_538_..', '.....', 0, None),
( '01_.... (.....)', '02_1505_..', '....', 0, None),
( '01_.... (.....)', '02_1505_..', '.....', 450000, 3),
( '01_.... (.....)', '01_271_..', '....', 0, None),
( '01_.... (.....)', '01_271_..', '.....', 450000, 3),
DATA_ROWS = [
```

```
from openpyxl.styles import Font, Alignment, PatternFill, Border, Side
import os
=== FILE: ./config.py ===
```

```
}
}
"sum": 0.0
"month": "....",
"sub_obj": ".....",
"direction": "02_.....",
"02_.....||.....||....": {
},
"sum": 850000.0
"month": ".....",
"sub_obj": ".....",
"direction": "02_.....",
"02_.....||.....||.....": {
},
"sum": 0.0
"month": "....",
"sub_obj": "04_537_..",
"direction": "02_1505_..",
"02_1505_..||04_537_..||....": {
},
"sum": 0.0
"month": ".....",
"sub_obj": "04_537_..",
"direction": "02_1505_..",
"02_1505_..||04_537_..||.....": {
},
"sum": 0.0
"month": "....",
"sub_obj": "03_538_..",
"direction": "02_1505_..",
"02_1505_..||03_538_..||....": {
},
"sum": 0.0
"month": ".....",
"sub_obj": "03_538_..",
"direction": "02_1505_..",
"02_1505_..||03_538_..||.....": {
},
"sum": 13.0
"month": "....",
"sub_obj": "... .. : 02.07",
"direction": "01_271_..",
"01_271_..||... .. : 02.07||....": {
{
=== FILE: ./changes.json ===
```

```

# === .....

FILL_BLOCKED = PatternFill(start_color="EAEAEA", end_color="EAEAEA", fill_type="solid")
# .....

}
".....": ROLE_EXEC_DOC,
"....": ROLE_INVOICE,
".....": ROLE_INVOICE,
"..... -3": ROLE_KS3,
"... -2": ROLE_KS2,
DOCUMENT_ROLES = {

ROLE_EXEC_DOC = {"....": 1, "...": 0, ".....": 0, "1 .... .": 1, "1 .... .": 1, "....": 0}
ROLE_INVOICE = {"....": 0, "...": 1, ".....": 0, "1 .... .": 1, "1 .... .": 1, "....": 0}
ROLE_KS3 = {"....": 0, "...": 1, ".....": 1, "1 .... .": 1, "1 .... .": 1, "....": 0}
ROLE_KS2 = {"....": 1, "...": 1, ".....": 1, "1 .... .": 1, "1 .... .": 1, "....": 0}
# 1 - ..... (.....), 0 - ..... (.....)
# === ..... (.....) ===

]
".....",
"....",
"1 .... .",
"1 .... .",
".....",
"....",
"....",
"..... (.....)",
"..... / ..... / .....",
". ./.",
HEADERS = [

ALIGN_R = Alignment(horizontal="right", vertical="center")
ALIGN_L = Alignment(horizontal="left", vertical="center", wrap_text=True)
ALIGN_C = Alignment(horizontal="center", vertical="center", wrap_text=True)

)
bottom=Side(style="thin", color="B0B0B0"),
top=Side(style="thin", color="B0B0B0"),
right=Side(style="thin", color="B0B0B0"),
left=Side(style="thin", color="B0B0B0"),
THIN_BORDER = Border(

FILL_MTH = PatternFill(start_color="EAEAEA", end_color="EAEAEA", fill_type="solid")
FILL_OBJ = PatternFill(start_color="D9D9D9", end_color="D9D9D9", fill_type="solid")
FILL_HDR = FILL_DIR = PatternFill(start_color="1F4E78", end_color="1F4E78", fill_type="solid")

FONT_DATA = Font(name="Calibri", size=11)
FONT_MTH = Font(name="Calibri", size=11, bold=True, italic=True, color="000000")
FONT_OBJ = Font(name="Calibri", size=11, bold=True, color="000000")
FONT_DIR = Font(name="Calibri", size=12, bold=True, color="FFFFFF")
FONT_HDR = Font(name="Calibri", size=11, bold=True, color="FFFFFF")

FULL_PATH = os.path.join(FOLDER_NAME, FILE_NAME)
FILE_NAME = "....._..._.xlsx"
FOLDER_NAME = ""

.....']
DOCUMENTS_LIST = ['... -2', '..... -3', '.....', '.....', '.....']
MONTHS_LIST = ['.....', '.....', '.....', '.....', '.....', '.....', '.....']

]

```

```

"yellow": FILL_YELLOW,
"green": FILL_GREEN,
Z1_STATUS_COLORS = {
# ..... 1 ....

}
"red": FILL_SDO_RED,
"yellow": FILL_SDO_YELLOW,
"green": FILL_SDO_GREEN,
SDO_STATUS_COLORS = {
# ..... (.....)

FILL_SDO_RED = FILL_RED
FILL_SDO_YELLOW = FILL_YELLOW
FILL_SDO_GREEN = FILL_GREEN
# .....
# === ..... (.....) ===

FILL_RED = PatternFill(start_color="FFC7CE", end_color="FFC7CE", fill_type="solid")
FILL_YELLOW = PatternFill(start_color="FFEB9C", end_color="FFEB9C", fill_type="solid")
FILL_GREEN = PatternFill(start_color="C6EFCE", end_color="C6EFCE", fill_type="solid")
# .....

VERSION = "v2. ...."

}
"1 .... ": ROLE_CUSTOMER,
"1 .... ": ROLE_CUSTOMER,
".....": ROLE_GEN_DIR,
"...": ROLE_KS2, # ... .. ROLE_KS2/ROLE_KS3 .....
ROLES = {

}
".....": 1,
"....": 1,
".....": 1,
".....-3": 1,
"...-2": 1,
ROLE_CUSTOMER = {
# .....

}
".....": 0,
"....": 1,
".....": 0,
".....-3": 1,
"...-2": 1,
ROLE_GEN_DIR = {
# ..... "....." .....
# ..... ROLE_KS2/ROLE_KS3/ROLE_INVOICE/ROLE_EXEC_DOC.
# === ..... ===

}
"1 .... ": "H",
"1 .... ": "G",
".....": "F",
"...": "E",
"....": "D",
COLUMN_MAPPING = {
# ....: D, ....: E, .....: F

```

```

},
}
"date": "08.07"
"value": 3,
"1 .... ": {
},
"date": "08.07"
"value": 3,
"1 .... ": {
},
"date": "08.07"
"value": 3,
".....": {
},
"date": "08.07"
"value": 3,
"...": {
"date": "02.07",
"value": 2,
"..... --3": {
},
}
"date": "08.07"
"value": 3,
"1 .... ": {
},
"date": "08.07"
"value": 3,
"1 .... ": {
},
"date": "08.07"
"value": 2,
".....": {
},
"date": "08.07"
"value": 1,
"...": {
},
"date": "08.07"
"value": 1,
".....": {
"date": "02.07",
"value": 2,
"... --2": {
"status": {
"sum": 3045600.78,
".....": {
"01_271_..": {
"01_..... (.....)": {
{
=== FILE: ./database.json ===

```

```

}
"red": FILL_RED,
"yellow": FILL_YELLOW,
"green": FILL_GREEN,
P1_STATUS_COLORS = {
# ..... 1 .... .
}
"red": FILL_RED,

```

```
"....": {
"... -2": {
"status": {
"sum": 4142769.05,
"....": {
"02_1505_..": {
},
}
}
"date": "07.07"
"value": 1,
"status": {
"sum": 13.0,
"....": {
},
}
}
"date": "08.07"
"value": 3,
"1 .... .": {
},
"date": "08.07"
"value": 3,
"1 .... .": {
},
"date": "08.07"
"value": 3,
"....": {
"date": "02.07",
"value": 2,
".....": {
},
}
"date": "08.07"
"value": 3,
"1 .... .": {
},
"date": "08.07"
"value": 3,
"1 .... .": {
},
"date": "08.07"
"value": 3,
"....": {
"date": "02.07",
"value": 2,
"....": {
},
}
"date": "08.07"
"value": 3,
"1 .... .": {
},
"date": "08.07"
"value": 3,
"1 .... .": {
},
"date": "08.07"
"value": 3,
"....": {
"date": "02.07",
"value": 2,
".....": {
```

```
"....": {
},
}
"date": ""
"value": "",
"status": {
"sum": 94859348.59,
"...": {
"01_.....": {
"03_.....3_.....": {
},
}
}
"status": ""
"sum": 0,
"....": {
},
"status": 2
"sum": 850000,
".....": {
".....": {
"02_.....": {
},
}
}
"status": ""
"sum": 0,
"....": {
},
"status": ""
"sum": 0,
".....": {
"04_537_..": {
},
}
"status": ""
"sum": 0,
"....": {
},
"status": ""
"sum": 0,
".....": {
"03_538_..": {
},
}
}
"status": ""
"sum": 0,
"....": {
},
}
}
"date": "08.07"
"value": 3,
"....": {
".....": {
},
}
"date": "08.07"
"value": 1,
"....": {
},
"date": "08.07"
"value": 3,
```



```

"...": {
".....": {
},
}
"date": ""
"value": "",
"...": {
".....": {
},
}
"date": "08.07"
"value": 3,
"...": {
".....": {
},
}
"date": ""
"value": "",
"...": {
"..... --3": {
},
}
"date": ""
"value": "",
"...": {
"... --2": {
"status": {
"sum": 12.0,
".....": {
},
}
}
"date": ""
"value": "",
".....": {
},
"date": "07.07"
"value": 3,
".....": {
},
"date": "07.07"
"value": 2,
".....": {
},
"date": "07.07"
"value": 2,
"..... --3": {
},
}
"date": "08.07"
"value": 1,
".....": {
"date": "07.07",
"value": 1,
"... --2": {
"status": {
"sum": 12.0,
".....": {
},
}
}
"date": ""
"value": "",
"status": {
"sum": 36532821.29,

```

```

def fmt_money(value):

]
".....", ".....", ".....", ".....", ".....", "....."
".....", ".....", ".....", ".....", ".....", ".....",
ALL_YEAR_MONTHS = [

import config
import datetime
import json
import os
=== FILE: ./db_core.py ===

}
}
"is_new_change": false
"last_changed_sub": "02_1505_...",
"last_changed_dir": "01_..... (.....)",
"_meta": {
},
}
}
}
"date": ""
"value": "",
"status": {
"sum": 3422671.63,
".....": {
"01_.....": {
"05_....._....._.....": {
},
}
}
}
"date": ""
"value": "",
"status": {
"sum": 0.0,
".....": {
"01_..... ..": {
"04_.....1": {
},
}
}
}
"date": "03.07"
"value": 3,
"status": {
"sum": 57197064.42,
".....": {
},
}
"date": ""
"value": "",
"status": {
"sum": 13.0,
".....": {
},
}
}
}
"date": ""
"value": "",

```

```

.....", "3": ". ...."]
colors = {1: ". ....", 2: ". ....", 3: ". ....", "": "...", "1": ". ....", "2": ".
return
print("=====")
print(".... .... database.json .....")
if not db:
db = db_core.load_db()
print("\n===== ..... =====")
"""..... . . . . ."""
def print_data():

import config
import db_core
=== FILE: ./db_viewer.py ===

print(f" [.....] ..... '{new_month}' ..... config.py")
f.write(code)
with open(config_path, "w", encoding="utf-8") as f:

code = re.sub(r"MONTHS_LIST\s*=\s*\[.*?\]", new_line_repr, code)
import re
else:
code = code.replace(old_line_repr, new_line_repr)
if old_line_repr in code:

new_line_repr = f"MONTHS_LIST = {repr(updated_months)}"
old_line_repr = f"MONTHS_LIST = {repr(config.MONTHS_LIST)}"
updated_months = [m for m in ALL_YEAR_MONTHS if m in config.MONTHS_LIST or m == new_month]

code = f.read()
with open(config_path, "r", encoding="utf-8") as f:
config_path = "config.py"
if new_month not in config.MONTHS_LIST:
"""..... . config.py . . . . ."""
def update_config_months(new_month):

json.dump(db, f, ensure_ascii=False, indent=4)
with open(json_path, "w", encoding="utf-8") as f:
json_path = "database.json"
def save_db(db):

return {}
return json.load(f)
with open(json_path, "r", encoding="utf-8") as f:
if os.path.exists(json_path):
json_path = "database.json"
def load_db():

return datetime.datetime.now().strftime("%d.%m")
"""....."""
def get_short_date():

return str(value)
except:
return f"{val_float:,.2f}".replace(",", " ")
return f"{int(val_float):,}".replace(",", " ")
if val_float.is_integer():
val_float = float(value)
return "0"
if value is None or value == "":
try:
"""....."""

```

```

if status_val == "" or status_val is None:

status_val = status_raw
else:
status_val = status_raw.get("value", "")
if isinstance(status_raw, dict):

from openpyxl.styles import Font
import config
"""....., ..... .....,....."""
def format_status_cell(cell, status_raw):

if alignment: cell.alignment = alignment
if border: cell.border = border
if fill: cell.fill = fill
if font: cell.font = font
cell = ws.cell(row=row_idx, column=col)
for col in range(1, ws.max_column + 1):
"""....."""
def apply_row_style(ws, row_idx, font, fill, border, alignment=None):

FONT_LOG_DATE = openpyxl.styles.Font(name="Calibri", size=9, italic=True, color="7A7A7A")
# ..... 5-... .....

import config
import openpyxl
=== FILE: ./excel_styler.py ===

print("\n=====")

if dt else '')")
print(f"      .. . {mth}: {fmt_sum} .... | ..... ..: {colors.get(val, '...')}{f' ({dt})'
dt = status_raw.get("date", "") if isinstance(status_raw, dict) else ""
val = status_raw.get("value", "") if isinstance(status_raw, dict) else status_raw
else:
print(f"      ... . {d_name} -> {colors.get(val, '...')}{f' ({dt})' if dt else ''")
dt = doc_status.get("date", "") if isinstance(doc_status, dict) else ""
val = doc_status.get("value", "") if isinstance(doc_status, dict) else ""
doc_status = status_raw.get(d_name, {})
if mask.get("....", 1) == 1:
mask = config.DOCUMENT_ROLES.get(d_name, {})
for d_name in config.DOCUMENTS_LIST:
print(f"      .. . {mth}: {fmt_sum} .... | ..... ..: [..... .. ..]")
if isinstance(status_raw, dict) and "value" not in status_raw:

status_raw = mth_data.get("status", "")
fmt_sum = db_core.fmt_money(mth_data.get("sum", 0))
# .....: ..... .....,.....
mth_data = db[direction][sub_obj][mth]
for mth in available_months:

available_months.append(m)
if m not in available_months:
for m in raw_months:

available_months = [m for m in db_core.ALL_YEAR_MONTHS if m in raw_months]
raw_months = list(db[direction][sub_obj].keys())

print(f"      .. . .....: {sub_obj}")
for sub_obj in db[direction].keys():
print(f"\n. ....: {direction}")
for direction in db.keys():

```

```

ws.column_dimensions['I'].width = 10
# ..... I (....)
ws.column_dimensions['H'].width = 7
ws.column_dimensions['F'].width = ws.column_dimensions['G'].width =
ws.column_dimensions['D'].width = ws.column_dimensions['E'].width =
ws.column_dimensions['C'].width = 8, 55, 16
ws.column_dimensions['A'].width, ws.column_dimensions['B'].width,
def set_column_widths(ws):

return current_row
sc.font, sc.alignment, sc.border = config.FONT_DATA, config.ALIGN_C, config.THIN_BORDER
sc = ws.cell(row=current_row, column=4, value=score)
ws.cell(row=current_row, column=2, value=text).font = config.FONT_DATA
current_row += 1
for score, text in specs:
(..... ....), (1, "..... (..... ....)")]
specs = [(3, "... .. (..... ....)"), (2, ". .... / .. .....
Font(name="Calibri", size=11, bold=True)
ws.cell(row=current_row, column=2, value=".....:").font =
current_row += 2
def append_specifications(ws, current_row):

fill=f_red, font=font_red))
ws.conditional_formatting.add("D2:I1000", CellIsRule(operator='equal', formula=['3'],
fill=f_ylw, font=font_ylw))
ws.conditional_formatting.add("D2:I1000", CellIsRule(operator='equal', formula=['2'],
fill=f_grn, font=font_grn))
ws.conditional_formatting.add("D2:I1000", CellIsRule(operator='equal', formula=['1'],
# ..... I (..... D, E, F, G, H, I)

Font(name="Calibri", size=11, color="9C6500"), Font(name="Calibri", size=11, color="9C0006")
font_grn, font_ylw, font_red = Font(name="Calibri", size=11, color="006100"),
f_red = config.PatternFill(start_color="FFC7CE", end_color="FFC7CE", fill_type="solid")
f_ylw = config.PatternFill(start_color="FFEB9C", end_color="FFEB9C", fill_type="solid")
f_grn = config.PatternFill(start_color="C6EFCE", end_color="C6EFCE", fill_type="solid")
def apply_conditional_formatting(ws):

import config
from openpyxl.formatting.rule import CellIsRule
from openpyxl.styles import Font
=== FILE: ./formatter.py ===

cell.fill = config.FILL_RED
elif status_str == "3":
cell.fill = config.FILL_YELLOW
elif status_str == "2":
cell.fill = config.FILL_GREEN
if status_str == "1":

cell.font = Font(name="Calibri", size=11, bold=True, color="000000")
# .....: .....

cell.alignment = config.ALIGN_C

cell.value = status_str
except ValueError:
cell.value = int(status_str)
try:

status_str = str(status_val).strip()

return

```

```

return False
print(" [.....] .....")
if result.returncode != 0:
result = subprocess.run(["python3", "main.py"])
print("\n [.....] ... 1: .....")
def run_production_pipeline(session_added_rows=None):

print(f" [.....] .. ..... changes.json: {e}")
except Exception as e:
print(" [.....] ..... changes.json .....")
json.dump({}, f)
with open(json_path, "w", encoding="utf-8") as f:
try:
if os.path.exists(json_path):
json_path = "changes.json"
def clear_changes_json():

return "....."

pass
except Exception:
return f".....: {' '.join(set(months_edited))}"
if months_edited:
months_edited.append(v["label"].strip())
elif isinstance(v, dict) and "label" in v:
months_edited.append(k.split("||")[-1])
if "||" in k:
for k, v in changes.items():
months_edited = []
if changes:
changes = json.load(f)
with open(json_path, "r", encoding="utf-8") as f:
try:
if os.path.exists(json_path) and os.path.getsize(json_path) > 2:
json_path = "changes.json"

return f".....: {' '.join(directions)} ({' '.join(months)})"
months = set([r[2] for r in session_added_rows])
directions = set([r[0] for r in session_added_rows])
if session_added_rows:
def generate_auto_changelog(session_added_rows=None):

return "v2.1"
except Exception:
return "v2.1"
return f"v{'.'.join(parts)}"
parts[-1] = str(int(parts[-1]) + 1)
parts = version[1:].split('.')
if version.startswith('v'):
version = res.stdout.strip()
text=True, check=True)
res = subprocess.run(["git", "describe", "--tags", "--abbrev=0"], capture_output=True,
try:
def get_current_git_version():

import config
import subprocess
import json
import os
=== FILE: ./git_deployer.py ===

ws.column_dimensions['J'].width = 40
# ..... J (.....)

```

```

pass
except:
changes = json.load(f)
with open(json_path, "r", encoding="utf-8") as f:
try:
if os.path.exists(json_path):

json_path = os.path.join(project_dir, "changes.json")
project_dir = "/Users/test/Desktop/cont_ks2_2"
changes = {}

".....", ".....", ".....", ".....", ".....", "....."]
valid_months = [".....", ".....", ".....", ".....", ".....", ".....",

sheet = model.CurrentController.ActiveSheet
model = event.Source
try:
"""
....., ..... changes.json.
..... LibreOffice Calc. ....
"""
def log_cell_change(event):

import os
import json
=== FILE: ./macros/cell_watcher.py ===

return True
clear_changes_json()
print(" [....] ..... .. Mac. ....")
else:
return False
print(" [..... GIT] .. .....")
else:
return True
clear_changes_json()
print(" [.....] ... .. GitHub!")
if push_res.returncode == 0:

push_res = subprocess.run(["git", "push", "origin", "main", "--tags"])
subprocess.run(["git", "tag", "-a", final_ver, "-m", final_logs])
subprocess.run(["git", "commit", "-m", commit_message])
subprocess.run(["git", "add", "."])

print(f"\n [GIT] .....: \"{commit_message}\"")
commit_message = f"{final_ver} · {final_logs}"

final_logs = user_logs if user_logs else default_logs
user_logs = input(f"..... [.. .....: {default_logs}]: ").strip()

final_ver = user_ver if user_ver else default_ver
user_ver = input(f"..... [.. .....: {default_ver}]: ").strip()
print("\n .....:")

default_logs = generate_auto_changelog(session_added_rows)
default_ver = get_current_git_version()
if push_choice in (".", "y", "yes"):

).strip().lower()
push_choice = input("..... GitHub? (../...):
print("\n [.....] ... 2: .....")

```

```

# 3. ....

ws.sheet_properties.outlinePr.summaryBelow = False
ws.sheet_view.showOutlineSymbols = True
ws.sheet_view.showGridLines = True
ws.title = "....."
ws = wb.active
wb = openpyxl.Workbook()
# 2. ....

mock_data = storage.group_linear_data(config.DATA_ROWS)
saved_statuses, saved_sums = storage.load_historical_data(file_path)
# 1. ....

file_path = config.FULL_PATH if config.FULL_PATH else "....._..._.xlsx"
print(".. [.....] .....")
def run():

import formatter
import builder
import storage
import config
import openpyxl
import os
=== FILE: ./main.py ===

g_exportedScripts = (log_cell_change,)

pass
except Exception:

json.dump(changes, f, ensure_ascii=False, indent=4)
with open(json_path, "w", encoding="utf-8") as f:

sub_obj = lbl
else:
}
"sum": float(val_cell) if val_cell else 0.0
"month": lbl,
"sub_obj": sub_obj,
"direction": direction,
changes[storage_key] = {
storage_key = f"{direction}|{sub_obj}|{lbl}"
if direction and sub_obj:
val_cell = sheet.getCellByPosition(2, r_idx).Value
if lbl in valid_months:

continue
lbl.startswith("....") or lbl == "..... / ..... / .....":
if (lbl.startswith(".") or lbl.startswith("...") or lbl.startswith(".....") or

continue
sub_obj = ""
direction = lbl
if lbl.startswith("01_") or lbl.startswith("02_"):

continue
if not lbl:
lbl = str(sheet.getCellByPosition(1, r_idx).String).strip()
for r_idx in range(1, 300):

sub_obj = ""
direction = ""

```



```

.....: .....
"""..... TXT->PDF ..... PyMuPDF (.....).
def text_to_pdf_from_fitz(txt_path: str, pdf_path: str):

return lines
lines.append(cur)
if cur:
cur = w
lines.append(cur)
else:
cur += " " + w
if len(cur) + 1 + len(w) <= max_chars:
continue
cur = w
if not cur:
for w in words:

cur = ""
lines = []
words = text.split(" ")
"""..... (.....)."""
def _wrap_text(text: str, max_chars: int) -> List[str]:

from typing import List
import os
=== FILE: ./pdf_from_text.py ===

run()
if __name__ == "__main__":

print(f". [.....] .....: {e}")
except Exception as e:
print(f". [.....] .... Excel/LibreOffice .....! .....")
except PermissionError:

.....!")
print(f". [.....] .... , . .....
else:
print(f". [.....] .....!")
print(f".. ..: {file_size_kb} ..")
print(f". ....: {os.path.abspath(file_path)}")
print(f". [.....] .....!")
file_size_kb = round(os.path.getsize(file_path) / 1024, 1)
if os.path.exists(file_path) and os.path.getsize(file_path) > 0:
# .....

wb.save(file_path)
try:
# 5. ....

formatter.set_column_widths(ws)
formatter.apply_conditional_formatting(ws)
formatter.append_specifications(ws, last_row)
last_row = builder.build_structure(ws, mock_data, saved_statuses, saved_sums)
# 4. ....

config.ALIGN_C)
builder.apply_row_style(ws, 1, config.FONT_HDR, config.FILL_HDR, config.THIN_BORDER,
ws.row_dimensions.height = 28
ws.append(config.HEADERS)

```

```

saved_statuses, saved_sums = {}, {}
def load_historical_data(file_path):

from config import MONTHS_LIST
import openpyxl
import os
=== FILE: ./storage.py ===

)
".. ..... (..... PyMuPDF 'fitz')."
".. ..... TXT . PDF. "+
raise RuntimeError(
if not ok:
ok = try_convert_text_to_pdf(txt_path, pdf_path)
def ensure_pdf(txt_path: str, pdf_path: str) -> None:

return False
except Exception:
return True
text_to_pdf_from_fitz(txt_path, pdf_path)
try:
"""
.. ..... False.

"""..... TXT . PDF.
def try_convert_text_to_pdf(txt_path: str, pdf_path: str) -> bool:

pdf.close()
pdf.save(pdf_path)

y -= line_height
page.insert_text((x, y), line, fontsize=9, fontname="courier")
y = 800
page = pdf.new_page(width=595, height=842)
if y - line_height < 30:
for line in lines:

line_height = 12
x, y = 30, 800

lines.extend(_wrap_text(paragraph, max_chars=max_chars))
else:
lines.append("")
if not paragraph.strip():
for paragraph in raw.splitlines():
lines = []
max_chars = 95
# .....: .....

raw = f.read()
with open(txt_path, "r", encoding="utf-8", errors="replace") as f:

page = pdf.new_page(width=595, height=842) # A4-ish
pdf = fitz.open()
# .....
doc = fitz.open()

import fitz # type: ignore
"""
PDF .. ..,

```

- [x] , 5 -> 4 (1-3)

.....

- [x] , `ai\_project\_snapshot.txt`

`.css/.html/.htm`, `.xml` (.... ..)

- [x] «.....»: `.py`, `.json`, `.txt`, `.md`, `.yaml/.yml`, `.js/.ts`,

(`.xlsx`, `.zip`, `.pdf`, `.odt`, `.png/.jpg`)

- [x] : `.git/`, `\_\_pycache\_\_/`, `node\_modules/` (.... ..),

- [x] `ai\_project\_snapshot.txt` (.....)

.... txt»

- [x] `wizard\_manager.py` : `5 -> 4` «.....

TODO txt

=== FILE: ./TODO.md ===

return mock_data

mock_data[direction][sub_obj][month] = [val, status]

if sub_obj not in mock_data[direction]: mock_data[direction][sub_obj] = {}

if direction not in mock_data: mock_data[direction] = {}

for direction, sub_obj, month, val, status in data_rows:

mock_data = {}

def group_linear_data(data_rows):

return saved_statuses, saved_sums

pass

except Exception:

old_wb.close()

if s_val is not None: saved_sums[(c_dir, c_sub, c_mth)] = s_val

s_val = old_ws.cell(row=row, column=3).value

if d_name == "... -2":

saved_statuses[(c_dir, c_sub, c_mth, d_name)] = (st_d, st_e, st_f, st_g)

if any(st in (1, 2, 3) for st in (st_d, st_e, st_f, st_g)):

..... 4

st_g = old_ws.cell(row=row, column=7).value

st_f = old_ws.cell(row=row, column=6).value

st_e = old_ws.cell(row=row, column=5).value

st_d = old_ws.cell(row=row, column=4).value

d_name = val_b.replace(".", "").strip()

elif val_b.startswith(". "):

elif val_b in MONTHS_LIST: c_mth = val_b

elif val_b.startswith("..... "): c_sub = val_b.replace("..... ", "").strip()

elif "02_....." in val_b: c_dir = "02_....."

if "01_....." in val_b: c_dir = "01_..... (.....)"

if not val_b: continue

val_b = str(old_ws.cell(row=row, column=2).value or "").strip()

for row in range(2, old_ws.max_row + 1):

c_dir, c_sub, c_mth = "", "", ""

old_ws = old_wb["....."]

return saved_statuses, saved_sums

old_wb.close()

if "....." not in old_wb.sheetnames:

old_wb = openpyxl.load_workbook(file_path, data_only=True)

try:

return saved_statuses, saved_sums

if not os.path.exists(file_path) or os.path.getsize(file_path) == 0:

.....

```

db["_meta"] = {
db[target_dir][target_sub][target_mth]["sum"] = cleaned_sum

continue
print(" . [.....] ..... !")
if cleaned_sum < 0:
cleaned_sum = float(new_sum_str.replace(" ", "").replace(",", "."))
try:
break
if new_sum_str.lower() in (".....", "0"):
new_sum_str = input("\n..... (.. '0' ..): ").strip()
while True:
if sub_choice == "1":

target_mth = months_in_db[m_idx]
return
if m_idx < 0 or m_idx >= len(months_in_db):
m_idx = int(m_choice) - 1
return
if not m_choice or not m_choice.isdigit():
m_choice = input("\n.....: ").strip()

print(f" {idx}. {m} (.....: {fmt_sum} .... | .....: {st_val})")
fmt_sum = db_core.fmt_money(db[target_dir][target_sub][m].get("sum", 0))
# .....: .....
)
)
else (st_raw if st_raw else "...")
if isinstance(st_raw, dict)
st_raw.get("value", "...")
else (
if isinstance(st_raw, dict) and "value" not in st_raw
"....."
st_val = (
st_raw = db[target_dir][target_sub][m].get("status", "")
for idx, m in enumerate(months_in_db, 1):
print("\n.....:")
months_in_db = list(db[target_dir][target_sub].keys())

return
if not target_dir or not target_sub:
target_dir, target_sub = select_target_func(db, allow_new=False)

return
if sub_choice not in ("1", "2", "3"):
sub_choice = input("..... (1-3): ").strip()
.....")
print(".....:\n 1. ....\n 2. ....\n 3.

return
if not db:
db = db_core.load_db()
print("\n>> .....")
""",....., ..... (..... 3)."""
def menu_edit_data(select_target_func):

import wizard_git
import config
import db_core
=== FILE: ./wizard_edits.py ===

```

```

else:
    status_key = "1 .... ."
    elif which == "4":
        status_key = "....."
    elif which == "3":
        status_key = "..."
    elif which == "2":
        status_key = "....."
    if which == "1":

    return
    print(" [.....] ..... .")
    if which not in ("1", "2", "3", "4", "5"):
        which = input("..... (1-5): ").strip()
    print("\n... ..? 1 - ..., 2 - ..., 3 - ....., 4 - 1 .... ., 5 - 1 .... .")
    elif sub_choice == "3":

    db_core.update_config_months(new_mth_name)
    wizard_git.register_action("branch_added")
    db_core.save_db(db)
}
"is_new_change": True
"last_changed_sub": target_sub,
"last_changed_dir": target_dir,
db["_meta"] = {
db[target_dir][target_sub][new_mth_name] = db[target_dir][target_sub].pop(target_mth)

    return
    wizard_git.register_action("sum_changed")
    db_core.save_db(db)
}
"is_new_change": True
"last_changed_sub": target_sub,
"last_changed_dir": target_dir,
db["_meta"] = {
db[target_dir][target_sub][new_mth_name]["sum"] += float(old.get("sum", 0))
old = db[target_dir][target_sub].pop(target_mth)
if input().strip() == "1":
    print(" . .... .! .....? (1-... / 2-....)")
    if new_mth_name in db[target_dir][target_sub]:

    new_mth_name = db_core.ALL_YEAR_MONTHS[int(new_mth_choice) - 1]

    return
    if not new_mth_choice or not new_mth_choice.isdigit():
        new_mth_choice = input("\n..... (1-12): ").strip()

    print(f" {i}. {m_name}")
    for i, m_name in enumerate(db_core.ALL_YEAR_MONTHS, 1):
        print("\n..... :")
    elif sub_choice == "2":

    print(" . [.....] ..... .!")
    except ValueError:
        break
    print(f" [.....] ..... : {db_core.fmt_money(cleaned_sum)} ....")
    # ..... : .....
    wizard_git.register_action("sum_changed")

    db_core.save_db(db)
}
"is_new_change": True
"last_changed_sub": target_sub,
"last_changed_dir": target_dir,

```

```

print(f" [.....] ..... {status_key} ..... ! (.....)
wizard_git.register_action("status_changed")
db_core.save_db(db)
}
"is_new_change": True
"last_changed_sub": target_sub,
"last_changed_dir": target_dir,
db["_meta"] = {

doc_entry[status_key] = {"value": status_value, "date": status_date}
# ... ..... doc_entry ..... status_key.
# .. ..... status_key (..... ..).
.....,
# .... .....: status[doc] = {value,date}, .. ..... value/date ... .....
else:
doc_entry[status_key] = {"value": status_value, "date": status_date}
if status_key in doc_entry and isinstance(doc_entry[status_key], dict):
# .... ..... status[doc][status_key]
# .....

db[target_dir][target_sub][target_mth].setdefault("status", {})[d_name] = doc_entry
doc_entry = {}
if not isinstance(doc_entry, dict):
doc_entry = db[target_dir][target_sub][target_mth].setdefault("status", {}).get(d_name)

continue
if mask.get(status_key, 1) != 1:
mask = config.DOCUMENT_ROLES.get(d_name, {})
for d_name in config.DOCUMENTS_LIST:

db[target_dir][target_sub][target_mth]["status"] = {}
if not isinstance(db[target_dir][target_sub][target_mth].get("status", {}), dict):
# ....., ... .. dict .. status

current_status = {}
if not isinstance(current_status, dict):
# .....: status[doc] = {value,date} ... status[doc][status_key] = {value,date}
else:
db[target_dir][target_sub][target_mth]["status"] = {"value": status_value, "date": status_date}
# .. .....
if isinstance(current_status, dict) and "value" in current_status:
# .....: ..... "status" = {value,date} .. ..

current_status = db[target_dir][target_sub][target_mth].get("status", {})
# .. .....
# .....: status = {<doc_name>: {<status_key>: {value,date}}
# ..... .., .. .....
if apply_mode == "1":

apply_mode = input("..... (1-2): ").strip()
print(" 2. ....")
print(" 1. ....")
print("\n... ..?")

status_date = "" if st_choice == "0" else db_core.get_short_date()
status_value = "" if st_choice == "0" else int(st_choice)

return
print(" [.....] .....")
if st_choice not in ("1", "2", "3", "0"):
st_choice = input("..... (0-3): ").strip()
print(f"\n..... {status_key}:\n 1. .... | 2. .... | 3. .... | 0. ....")

status_key = "1 .... ."

```

```

wizard_git.register_action("status_changed")
db_core.save_db(db)
}
"is_new_change": True
"last_changed_sub": target_sub,
"last_changed_dir": target_dir,
db["_meta"] = {

doc_entry[status_key] = {"value": status_value, "date": status_date}
# ..... , .. .

db[target_dir][target_sub][target_mth].setdefault("status", {})[target_doc] = doc_entry
doc_entry = {}
if not isinstance(doc_entry, dict):
doc_entry = db[target_dir][target_sub][target_mth].setdefault("status", {}).get(target_doc)
# ..... entry .....

}
}
"date": current_status.get("date", ""),
"value": current_status.get("value", ""),
status_key: {
db[target_dir][target_sub][target_mth]["status"][d_name] = {
for d_name in config.DOCUMENTS_LIST:
db[target_dir][target_sub][target_mth]["status"] = {}
# .....: .....
# ..... status[doc] = {status_key: {value,date}}.
# .....: status = {value,date} (.... ....) .
if "value" in current_status:

current_status = {}
if not isinstance(current_status, dict):
# ..... status[doc] .... status[doc][status_key].

current_status = db[target_dir][target_sub][target_mth].get("status", {})

target_doc = allowed_docs[doc_idx]

return
if doc_idx < 0 or doc_idx >= len(allowed_docs):
doc_idx = int(doc_choice) - 1

return
if not doc_choice or not doc_choice.isdigit():
doc_choice = input(f"..... (1-{len(allowed_docs)}): ").strip()

print(f" {idx}. {doc_name}")
for idx, doc_name in enumerate(allowed_docs, 1):
print(f"\n..... {status_key}:")

return
print(f" . [.....] .... {status_key}!")
if not allowed_docs:

allowed_docs.append(doc_name)
if mask.get(status_key, 1) == 1:
mask = config.DOCUMENT_ROLES.get(doc_name, {})
for doc_name in config.DOCUMENTS_LIST:
allowed_docs = []
elif apply_mode == "2":

")

```

```
print("\n=== ..... ==")

suggested_ver = calculate_next_version(current_ver)
current_ver = getattr(config, "VERSION", getattr(config, "CURRENT_VERSION", "v1.0.0"))
print("\n..... . GitHub...")
def run_git_sync():

f.writelines(lines)
with open("config.py", "w", encoding="utf-8") as f:

lines.append(f'\nVERSION = "{new_version}"\n')
if not version_replaced:

break
version_replaced = True
lines[idx] = f'{var_name} = "{new_version}"\n'
var_name = "VERSION" if line.strip().startswith("VERSION") else "CURRENT_VERSION"
if line.strip().startswith("VERSION") or line.strip().startswith("CURRENT_VERSION"):
for idx, line in enumerate(lines):
version_replaced = False

lines = f.readlines()
with open("config.py", "r", encoding="utf-8") as f:
return
if not os.path.exists("config.py"):
def update_config_version(new_version):

return f"v{major}.{minor}.{patch}"

patch += 1
elif session_flags["sum_changed"] or session_flags["status_changed"]:
patch = 0
minor += 1
if session_flags["branch_added"]:

major, minor, patch = parts

parts = [1, 0, 0]
except ValueError:
parts += [0] * (3 - len(parts))
if len(parts) < 3:
parts = [int(p) for p in ver_clean.split(".")]
try:
ver_clean = current_ver.strip().lower().replace("v", "")
def calculate_next_version(current_ver):

session_flags[action_type] = True
if action_type in session_flags:
def register_action(action_type):

}
"branch_added": False
"status_changed": False,
"sum_changed": False,
session_flags = {

import config
import sys
import os
=== FILE: ./wizard_git.py ===

print(f"[.....] ..... {status_key} ..... '{target_doc}' .....!")
```



```

return True

session_flags[key] = False
for key in session_flags:

print(". [GITHUB] ... ..!")
os.system("git push")
os.system(f'git commit -m "{commit_msg}"')
os.system("git add .")
print("\n. .... GitHub...")

update_config_version(final_version)
commit_msg = f"{final_version} {comment}".strip()
comment = input("\n.....: ").strip()

print(" . [.....] .....!")
else:
sys.exit(0)
print("\n.....!")
elif choice == "0":
return False
print("\n. ....")
elif choice == "9":
break
final_version = f"v{final_version}"
if not final_version.startswith("v"):
final_version = input("..... (....., v1.2.5): ").strip()
elif choice == "3":
break
final_version = current_ver
elif choice == "2":
break
final_version = suggested_ver
if choice == "1":

choice = input("\n.....: ").strip()

print(" 0. ....")
print(" 9. .... (....., .....)")
print(" 3. ....")
print(f" 2. .... ({current_ver})")
print(f" 1. .... ({suggested_ver})")
print("\n.....:")
while True:

print("=====")
print(f".....: .... ({current_ver})")
else:
print(f".....: {suggested_ver}")
if current_ver != suggested_ver:
print(f"\n.....: {current_ver}")

for a in actions: print(a)
else:
print(" ..... (..... Excel).")
if not actions:

if session_flags["status_changed"]: actions.append(".....")
if session_flags["sum_changed"]: actions.append(".....")
(...../...../.....)
if session_flags["branch_added"]: actions.append(".....")
actions = []

```

```

.xml",
.htm",
.html",
.css",
.ts",
.js",
.yaml",
.yml",
.md",
.txt",
.json",
.py",
code_exts = {

}
.dll",
.so",
.dylib",
.exe",
.bin",
.a",
.o",
.obj",
.gz",
.tar",
.rar",
.7z",
.avi",
.wav",
.mov",
.mp4",
.mp3",
.webp",
.gif",
.jpeg",
.jpg",
.png",
.odt",
.pdf",
.zip",
.xls",
.xlsx",
exclude_exts = {

}
"build",
"dist",
".mypy_cache",
"node_modules",
".blackbox",
"__pycache__",
".git",
exclude_dirs = {
def build_code_snapshot_txt(output_path: str):

import os
import wizard_git
import config
import builder
import db_viewer
import wizard_edits
import wizard_menus
=== FILE: ./wizard_manager.py ===

```

```

print(" 5. ....")
print(" 4. .... Excel (. .... GitHub)")
print(" 3. .... (..... / ..... / .....)")
print(" 2. .... (..... / ..... / .....)")
print(" 1. .... (.....)")
print("\n>>> ..... <<<")
while True:
def main_menu():

out.write("\n\n")
out.write(f"[SKIPPED: {e}]\n")
except Exception as e:
out.write("[SKIPPED: not UTF-8]\n")
except UnicodeDecodeError:
out.write("\n")
if not content.endswith("\n"):
out.write(content)
content = f.read()
with open(fp, "r", encoding="utf-8") as f:
try:
out.write(f"=== FILE: {rel} ===\n")
rel = os.path.relpath(fp, root_dir)
for fp in files_collected:

out.write("=====\n\n")
out.write(f"Files: {len(files_collected)}\n")
out.write(f"Generated: {__import__('datetime').datetime.now().isoformat()}\n")
out.write(f"Root: {root_dir}\n")
out.write("=== PROJECT CODE SNAPSHOT ===\n")
with open(output_path, "w", encoding="utf-8") as out:

files_collected.sort(key=lambda p: os.path.relpath(p, root_dir).lower())

continue
except Exception:
files_collected.append(fp)
if should_include_file(fp):
try:
continue
if os.path.abspath(fp) == output_path_abs:
fp = os.path.join(cur_root, fn)
for fn in files:

dirs[:] = [d for d in dirs if not should_skip_dir(d)]
for cur_root, dirs, files in os.walk(root_dir):

output_path_abs = os.path.abspath(output_path)
root_dir = os.path.abspath(os.getcwd())
files_collected = []

return ext in code_exts
return False
if ext == "":
return False
if ext in exclude_exts:
ext = ext.lower()
_, ext = os.path.splitext(file_path)
def should_include_file(file_path: str) -> bool:

return dir_name in exclude_dirs
def should_skip_dir(dir_name: str) -> bool:

}

```

```

)
f". [.....] TXT .. .....: {txt_path}. .... 5->4 (..... txt)."
print(
if not os.path.exists(txt_path):

pdf_path = os.path.join(os.getcwd(), pdf_name)
pdf_name = "ai_project_snapshot.pdf"
txt_path = os.path.join(os.getcwd(), txt_name)
txt_name = "ai_project_snapshot.txt"
elif srv_choice == "5":

input("..... Enter...")
print(f"\n. [.....] .....: {output_path}")
build_code_snapshot_txt(output_path)
output_path = os.path.join(os.getcwd(), output_name)
output_name = "ai_project_snapshot.txt"
if srv_choice == "4":

srv_choice = input(".....: ").strip()

print("5 - ..... ai_project_snapshot.txt . PDF")
print("4 - ..... txt (ai_project_snapshot.txt)")
print("3 - ..... builder.py")
print("2 - ..... DEBUG .....")
print("1 - ..... DEBUG .....")
print("\n[.....]")
elif choice == "5":

print(f". [.....] .....: {e}")
except Exception as e:
print(". [.....] .... Excel .....! .....")
except PermissionError:

wizard_git.run_git_sync()
print(f". [.....] .....: {file_path}")
wb.save(file_path)
try:
file_path = config.FULL_PATH if config.FULL_PATH else "....._..._.xlsx"

builder.build_structure(ws)
ws.sheet_view.showGridLines = True

ws.title = "....."
ws = wb.active
wb = openpyxl.Workbook()

import openpyxl
print("\n. .... Excel...")
elif choice == "4":
wizard_edits.menu_edit_data(wizard_menus.select_hierarchy_target)
elif choice == "3":
wizard_git.register_action("branch_added")
wizard_menus.menu_add_record()
elif choice == "2":
db_viewer.print_data()
if choice == "1":

continue
print(" [.....] .....")
if choice not in ("1", "2", "3", "4", "5", "0"):

choice = input("\n..... (0-4): ").strip()

print(" 0. ....")

```

```

d_idx = int(d_choice) - 1
if not d_choice or not d_choice.isdigit(): return None, None
d_choice = input(f"\n..... (1-{{limit}}): ").strip()
limit = len(directions) + 1 if allow_new else len(directions)

if allow_new: print(f" {{len(directions)} + 1}. [.....]")
for idx, d in enumerate(directions, 1): print(f" {{idx}}. {{d}}")
print("\n.....:")
if directions:

directions = list(db.keys())
target_dir = ""
""",....., """"
def select_hierarchy_target(db, allow_new=False):

import config
import db_core
=== FILE: ./wizard_menus.py ===

main_menu()
if __name__ == "__main__":

print(" [.....] .....")
else:
break
print("\n.....!")
elif choice == "0":

print(" [.....] .....")
else:
print(" [.] ..... DEBUG .....")
logging.getLogger().setLevel(logging.DEBUG)
elif srv_choice == "2":
print(" [.] ..... DEBUG .....")
logging.getLogger().setLevel(logging.WARNING)
if srv_choice == "1":

import logging
else:

print(" [.....] . builder.py .. ..... test_columns_width_logic().")
else:
builder.test_columns_width_logic()
if hasattr(builder, "test_columns_width_logic"):

print("..... (D, E, F, G, H)...")
print(".....: 1.35 . -> ..... . builder.py/openpyxl: 14.0")
print("\n[.....]")
elif srv_choice == "3":

input("..... Enter...")
print(f". [.....] .. ..... PDF: {{e}}")
except Exception as e:
print(f"\n. [.....] PDF .....: {{pdf_path}}")
pdf_from_text.ensure_pdf(txt_path, pdf_path)

import pdf_from_text
try:
else:
input("..... Enter...")

```

```

continue
print(" . [.....] ..... .. 1 .. 12!")
if mth_idx < 0 or mth_idx >= 12:
mth_idx = int(mth_choice) - 1

continue
print(" . [.....] ..... .. 1 .. 12!")
if not mth_choice or not mth_choice.isdigit():

return
if mth_choice == "0":
mth_choice = input("\n..... (1-12) ... '0' ... : ").strip()

print(f" {i}. {m_name}{marker}")
marker = " [... ..]" if m_name in active_months else ""
for i, m_name in enumerate(db_core.ALL_YEAR_MONTHS, 1):

active_months = list(db[target_dir][target_sub].keys())
if isinstance(db[target_dir][target_sub], dict):
if target_dir in db and target_sub in db[target_dir]:
active_months = []

print("\n..... :")
while True:

if not target_dir or not target_sub: return
target_dir, target_sub = select_hierarchy_target(db, allow_new=True)

db = db_core.load_db()
print("\n>> ..... ")
def menu_add_record():

return target_dir, target_sub

else: return None, None
if allow_new: target_sub = input("..... : ").strip()
else:
else: return None, None
target_sub = input("..... : ").strip()
elif allow_new and s_idx == len(sub_objs):
if 0 <= s_idx < len(sub_objs): target_sub = sub_objs[s_idx]

s_idx = int(s_choice) - 1
if not s_choice or not s_choice.isdigit(): return None, None
s_choice = input(f"..... (1-{{limit}}): ").strip()
limit = len(sub_objs) + 1 if allow_new else len(sub_objs)

if allow_new: print(f" {{len(sub_objs) + 1}}. [.....]")
for idx, s in enumerate(sub_objs, 1): print(f" {idx}. {s}")
print(f"\n..... '{target_dir}':")
sub_objs = list(db[target_dir].keys())
if target_dir in db and db[target_dir]:

target_sub = ""
if not target_dir: return None, None

else: return None, None
if allow_new: target_dir = input(".... : ").strip()
else:
else: return None, None
target_dir = input("..... : ").strip()
elif allow_new and d_idx == len(directions):
if 0 <= d_idx < len(directions): target_dir = directions[d_idx]

```

```

elif choice == "4":
wizard_menus.menu_edit_data()
elif choice == "3":
wizard_menus.menu_add_record()
elif choice == "2":
print_data()
if choice == "1":

choice = input("..... (1-5): ").strip()
print("5. ....")
print("4. .... Excel (. .... GitHub)")
print("3. ....")
print("2. .... (...../.....)")
print("1. .... (.....)")
print("\n--- ..... (JSON ....) ---")
while True:
def run_wizard():

print("=====")
{data.get('status', '...')}")
print(f" .. {mth}: {db_core.fmt_money(data.get('sum', 0))} .... | ..... :
for mth, data in months.items():
print(f" .. .....: {sub_obj}")
for sub_obj, months in sub_objs.items():
print(f"\n .....: {direction}")
for direction, sub_objs in db.items():
return
print("....")
if not db:
print("\n=== ..... ===")
db = db_core.load_db()
def print_data():

import git_deployer
import wizard_menus
import db_core
-----
### .....: wizard_manager.py ###

=====
..... (..... JSON)
=====
=== FILE: ./....._....._......txt ===

print(f" [.....] ..... '{month}' ..... !")
db_core.update_config_months(month)
db_core.save_db(db)
db[target_dir][target_sub][month] = {"sum": 0.0, "status": {"value": "", "date": ""}}

if target_sub not in db[target_dir]: db[target_dir][target_sub] = {}
if target_dir not in db: db[target_dir] = {}

break

return
input("\n..... Enter...")
print(" ..... -> ..... 3 .....")
print(f"\n . [.....] ..... '{month}' ..... !")
if target_dir in db and target_sub in db[target_dir] and month in db[target_dir][target_sub]:

month = db_core.ALL_YEAR_MONTHS[mth_idx]

```

```

db = db_core.load_db()
print("\n>> .....")
"""..... (..... 2)."""
def menu_add_record():

return target_dir, target_sub

target_sub = input(".....: ").strip()
else:
target_sub = input(".....: ").strip()
elif allow_new and s_idx == len(sub_objs):
if 0 <= s_idx < len(sub_objs): target_sub = sub_objs[s_idx]

s_idx = int(s_choice) - 1
if not s_choice or not s_choice.isdigit(): return None, None
s_choice = input(f"..... (1-{{limit}}): ").strip()
limit = len(sub_objs) + 1 if allow_new else len(sub_objs)

if allow_new: print(f" {len(sub_objs) + 1}. [.....]")
for idx, s in enumerate(sub_objs, 1): print(f" {idx}. {s}")
print(f"\n..... '{target_dir}':")
sub_objs = list(db[target_dir].keys())
if target_dir in db and db[target_dir]:
target_sub = ""

if not target_dir: return None, None

target_dir = input(".... ..: ").strip()
else:
target_dir = input(".....: ").strip()
elif allow_new and d_idx == len(directions):
if 0 <= d_idx < len(directions): target_dir = directions[d_idx]

d_idx = int(d_choice) - 1
if not d_choice or not d_choice.isdigit(): return None, None
d_choice = input(f"\n..... (1-{{limit}}): ").strip()
limit = len(directions) + 1 if allow_new else len(directions)

if allow_new: print(f" {len(directions) + 1}. [.....]")
for idx, d in enumerate(directions, 1): print(f" {idx}. {d}")
print("\n.....:")
if directions:

directions = list(db.keys())
target_dir = ""
""".....,."""
def select_hierarchy_target(db, allow_new=False):

import db_core
-----
### ....: wizard_menus.py ###

-----

run_wizard()
if __name__ == "__main__":

print(" .....")
else:
print(".. .....!"); break
elif choice == "5":
git_deployer.run_production_pipeline(session_added_rows=["....."])

```



```

print("\n..... :")
elif sub_choice == "2":

except: print(" [.....] ..... :")
print(" [.....] ..... :")
db_core.save_db(db)
db[target_dir][target_sub][target_mth]["sum"] = float(new_sum_str.replace(",", "."))
try:
new_sum_str = input(f"\n..... : ").strip()
if sub_choice == "1":

target_mth = months_in_db[m_idx]
if m_idx < 0 or m_idx >= len(months_in_db): return
m_idx = int(m_choice) - 1
if not m_choice or not m_choice.isdigit(): return
m_choice = input("\n..... : ").strip()

.... | .....: {st_val})")
print(f" {idx}. {m} (.....: {db_core.fmt_money(db[target_dir][target_sub][m].get('sum', 0))}
"...")
st_val = st_raw.get("value", "...") if isinstance(st_raw, dict) else (st_raw if st_raw else
st_raw = db[target_dir][target_sub][m].get("status", "")
for idx, m in enumerate(months_in_db, 1):
print(f"\n..... :")
months_in_db = list(db[target_dir][target_sub].keys())

if not target_dir or not target_sub: return
target_dir, target_sub = select_hierarchy_target(db, allow_new=False)

if sub_choice not in ("1", "2", "3"): return
sub_choice = input("..... (1-3): ").strip()
..... :")
print("..... :\n 1. ....\n 2. ....\n 3.

if not db: return
db = db_core.load_db()
print("\n>> ..... :")
""",....., ..... (..... 3).""
def menu_edit_data():

print(f" [.....] ..... '{month}' .....!")
db_core.update_config_months(month)
db_core.save_db(db)
db[target_dir][target_sub][month] = {"sum": 0.0, "status": {"value": "", "date": ""}}

if month in db[target_dir][target_sub]: return
if target_sub not in db[target_dir]: db[target_dir][target_sub] = {}
if target_dir not in db: db[target_dir] = {}

month = db_core.ALL_YEAR_MONTHS[mth_idx]
if mth_idx < 0 or mth_idx >= 12: return
mth_idx = int(mth_choice) - 1
if not mth_choice or not mth_choice.isdigit(): return
mth_choice = input("\n..... (1-12): ").strip()

print(f" {i}. {m_name}{marker}")
marker = " [·····]" if m_name in active_months else ""
for i, m_name in enumerate(db_core.ALL_YEAR_MONTHS, 1):
db[target_dir]) else []
active_months = list(db[target_dir][target_sub].keys()) if (target_dir in db and target_sub in
print("\n..... :")

if not target_dir or not target_sub: return
target_dir, target_sub = select_hierarchy_target(db, allow_new=True)

```

```

json_path = "database.json"
def load_db():

return datetime.datetime.now().strftime("%d.%m")
"""....."""
def get_short_date():

return str(value)
except:
return f"{thousand_part},{parts}"
thousand_part = f"{int(parts):_}".replace("_", " ")
parts = f"{val_float:.2f}".split(".")
else:
return f"{int(val_float):_}".replace("_", " ")
if val_float.is_integer():
val_float = float(value)
try:
"""....."""
def fmt_money(value):

]
".....", ".....", ".....", ".....", ".....", "....."
".....", ".....", ".....", ".....", ".....", ".....",
ALL_YEAR_MONTHS = [

import config
import datetime
import json
import os
-----
### ....: db_core.py ###

-----

print(" [.....] .....")
db_core.save_db(db)
db[target_dir][target_sub][target_mth]["status"] = {"value": "", "date": ""}
elif st_choice == "0":
print(" [.....] .....")
db_core.save_db(db)
db_core.get_short_date()}
db[target_dir][target_sub][target_mth]["status"] = {"value": int(st_choice), "date":
if st_choice in ("1", "2", "3"):
st_choice = input("..... (0-3): ").strip()
print("\n..... : \n 1. .... | 2. .... | 3. .... | 0. ....")
elif sub_choice == "3":

db_core.update_config_months(new_mth_name)
db_core.save_db(db)
db[target_dir][target_sub][new_mth_name] = db[target_dir][target_sub].pop(target_mth)

return
db_core.save_db(db)
db[target_dir][target_sub][new_mth_name]["sum"] += float(old.get("sum", 0))
old = db[target_dir][target_sub].pop(target_mth)
if input().strip() == "1":
print(" .. ..! .....? (1-... / 2-....)")
if new_mth_name in db[target_dir][target_sub]:

new_mth_name = db_core.ALL_YEAR_MONTHS[int(new_mth_choice) - 1]
if not new_mth_choice or not new_mth_choice.isdigit(): return
new_mth_choice = input("\n..... (1-12): ").strip()
for i, m_name in enumerate(db_core.ALL_YEAR_MONTHS, 1): print(f" {i}. {m_name}")

```

```

if not os.path.exists(json_path): return ws.max_row
json_path = "database.json"

config.ALIGN_C)
excel_styler.apply_row_style(ws, 1, config.FONT_HDR, config.FILL_HDR, config.THIN_BORDER,
ws.append(config.HEADERS)
if ws.max_row == 1 and ws.cell(row=1, column=1).value is None:

ws.oddHeader.right.color = "7A7A7A"
ws.oddHeader.right.size = 9
ws.oddHeader.right.text = f".....: {now_str}"
now_str = datetime.datetime.now().strftime("%d.%m.%Y %H:%M")

ws.sheet_properties.outlinePr.summaryBelow = False
ws.freeze_panes = "A2"
"""..... 5-... .."""
def build_structure(ws, mock_data=None, saved_statuses=None, saved_sums=None):

excel_styler.apply_row_style(ws, row_idx, font, fill, border, alignment)
"""..... .. main.py"""
def apply_row_style(ws, row_idx, font, fill, border, alignment=None):

import excel_styler
import config
import datetime
import os
import json
import openpyxl
-----
### .....: builder.py ###

-----

print(f" [.....] ..... '{new_month}' ..... .. config.py")
f.write(code)
with open(config_path, "w", encoding="utf-8") as f:

code = re.sub(r"MONTHS_LIST\s*=\s*\[.*?\]", new_line_repr, code)
import re
else:
code = code.replace(old_line_repr, new_line_repr)
if old_line_repr in code:

new_line_repr = f"MONTHS_LIST = {repr(updated_months)}"
old_line_repr = f"MONTHS_LIST = {repr(config.MONTHS_LIST)}"
updated_months = [m for m in ALL_YEAR_MONTHS if m in config.MONTHS_LIST or m == new_month]

code = f.read()
with open(config_path, "r", encoding="utf-8") as f:
config_path = "config.py"
if new_month not in config.MONTHS_LIST:
"""..... config.py .."""
def update_config_months(new_month):

json.dump(db, f, ensure_ascii=False, indent=4)
with open(json_path, "w", encoding="utf-8") as f:
json_path = "database.json"
def save_db(db):

return {}
return json.load(f)
with open(json_path, "r", encoding="utf-8") as f:
if os.path.exists(json_path):

```

```

excel_styler.apply_row_style(ws, log_row, excel_styler.FONT_LOG_DATE, None, config.THIN_BORDER,
# .....
log_row = ws.max_row
ws.append(["", f" .. .: {status_date}", "", "", "", "", "", "", "", ""])
if mask.get("....", 1) == 1 and status_date:
# ..... 5-: .....
excel_styler.format_status_cell(cell, status_raw)
if col_name == "....":
else:
cell.value = ""
cell.fill = config.FILL_BLOCKED
if mask.get(col_name, 1) == 0:
cell = ws.cell(row=doc_row, column=col_idx)
for col_name, col_idx in column_mapping.items():
column_mapping = {"....": 4, "...": 5, ".....": 6, "1 .... .": 7, "1 .... .": 8, "....": 9}
.... .": 1, "....": 1})
mask = config.DOCUMENT_ROLES.get(d_name, {"....": 1, "...": 1, ".....": 1, "1 .... .": 1, "1
ws.row_dimensions[doc_row].outline_level = 3
ws.cell(row=doc_row, column=3).number_format = '#,##0.00'
config.ALIGN_L)
excel_styler.apply_row_style(ws, doc_row, config.FONT_DATA, None, config.THIN_BORDER,
doc_row = ws.max_row
ws.append(["", f".{d_name}", "", "", "", "", "", "", "", ""])
for d_name in config.DOCUMENTS_LIST:
# .....: ..... 3
row_counter += 1
ws.row_dimensions[current_row].outline_level = 2
ws.cell(row=current_row, column=3).number_format = '#,##0.00'
ws.cell(row=current_row, column=1).alignment = config.ALIGN_C
config.THIN_BORDER, config.ALIGN_L)
excel_styler.apply_row_style(ws, current_row, config.FONT_MTH, config.FILL_MTH,
current_row = ws.max_row
ws.append([row_counter, mth, float(mth_data.get("sum", 0.0)), "", "", "", "", "", "", ""])
# .....: ..... 2
status_date = status_raw.get("date", "") if isinstance(status_raw, dict) else ""
status_raw = mth_data.get("status", "")
# .....
mth_data = db[direction][sub_obj][mth]
if mth not in db[direction][sub_obj]: continue
for mth in config.MONTHS_LIST:
ws.row_dimensions[ws.max_row].outline_level = 1
config.THIN_BORDER, config.ALIGN_L)
excel_styler.apply_row_style(ws, ws.max_row, config.FONT_OBJ, config.FILL_OBJ,
ws.append(["", str(sub_obj), "", "", "", "", "", "", "", ""])
for sub_obj in db[direction].keys():
ws.row_dimensions[ws.max_row].outline_level = 0
config.THIN_BORDER, config.ALIGN_L)
excel_styler.apply_row_style(ws, ws.max_row, config.FONT_DIR, config.FILL_DIR,
ws.append(["", str(direction), "", "", "", "", "", "", "", ""])
for direction in db.keys():
row_counter = 1
db = json.load(f)
with open(json_path, "r", encoding="utf-8") as f:

```

```

('01_..... (.....)', '02_1505_...', '.....', 450000, 3),
('01_..... (.....)', '01_271_..', '.....', 0, None),
('01_..... (.....)', '01_271_..', '.....', 450000, 3),
DATA_ROWS = [

from openpyxl.styles import Font, Alignment, PatternFill, Border, Side
import os
-----
### ....: config.py ###

-----

return cell.value
cell.alignment = config.ALIGN_C
cell.font = config.FONT_DATA

cell.value = str(status_val)
except ValueError:
cell.value = int(status_val)
# .....: ..... int, ..... Excel
try:

return ""
if status_val == "" or status_val is None:

status_val = status_raw
else:
status_val = status_raw.get("value", "")
if isinstance(status_raw, dict):

status_val = ""
""....., ""
def format_status_cell(cell, status_raw):

if alignment: cell.alignment = alignment
if border: cell.border = border
if fill: cell.fill = fill
if font: cell.font = font
cell = ws.cell(row=row_idx, column=col)
for col in range(1, ws.max_column + 1):
"".....
def apply_row_style(ws, row_idx, font, fill, border, alignment=None):

FONT_LOG_DATE = openpyxl.styles.Font(name="Calibri", size=9, italic=True, color="7A7A7A")
# ..... 5-.....

import config
import openpyxl
-----
### ....: excel_styler.py ###

-----

return ws.max_row

ws.cell(row=row, column=10).value = f"=IF(C{row}>0, \".. \", \"\")"
if ws.cell(row=row, column=2).value in config.MONTHS_LIST:
for row in range(2, ws.max_row + 1):

ws.row_dimensions[log_row].outline_level = 4
# ..... 4 ( Excel ..... 5-.....)
config.ALIGN_L)

```

```

import subprocess
import json
import os

-----

### .....: git_deployer.py ###

-----

FILL_BLOCKED = PatternFill(start_color="EAEAEA", end_color="EAEAEA", fill_type="solid")
# .....

}
".....": ROLE_EXEC_DOC
"....": ROLE_INVOICE,
".....": ROLE_INVOICE,
"..... -3": ROLE_KS3,
"... -2": ROLE_KS2,
DOCUMENT_ROLES = {

ROLE_EXEC_DOC = {".....": 1, "...": 0, ".....": 0, "1 .... .": 1, "1 .... .": 1, ".....": 0}
ROLE_INVOICE = {".....": 0, "...": 1, ".....": 0, "1 .... .": 1, "1 .... .": 1, ".....": 0}
ROLE_KS3 = {".....": 0, "...": 1, ".....": 1, "1 .... .": 1, "1 .... .": 1, ".....": 0}
ROLE_KS2 = {".....": 1, "...": 1, ".....": 1, "1 .... .": 1, "1 .... .": 1, ".....": 0}
# 1 - ..... (.....), 0 - ..... (.....)
# == ..... (.....) ==

".....", "1 .... .", "1 .... .", ".....", "..... .."]
HEADERS = ["./.", "..... / ..... / .....", "..... (....)", ".....", "....",

ALIGN_R = Alignment(horizontal="right", vertical="center")
ALIGN_L = Alignment(horizontal="left", vertical="center", wrap_text=True)
ALIGN_C = Alignment(horizontal="center", vertical="center", wrap_text=True)

color="B0B0B0"))
color="B0B0B0"), top=Side(style="thin", color="B0B0B0"), bottom=Side(style="thin",
THIN_BORDER = Border(left=Side(style="thin", color="B0B0B0"), right=Side(style="thin",

FILL_MTH = PatternFill(start_color="EAEAEA", end_color="EAEAEA", fill_type="solid")
FILL_OBJ = PatternFill(start_color="D9D9D9", end_color="D9D9D9", fill_type="solid")
FILL_HDR = FILL_DIR = PatternFill(start_color="1F4E78", end_color="1F4E78", fill_type="solid")

FONT_DATA = Font(name="Calibri", size=11)
FONT_MTH = Font(name="Calibri", size=11, bold=True, italic=True, color="000000")
FONT_OBJ = Font(name="Calibri", size=11, bold=True, color="000000")
FONT_DIR = Font(name="Calibri", size=12, bold=True, color="FFFFFF")
FONT_HDR = Font(name="Calibri", size=11, bold=True, color="FFFFFF")

FULL_PATH = os.path.join(FOLDER_NAME, FILE_NAME)
FILE_NAME = "....._..._.xlsx"
FOLDER_NAME = ""

.....']
DOCUMENTS_LIST = ['... -2', '..... -3', '.....', '.....', '.....']
MONTHS_LIST = ['.....', '.....', '.....', '.....']

]
('02_.....', '.....', '.....', 0, None),
('02_.....', '.....', '.....', 850000, 2),
('01_..... (.....)', '04_537_', '.....', 0, None),
('01_..... (.....)', '04_537_', '.....', 0, None),
('01_..... (.....)', '03_538_', '.....', 0, None),
('01_..... (.....)', '03_538_', '.....', 0, None),
('01_..... (.....)', '02_1505_...', '.....', 0, None),

```

```

default_logs = generate_auto_changelog(session_added_rows)
default_ver = get_current_git_version()
if push_choice in ("..", "y", "yes"):

    ").strip().lower()
    push_choice = input("..... GitHub? (../..):")
    print("\n [.....] ... 2: .....")

    return False
    print(" [.....] .....")
    if result.returncode != 0:
        result = subprocess.run(["python3", "main.py"])
        print("\n [.....] ... 1: .....")
    def run_production_pipeline(session_added_rows=None):

        print(f" [.....] .. ..... changes.json: {e}")
        except Exception as e:
            print(" [.....] ..... changes.json .....")
            json.dump({}, f)
            with open(json_path, "w", encoding="utf-8") as f:
                try:
                    if os.path.exists(json_path):
                        json_path = "changes.json"
                def clear_changes_json():

            return "....."

        pass
        except Exception:
            return f".....: {' ', '.join(set(months_edited))}"
            if months_edited:
                months_edited.append(v["label"].strip())
            elif isinstance(v, dict) and "label" in v:
                months_edited.append(k.split("||")[-1])
            if "||" in k:
                for k, v in changes.items():
                    months_edited = []
                if changes:
                    changes = json.load(f)
                    with open(json_path, "r", encoding="utf-8") as f:
                        try:
                            if os.path.exists(json_path) and os.path.getsize(json_path) > 2:
                                json_path = "changes.json"

            return f".....: {' ', '.join(directions)} ({' ', '.join(months)})"
            months = set([r[2] for r in session_added_rows])
            directions = set([r[0] for r in session_added_rows])
            if session_added_rows:
                def generate_auto_changelog(session_added_rows=None):

                    return "v2.1"
                except Exception:
                    return "v2.1"
                return f"v{' '.join(parts)}"
                parts[-1] = str(int(parts[-1]) + 1)
                parts = version[1:].split('.')
                if version.startswith('v'):
                    version = res.stdout.strip()
                    text=True, check=True)
                res = subprocess.run(["git", "describe", "--tags", "--abbrev=0"], capture_output=True,
                    try:
                        def get_current_git_version():

import config

```

```

last_row = builder.build_structure(ws, mock_data, saved_statuses, saved_sums)
# 4. ....

config.ALIGN_C)
builder.apply_row_style(ws, 1, config.FONT_HDR, config.FILL_HDR, config.THIN_BORDER,
ws.row_dimensions.height = 28
ws.append(config.HEADERS)
# 3. ....

ws.sheet_properties.outlinePr.summaryBelow = False
ws.sheet_view.showOutlineSymbols = True
ws.sheet_view.showGridLines = True
ws.title = "....."
ws = wb.active
wb = openpyxl.Workbook()
# 2. ....

mock_data = storage.group_linear_data(config.DATA_ROWS)
saved_statuses, saved_sums = storage.load_historical_data(file_path)
# 1. ....

file_path = config.FULL_PATH if config.FULL_PATH else "....._..._.xlsx"
print(".. [.....] .....")
def run():

import formatter
import builder
import storage
import config
import openpyxl
import os

-----
### ....: main.py ###

-----

return True
clear_changes_json()
print(" [....] ..... .. Mac. ....")
else:
return False
print(" [..... GIT] .. .....")
else:
return True
clear_changes_json()
print(" [.....] ... .. .. GitHub!")
if push_res.returncode == 0:

push_res = subprocess.run(["git", "push", "origin", "main", "--tags"])
subprocess.run(["git", "tag", "-a", final_ver, "-m", final_logs])
subprocess.run(["git", "commit", "-m", commit_message])
subprocess.run(["git", "add", "."])

print(f"\n [GIT] ..... : \"{commit_message}\"")
commit_message = f"{final_ver} . {final_logs}"

final_logs = user_logs if user_logs else default_logs
user_logs = input(f"..... [.. .....: {default_logs}]: ").strip()

final_ver = user_ver if user_ver else default_ver
user_ver = input(f"..... [.. .....: {default_ver}]: ").strip()
print("\n .....: ")

```



```

"sum": 0,
"....": {
},
"status": ""
"sum": 0,
".....": {
"03_538_..": {
},
}
"status": ""
"sum": 0,
"....": {
},
"status": 3
"sum": 450000,
".....": {
"02_1505_..": {
},
}
"status": ""
"sum": 0.0,
"....": {
},
}
"date": "01.07"
"value": 2,
"status": {
"sum": 17730436.92,
"....": {
"01_271_..": {
"01_..... (.....)": {
{
-----
### .....: database.json ###

-----

run()
if __name__ == "__main__":

print(f". [.....] .....: {e}")
except Exception as e:
print(f". [.....] .... Excel/LibreOffice .....! .....")
except PermissionError:

.....!")
print(f". [.....] .... .., . ..... ..")
else:
print(f". [.....] ..... ..!")
print(f".. ..: {file_size_kb} ..")
print(f". ....: {os.path.abspath(file_path)}")
print(f". [.....] ..... ..!")
file_size_kb = round(os.path.getsize(file_path) / 1024, 1)
if os.path.exists(file_path) and os.path.getsize(file_path) > 0:
# .....

wb.save(file_path)
try:
# 5. ....

formatter.set_column_widths(ws)
formatter.apply_conditional_formatting(ws)
formatter.append_specifications(ws, last_row)

```

```

.          ... 3: ..... (.....: ... --2, ..... --3, ...,
.          ... 2: ..... (....., .....-----)
.          ... 1: ..... (.....)
.          ... 0: ..... (.....)
-          B: .....:
.          .....
-          A: ..... , .....
.          ..... (Outline Level) .. 0 .. 4:
.          Excel-..... .. JSON. ....
3. .... EXCEL (5 ..... OUTLINE)

```

```

... status (dict: {"value": 3, "date": "01.07"})
... sum (float: ....., .....)
... .... (....., ".....")
... ..... (....., "01_271_..")
... ..... (....., "01_..... (.....)")
... ..... (.....):
... ..... Excel-..... .. `database.json`.
... ..... ".....". ....
2. .... (Single Source of Truth)

```

```

- .....: LibreOffice Calc / MS Excel (macOS)
- ...../:.....: ..... GitHub
- .....: ..... JSON
- .....: openpyxl (..... Excel)
- .....: Python 3
....., ..... (---2, ---3).
.....
1. ....

```

```

=====
.....
=====
=== FILE: ./..... _......txt ===

```

```

-----
}
}
}
}
"status": ""
"sum": 0,
".....": {
},
"status": 2
"sum": 850000,
".....": {
".....": {
"02_.....": {
},
}
}
"status": ""
"sum": 0,
".....": {
},
"status": ""
"sum": 0,
".....": {
"04_537_..": {
},
}
"status": ""

```

=====
.....,!
=====

`.xlsx` `\_\_pycache\_\_`.
- `.gitignore`, GitHub
..... 5-
- `TextBlock` . `append` `RichText`
.....,
- ,
- (.....).
..... macOS . openpyxl:
6. MAC

- `config.py` ,
- `git\_deployer.py` GitHub
.....,
- `excel\_styler.py` ,
.....,
..... (`freeze\_panes`)
- `builder.py` Excel. JSON ,
..... ,
- `db\_core.py` /..... JSON,
.....)
..... (..... 1-12,
- `wizard\_menus.py` ,
..... (..... 1-5).
- `wizard\_manager.py` ,
.....;
.....
5.

..... LibreOffice Calc.
..... (1, 2, 3),
- (`#EAEAEA`).
-
.....),:
..... (..... ,
..... (.....).
..... , . `config.py`
4. (.....)

01.07")
..... 4: (..... : ".....
..... , ..)